keywords
FYP

# MERCY:

# Monocular Event-boosted Locally Rigid Motion and Coherent 4D Gaussian Reconstruction

SPEAKER:

Liang Yirui 2330034030

Chen Qianqi 2330034001

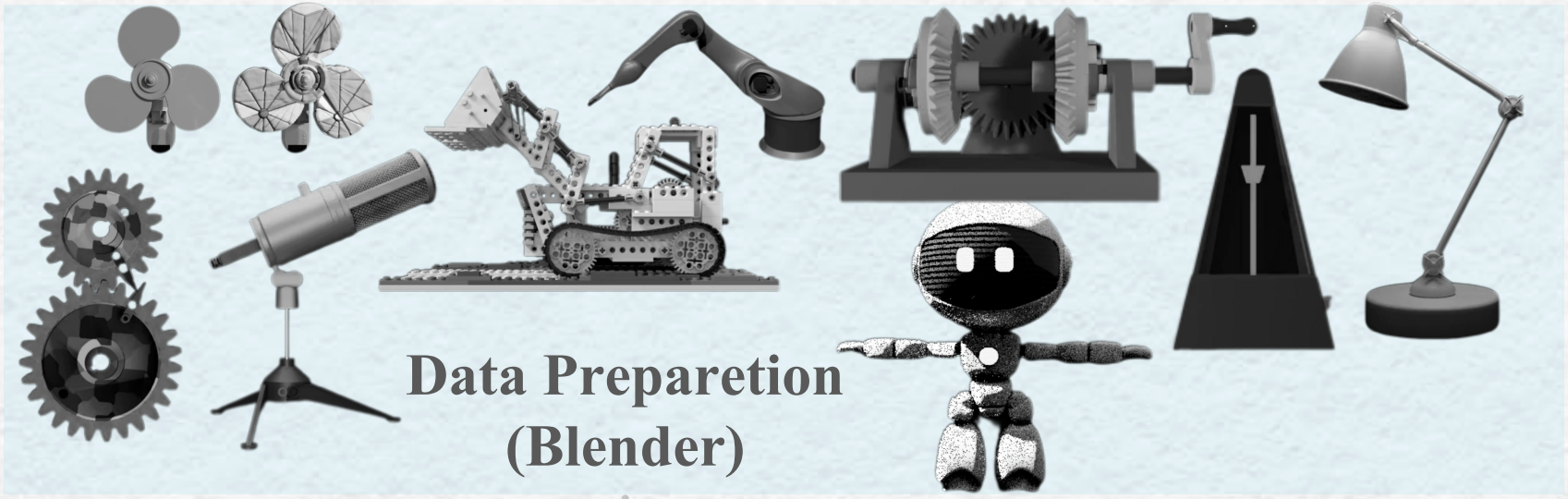
Li Minghao 2330034027

DATE: 2026.11.11

- **Achievement:**
  1.
  2.

# Timeline

1. IncEventGS
2. E-4DGS
3. E-D3DGS
.....




**April** — Literature Review — **June** — **July** >> Coding >> **September**

Fan

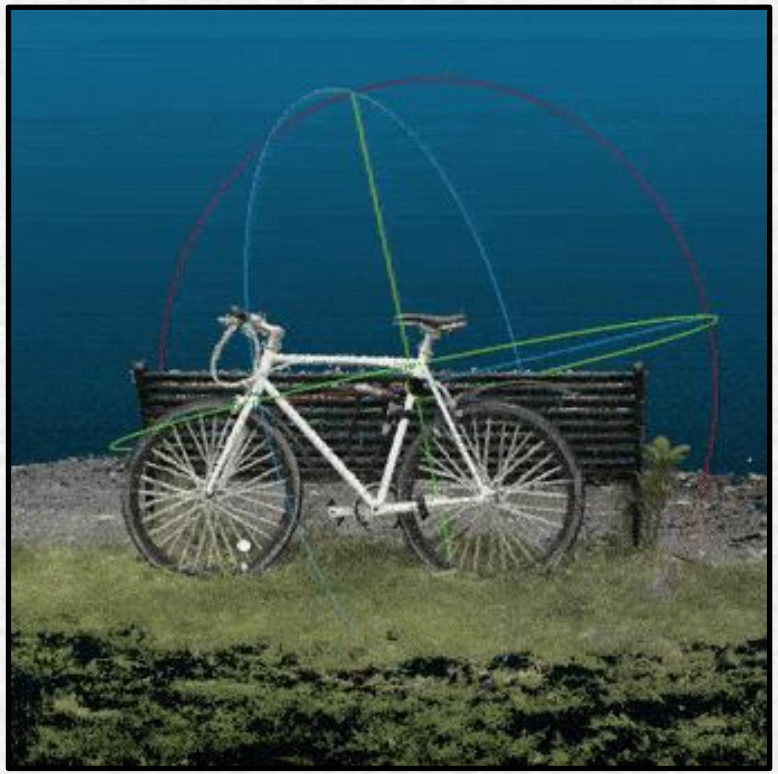
**Local rigid object**

Our **MERCY**

Frame

# 1. Introduction- Background

**3D Gaussian Splatting (3DGS)**



(x, y, z)

**4D Gaussian Splatting (4DGS)**

(x, y, z, t)

# 1. Introduction- Background

## Event Camera

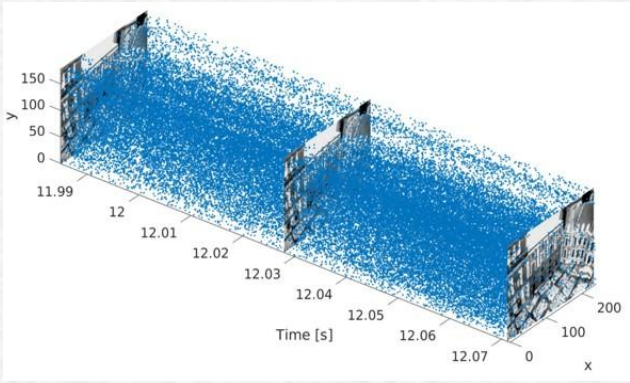
An event is triggered when the change in log-intensity at a pixel exceeds a predefined contrast threshold.

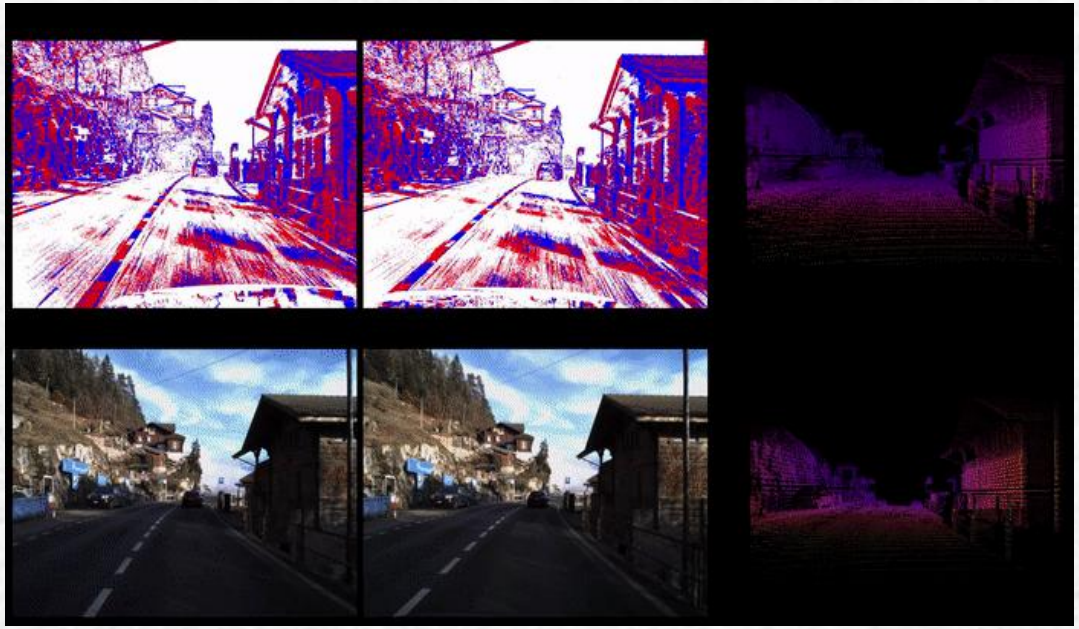
Output: $e_i = (x_i, y_i, t_i, p_i)$

Pixel Coordinates

Timestamp

Polarity (ON / OFF)



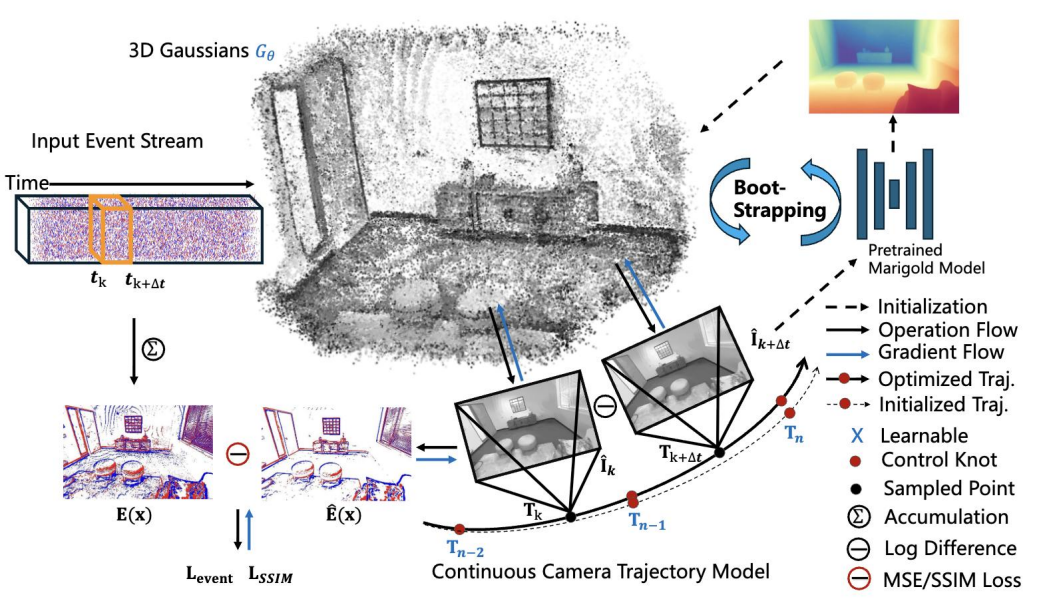
Event camera data visualization

# 1. Introduction- Background

IncEventGS: Pose-Free Gaussian Splatting from a Single Event Camera

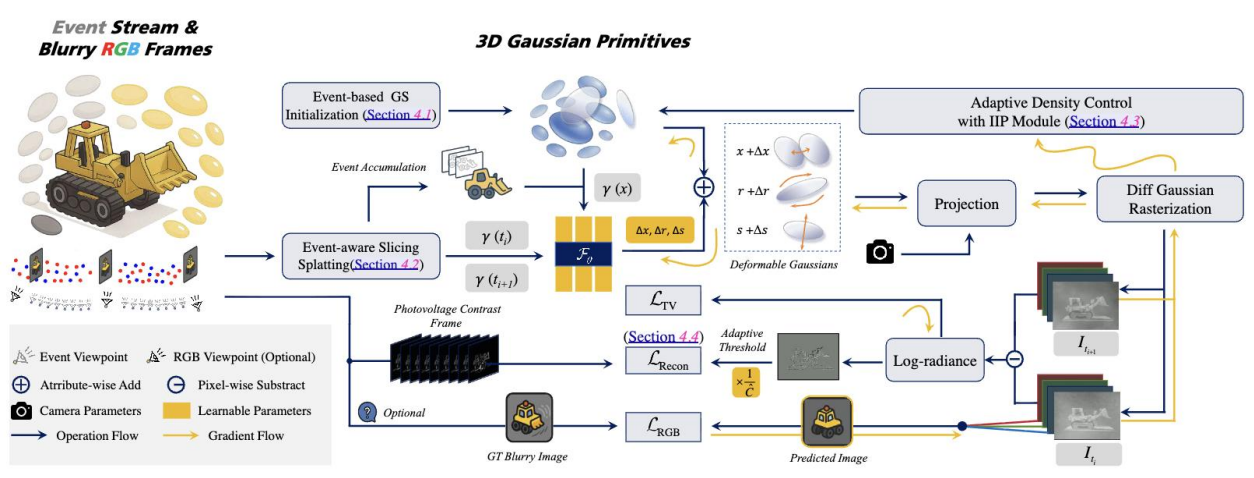
**3DGS**

**Monocular Moving Camera**




E-4DGS: High-Fidelity Dynamic Reconstruction from the Multi-view Event Cameras

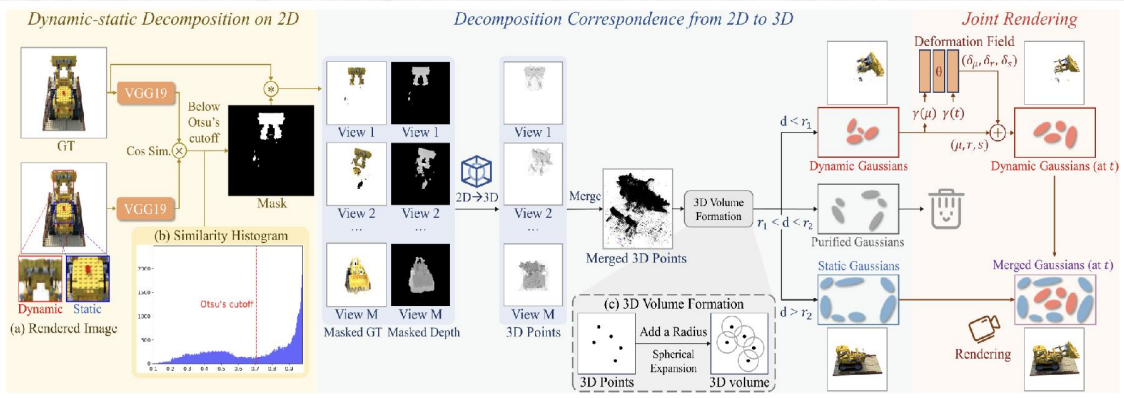
**Multi-camera Moving Camera**



# 1. Introduction- Background

Event-boosted Deformable 3D Gaussians for Dynamic Scene Reconstruction

**Monocular** **Moving Camera**




Our **MERCY**

**Monocular** **Static Camera**

# 1. Introduction- Motivation

Our **MERCY** **Monocular** **Static Camera**

**Real-World Challenges:**

① Limited Accessibility

② Restricted Camera Movement

③ High Deployment Costs

**Can we achieve coherent 4D Gaussian reconstruction of locally rigid moving objects from a fixed monocular viewpoint?**

# 2. Data Preparation

Blender Modeling Duration: 20 Days

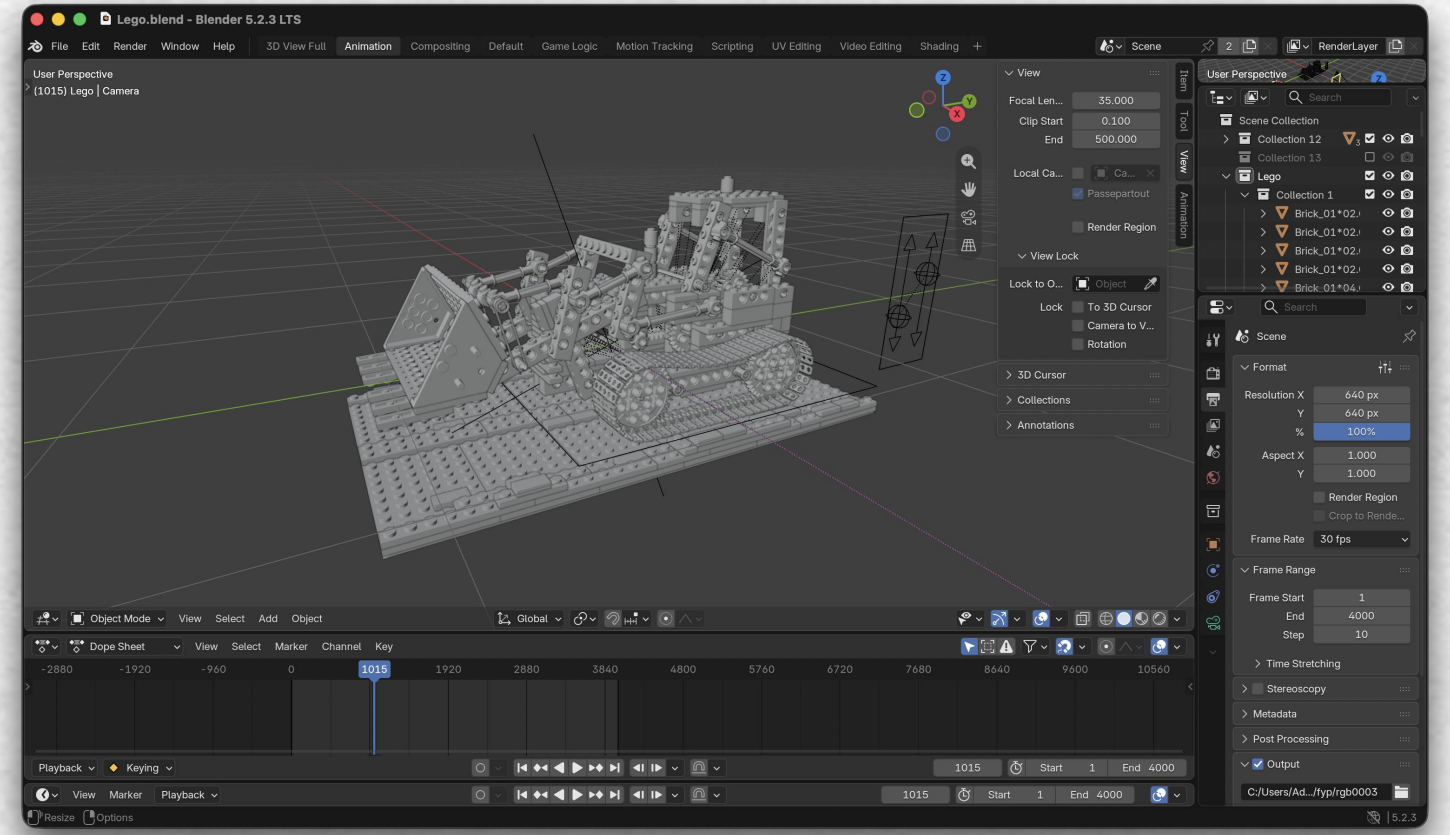



Figure: 3D Lego Model Created in Blender

Workflow:

- Motion Design
- Animation
- Texture Mapping
- Lighting Setup
- Camera Setup
- Video Rendering

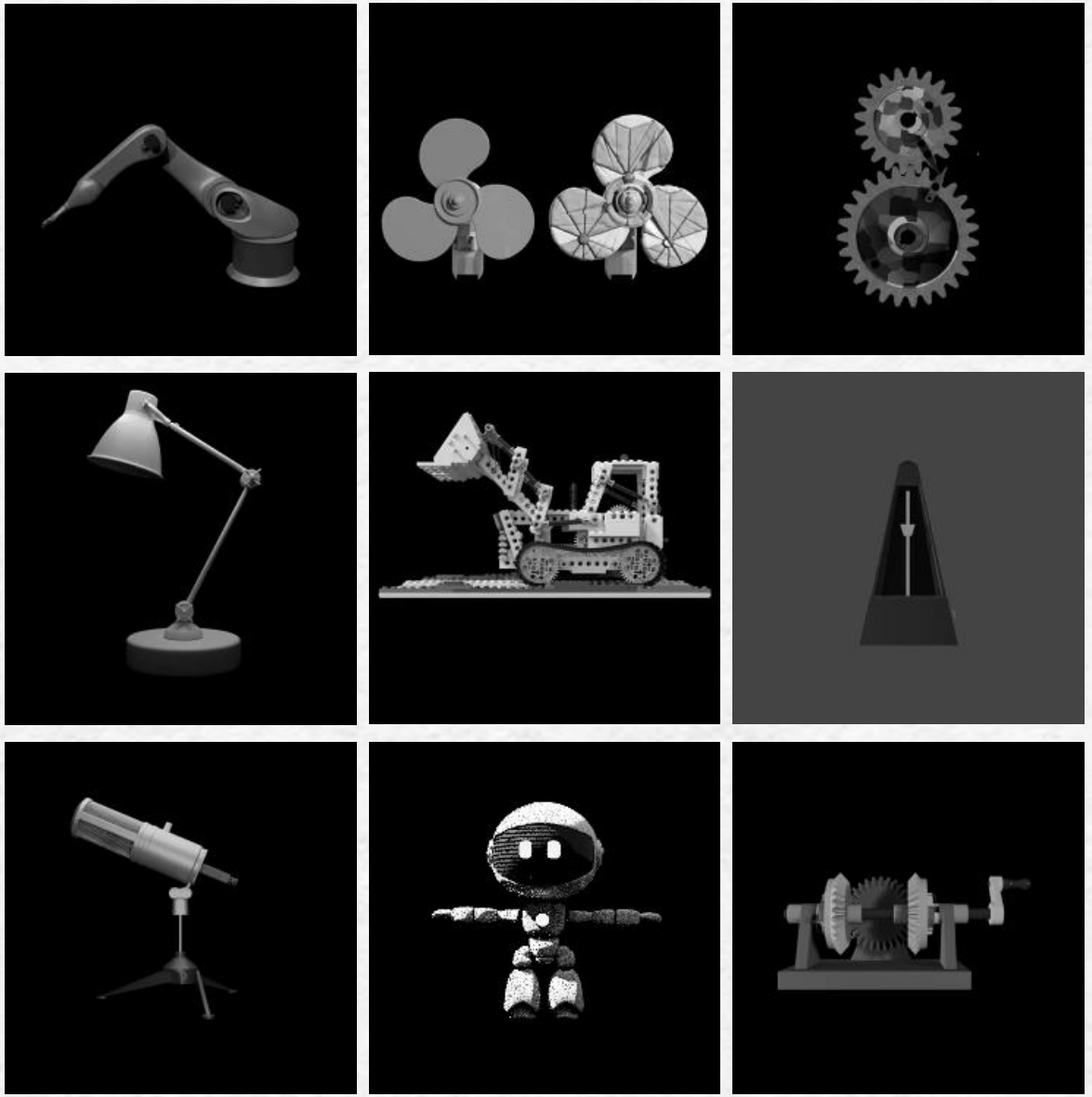
# 2. Data Preparation

## Video Rendering

**9 Object:**

| | | |
|---|---|---|
| Arm | Fan | Gears |
| Lamp | Lego | Metronome |
| Micphone | Robot | Rotation |

Video: 640 × 640 · 30 FPS



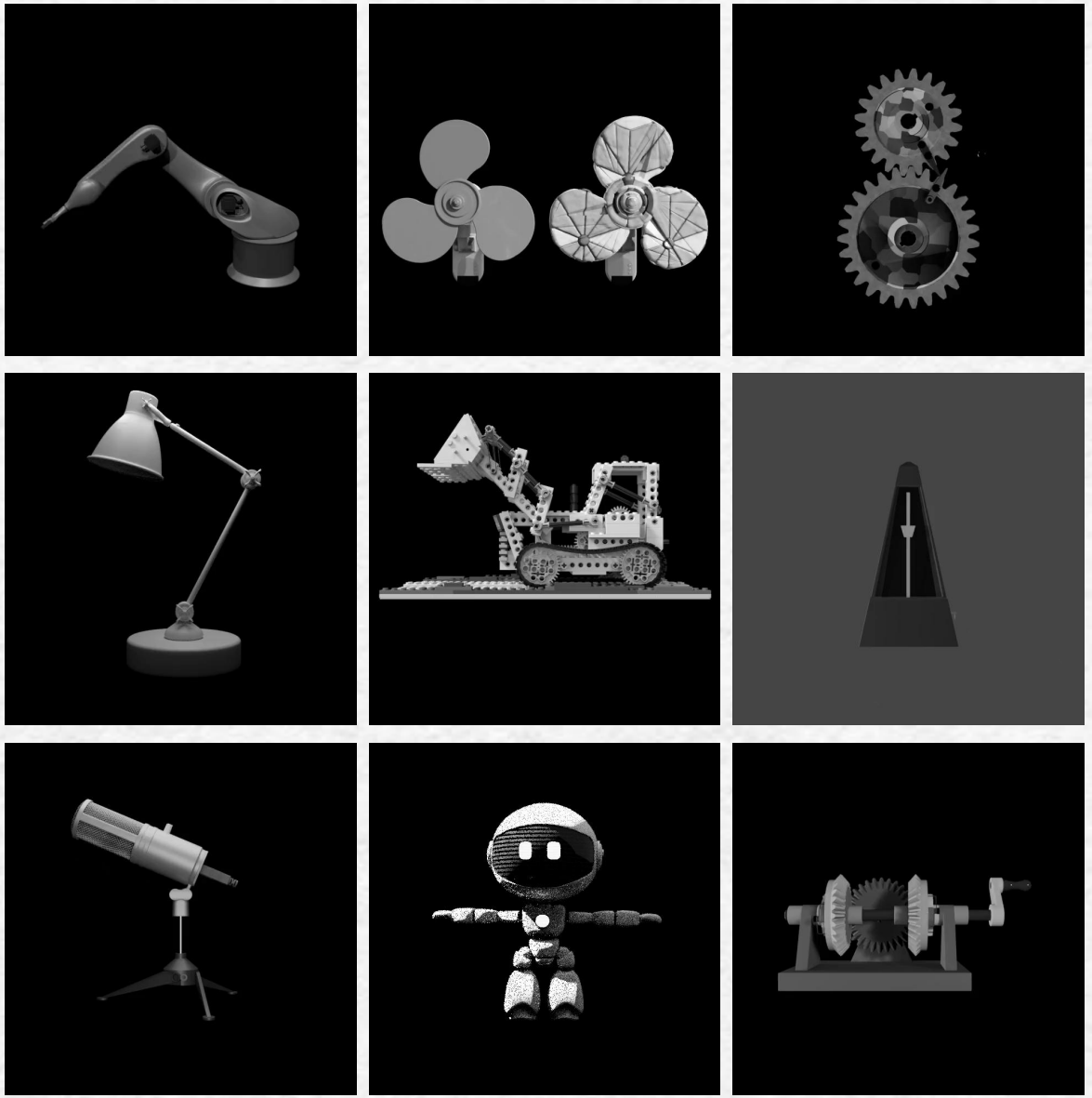
The Rendered Videos

# 2. Data Preparation

## First-frame Image Extraction

**Input:** Rendered video

**Purpose:** Provides appearance and structural cues for initial 3D reconstruction.



First Frame of the Rendered Video

# 2. Data Preparation

Blender-rendered Video (640 × 640)

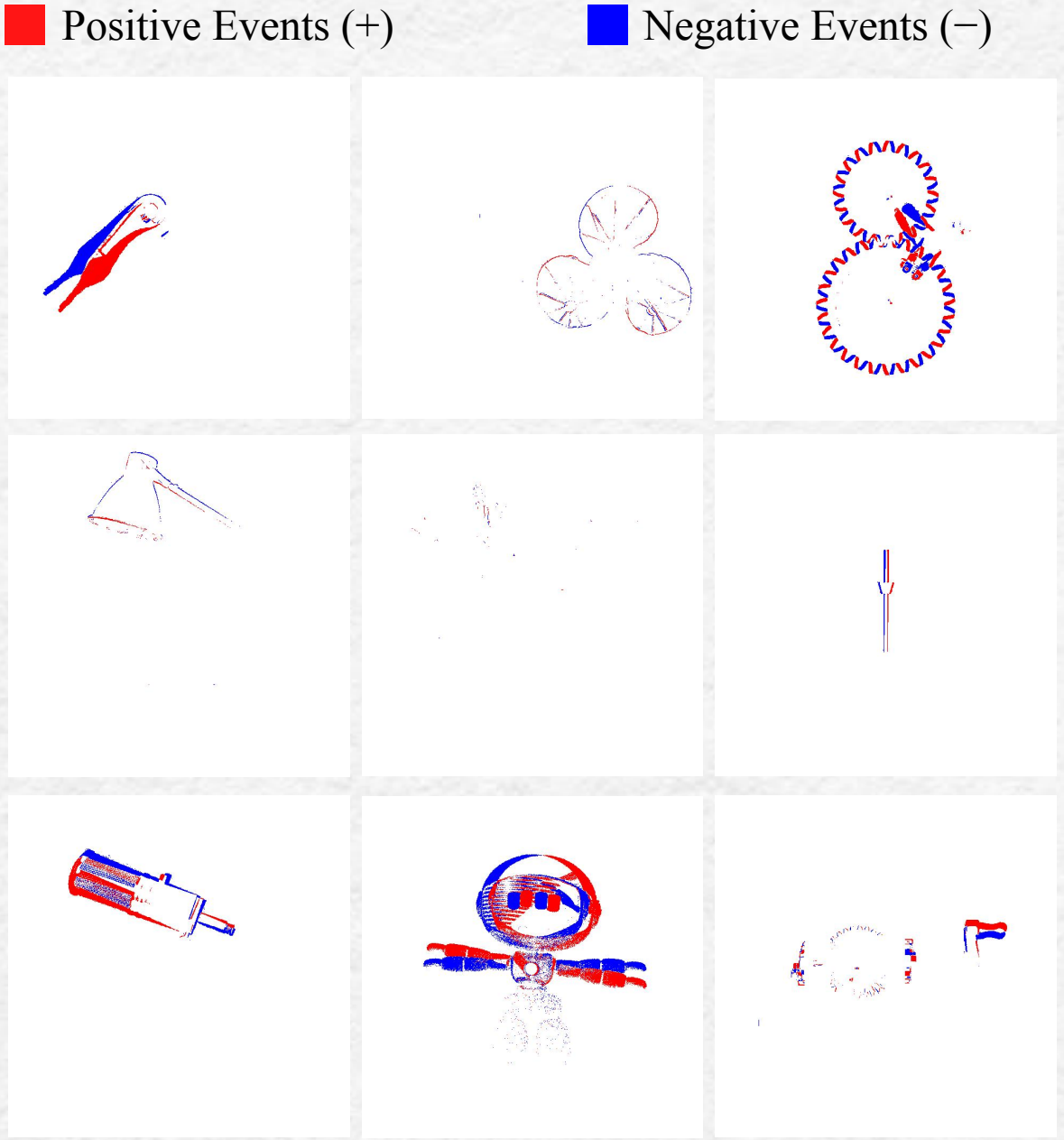
↓ Log-intensity Changes

**ESIM Event Camera Simulator**

↓

Event Stream: (x, y, t, p)

| Objects | Total Events |
| --- | --- |
| 9 | 31.01M |




Event Stream Visualization

# 2. Data Preparation - ESIM Event Camera Simulator

## Evaluation:

| | |
|---|---|
| 1. Spatial Alignment | Percentage of events located in regions exhibiting image-intensity changes; a proxy for spatial precision. Higher is better. |
| 2. Intensity-Change Coverage | Percentage of image-intensity-change pixels with nearby events; a proxy for event coverage. Higher is better. |
| 3. Polarity Consistency | Percentage of events whose polarity agrees with the direction of image-intensity change. Higher is better. |

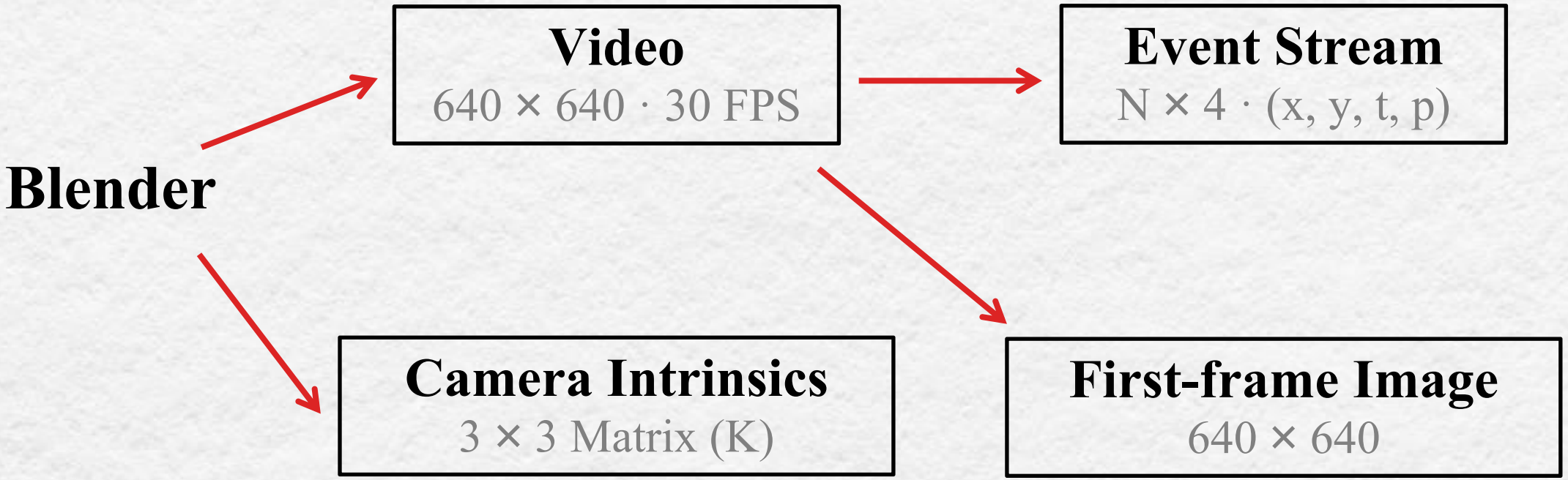
# 2. Data Preparation - ESIM Event Camera Simulator

Evaluation:

| Object | Events | Alignment | Coverage | Polarity |
|---|---|---|---|---|
| Arm | 2.64M | 96.91% | 88.19% | 99.55% |
| Fan | 7.02M | 99.68% | 91.65% | 99.84% |
| Gears | 3.32M | 99.61% | 71.91% | 99.03% |
| Lamp | 1.45M | 98.88% | 77.81% | 99.67% |
| Lego | 5.82M | 99.84% | 86.42% | 99.45% |
| Metronome | 0.60M | 94.01% | 75.52% | 99.38% |
| Micphone | 2.85M | 99.11% | 85.29% | 99.79% |
| Robot | 6.61M | 98.31% | 99.05% | 99.37% |
| Rotation | 0.70M | 98.00% | 72.75% | 98.79% |
| **Average:** | | 98.26% | 83.18% | 99.43% |

# 2. Data Preparation

**Dataset Ready → MERCY Proposed Methodology**



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

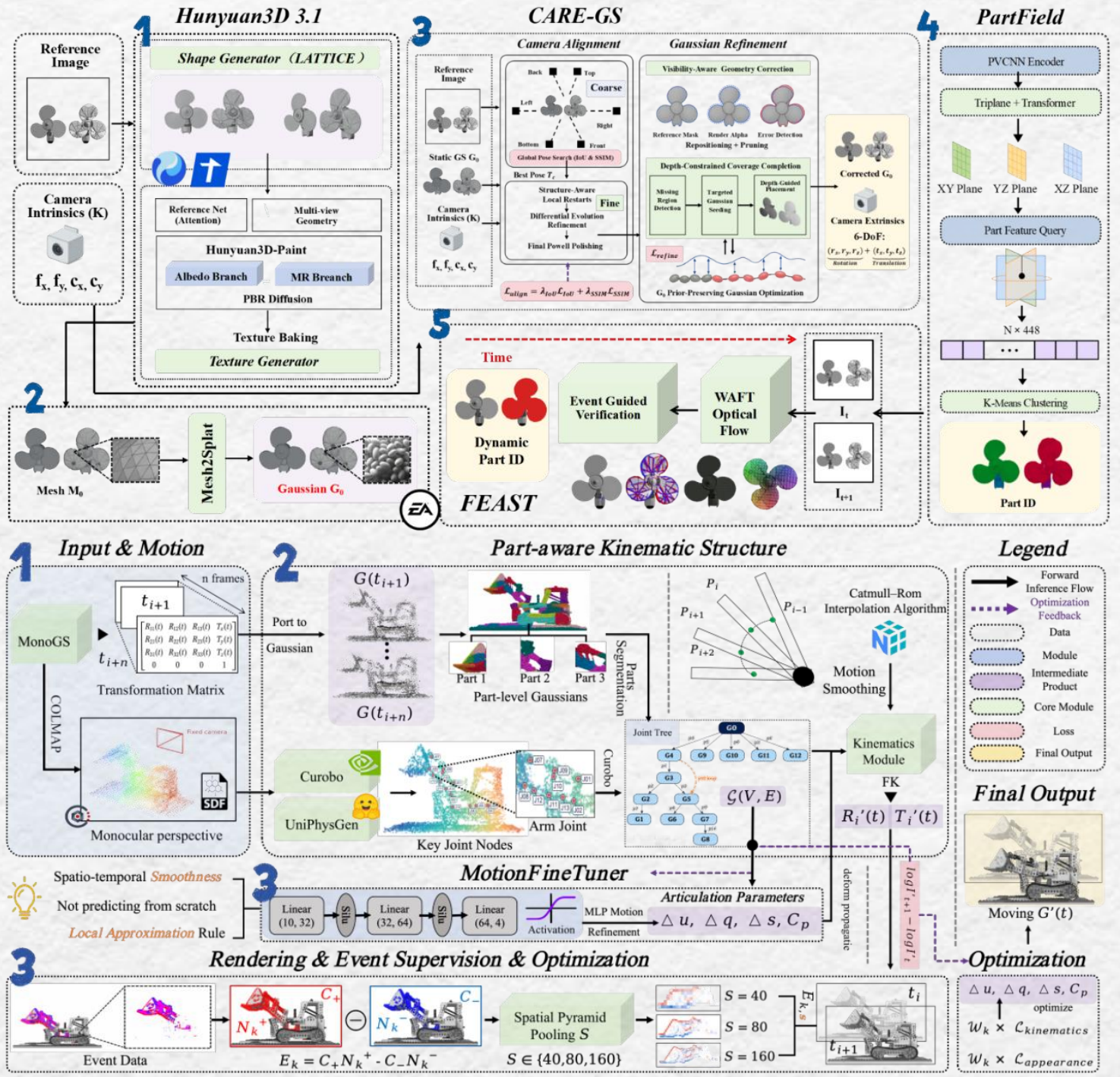
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

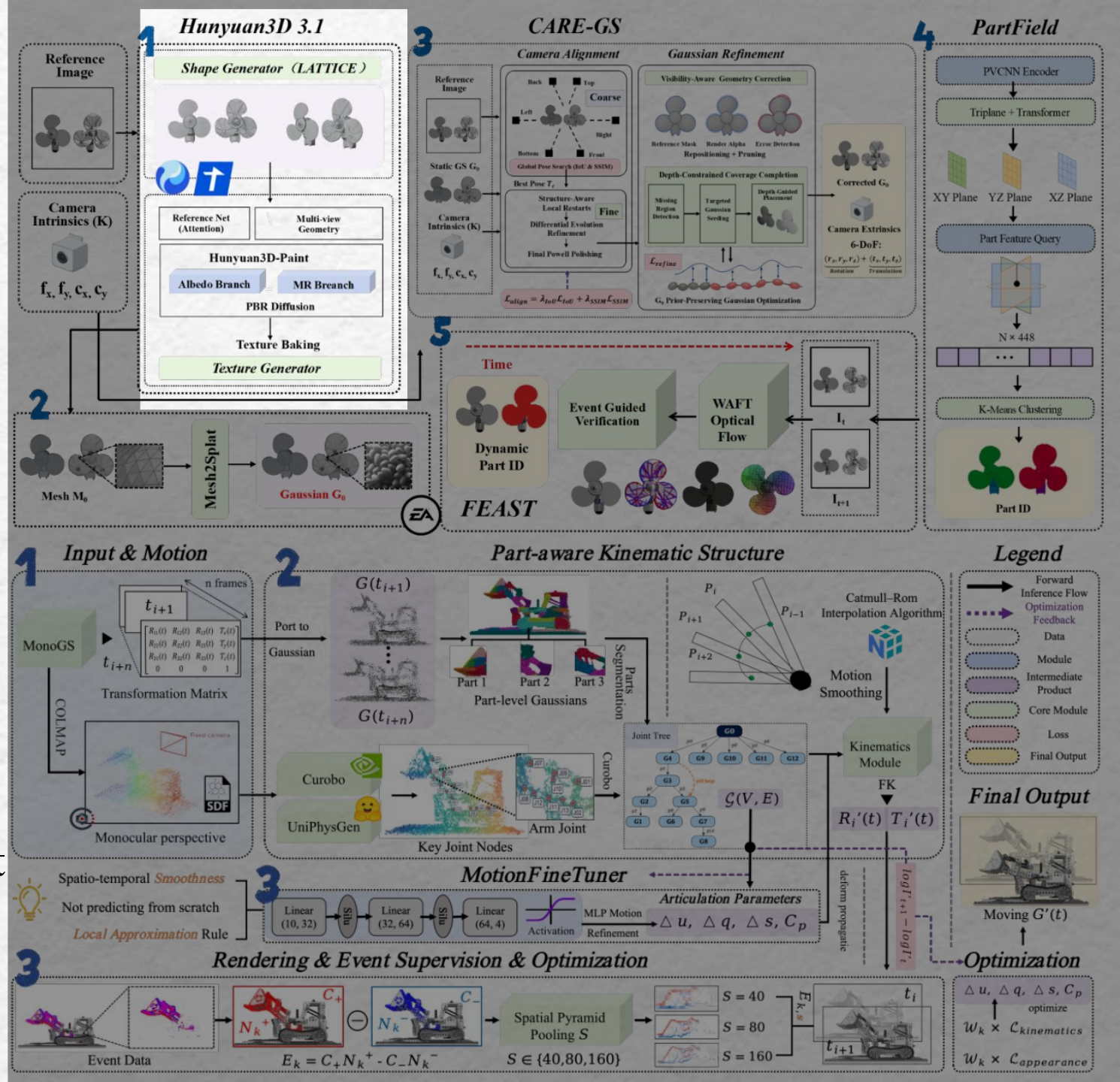
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

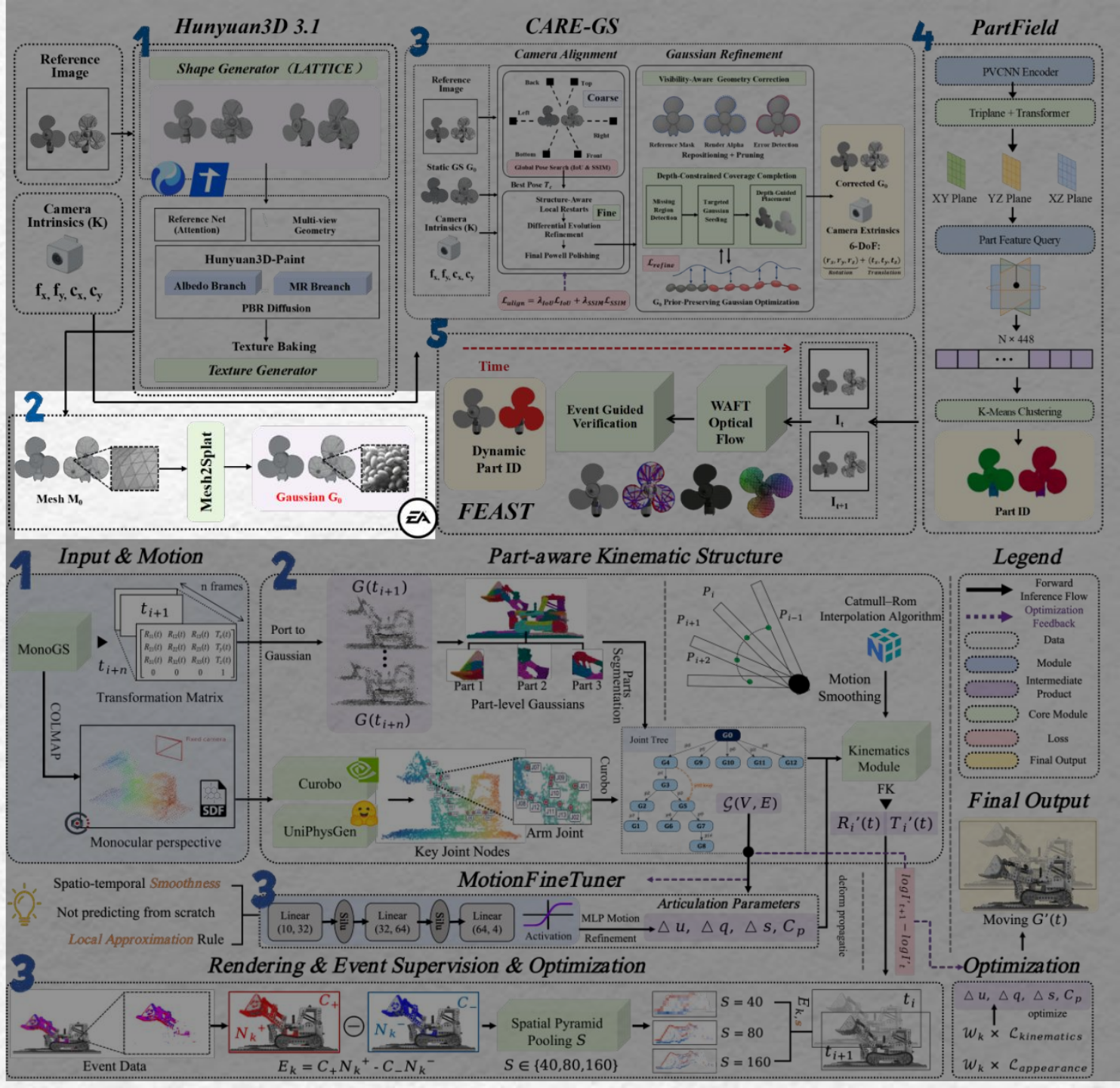
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

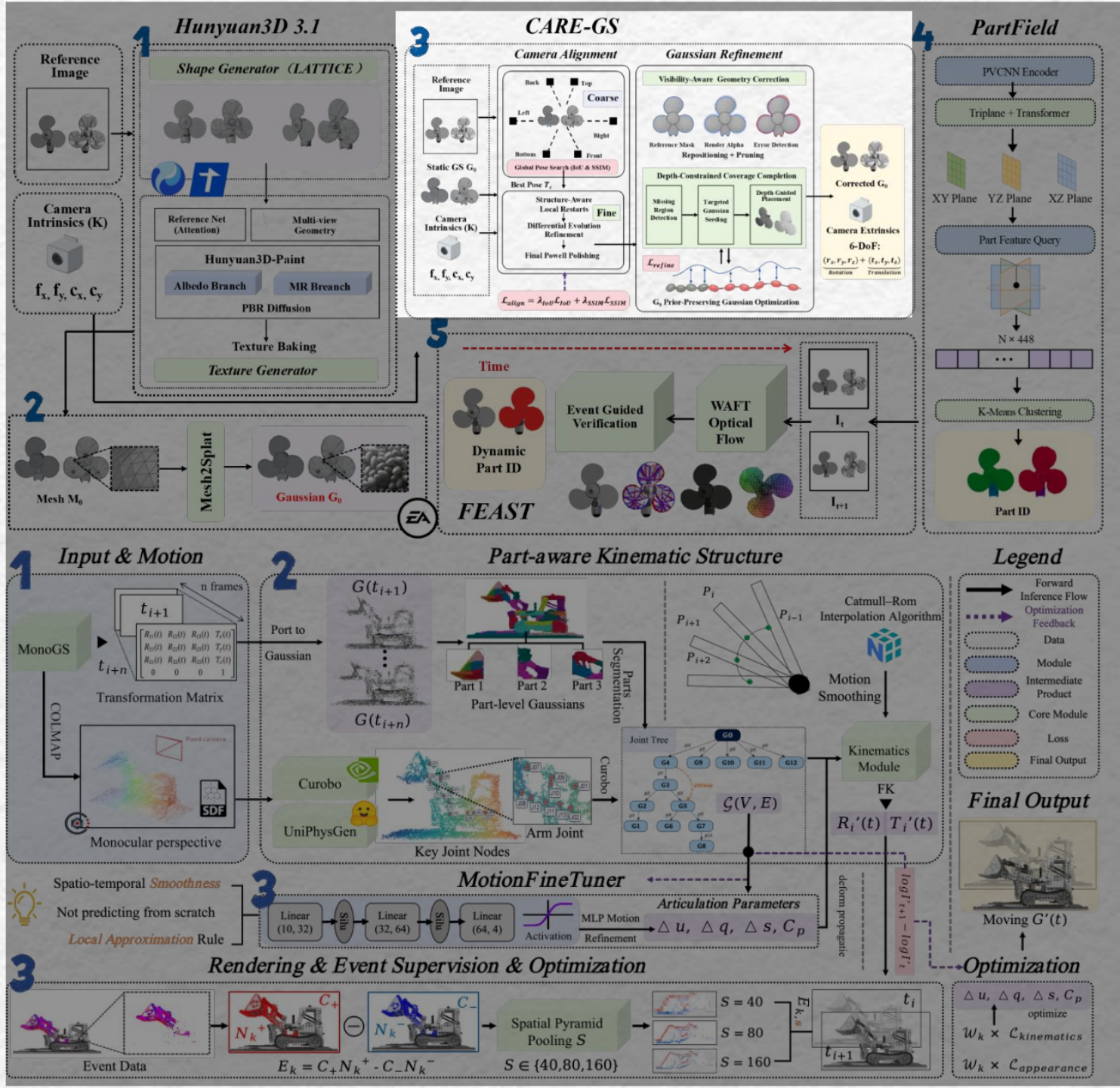
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

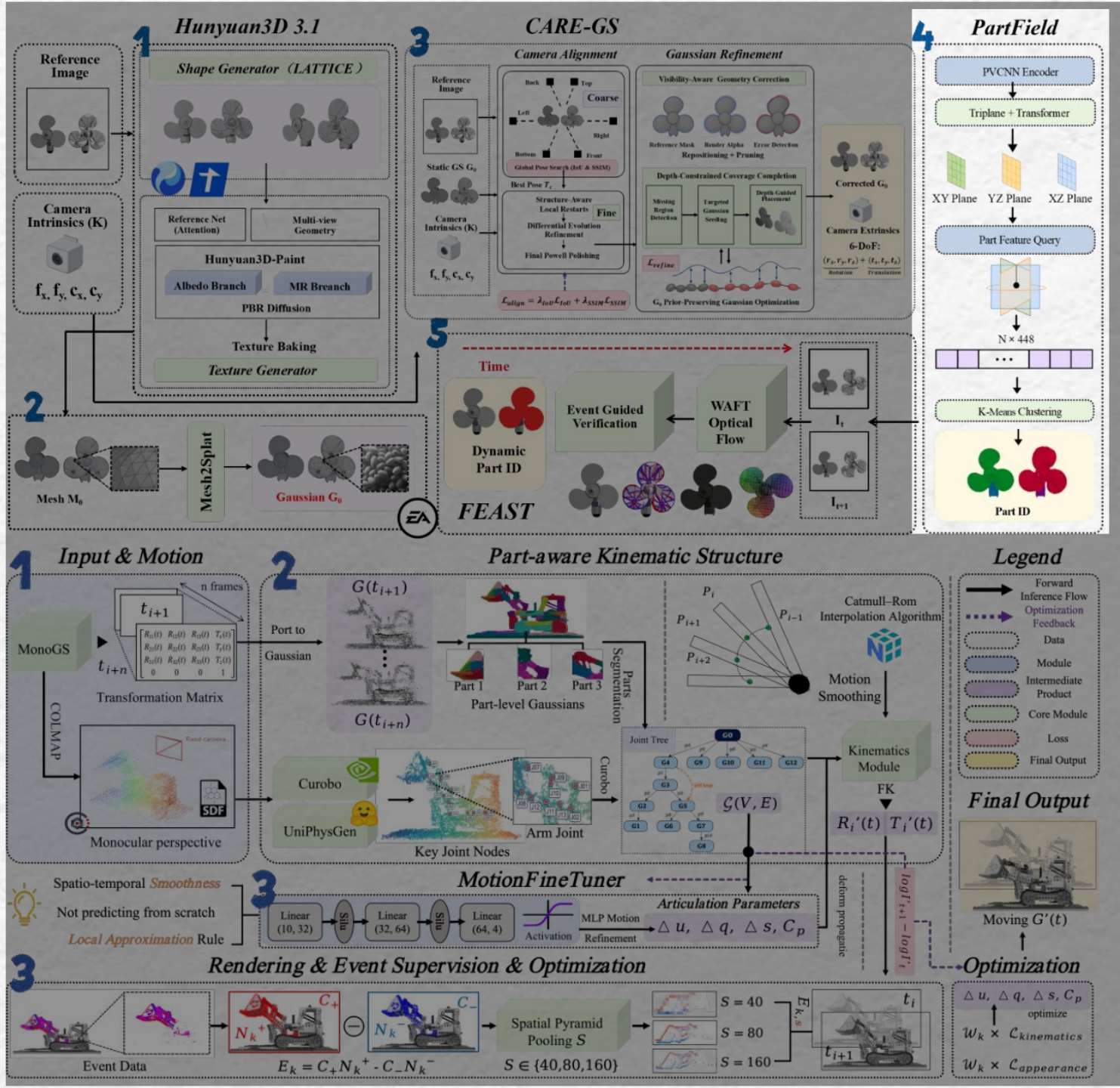
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# 5. PartField

## Why PartField?

Problem:

Estimating motion independently for millions of Gaussians is computationally complex and may cause inconsistent deformation.

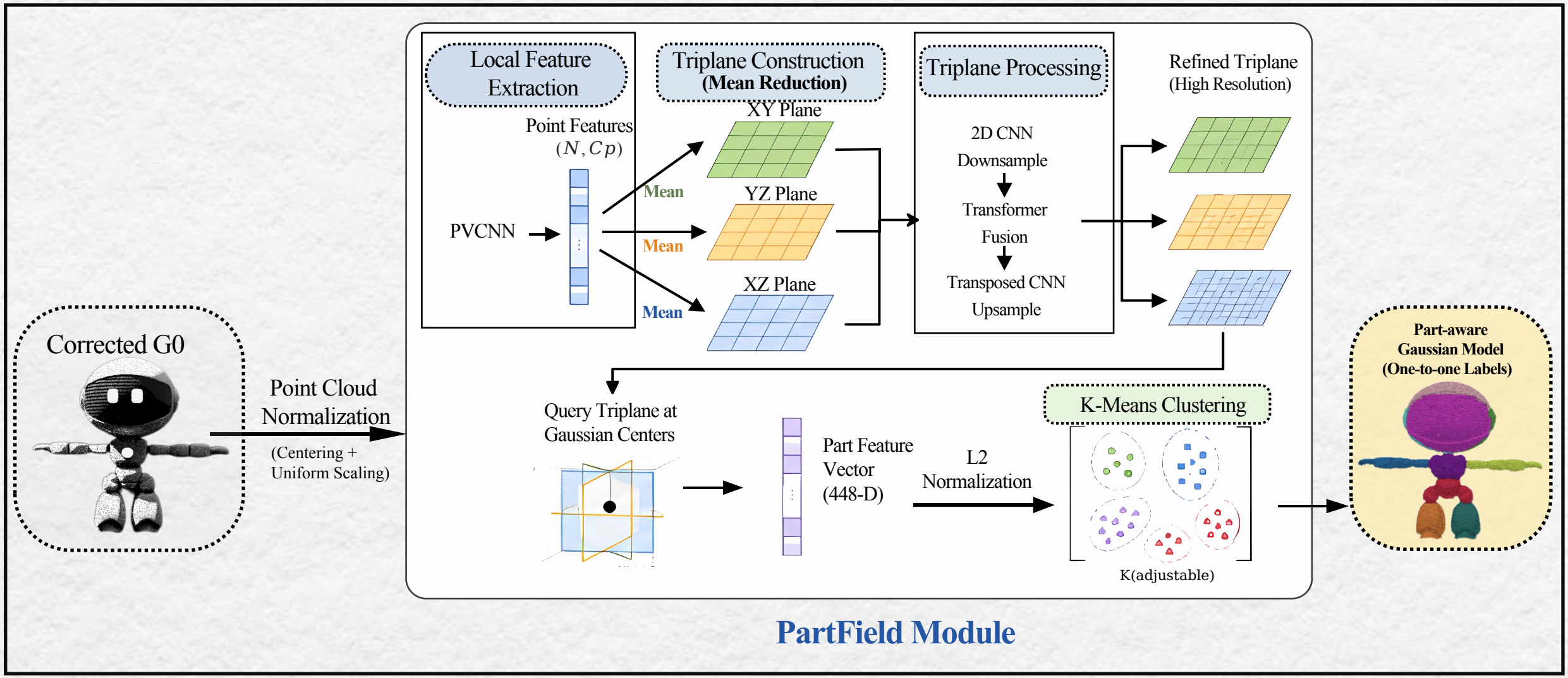
Our Solution:

Use **PartField** to assign part labels to Gaussians, enabling part-level rigid motion estimation instead of independent Gaussian-wise motion estimation.

Benefits:

· Reduced motion complexity · Satisfy local rigidity · Motion consistency

# 5. PartField

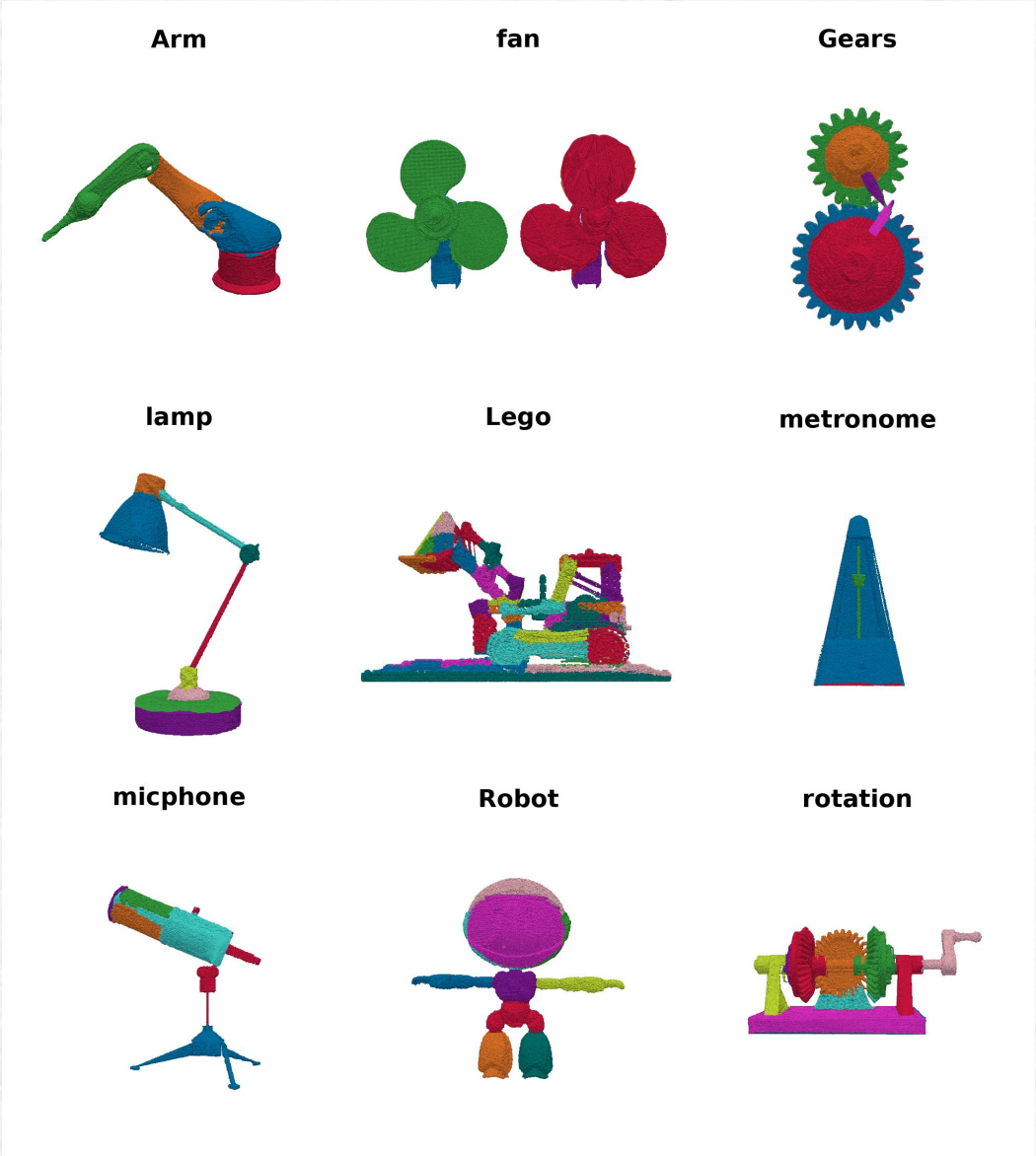

Corrected G0
Point Cloud Normalization
(Centering + Uniform Scaling)
Local Feature Extraction
Point Features
$(N, Cp)$
PVCNN
Triplane Construction
(Mean Reduction)
XY Plane
YZ Plane
XZ Plane
Mean
Mean
Mean
Triplane Processing
2D CNN Downsample
Transformer Fusion
Transposed CNN Upsample
Refined Triplane
(High Resolution)
Query Triplane at Gaussian Centers
Part Feature Vector
(448-D)
L2 Normalization
K-Means Clustering
K(adjustable)
Part-aware Gaussian Model
(One-to-one Labels)
PartField Module

# 5. PartField

Output:

Gaussian-wise Part Labels

Each Gaussian is assigned a structural part ID.




Component-wise Gaussian Segmentation Results

# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

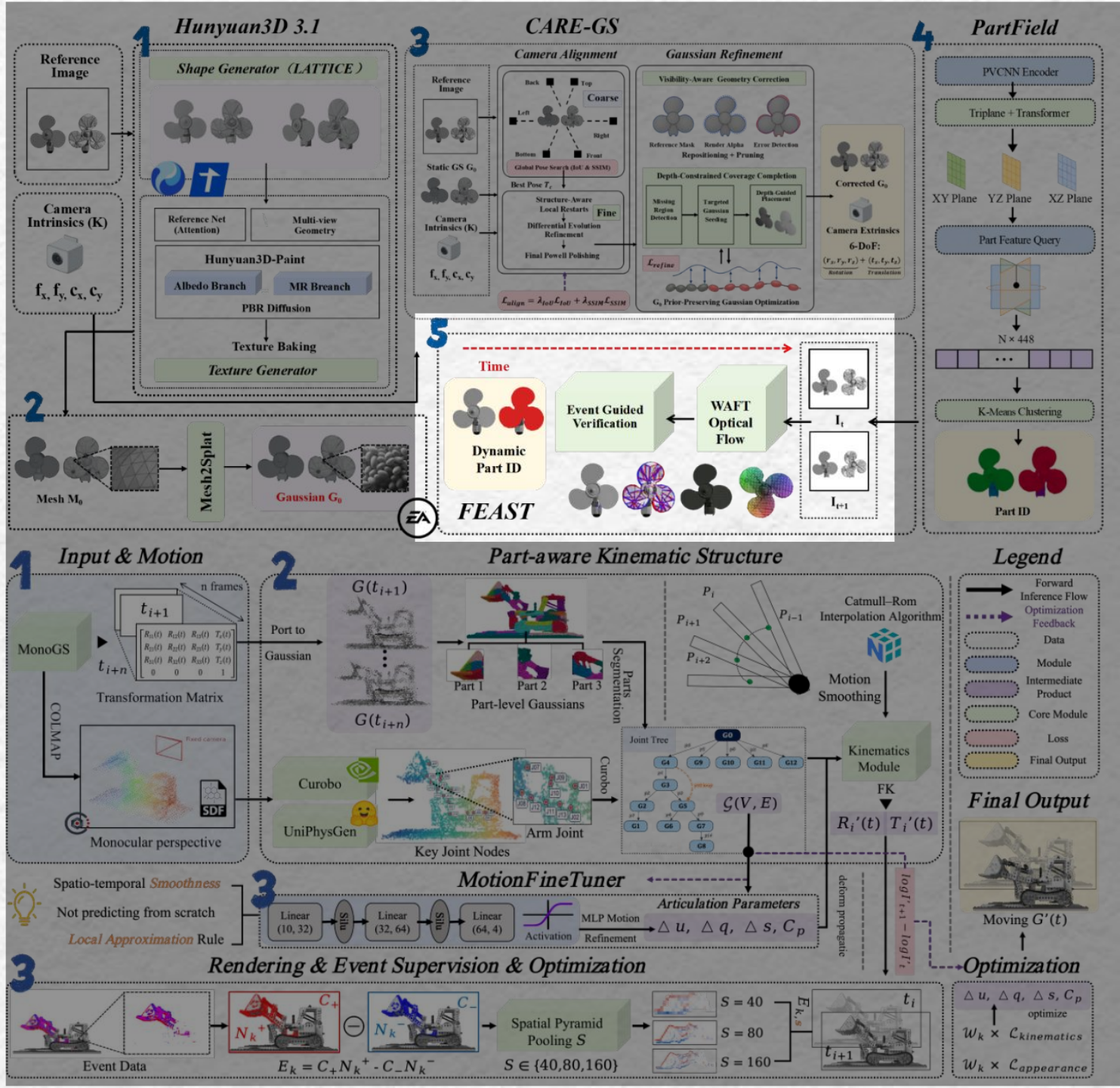
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization

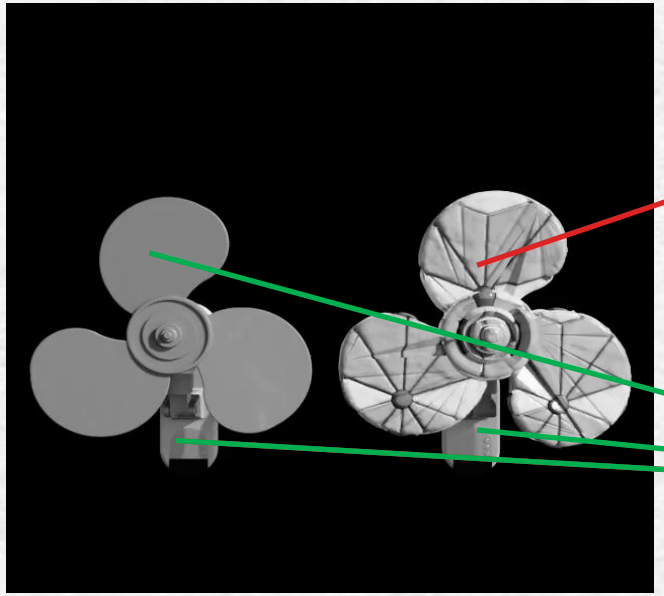


# 6. FEAST

**Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours**

## Why Do We Need Dynamic–Static Separation?

1. Enable Shared Rigid Motion: Identify dynamic parts so that Gaussians within each rigid component can share a common SE(3) transformation.
2. Prevent Unnecessary Deformation: Keep static regions fixed to reduce motion ambiguity, unnecessary optimization, and geometric distortion.

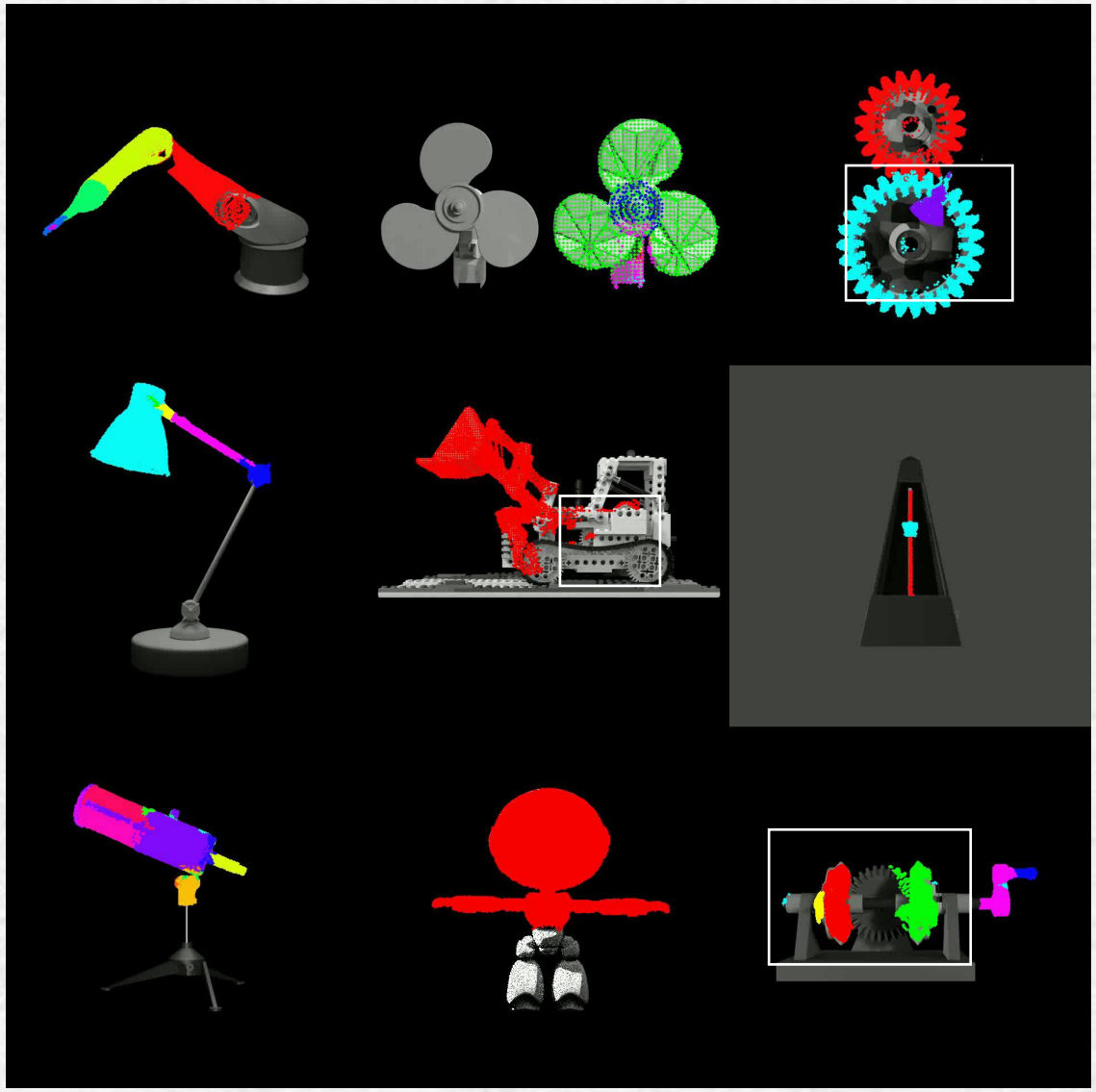


**Dynamic Part**

**Static Part**

# 6. FEAST

## Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours

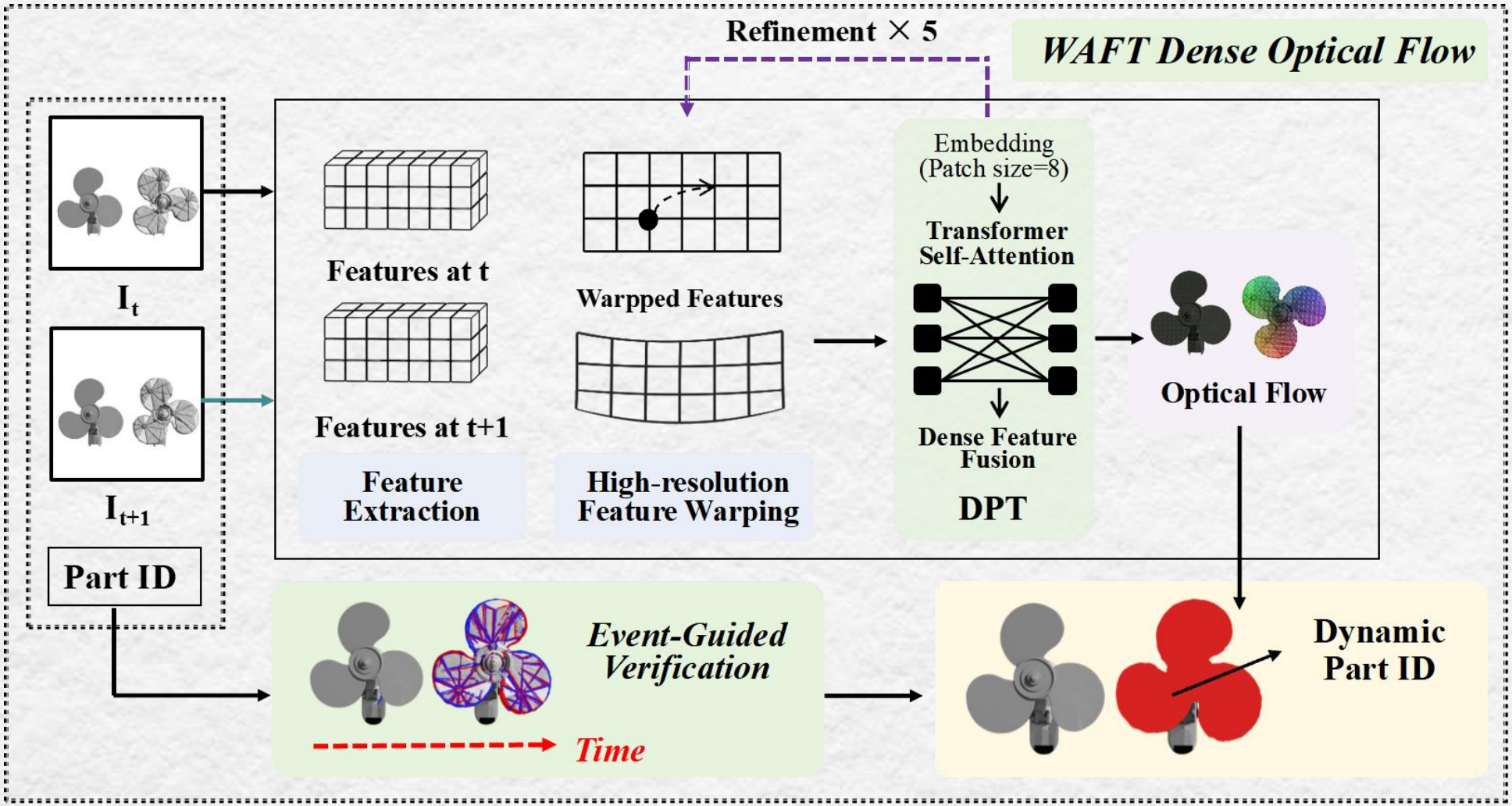


## First Version: Cotracker3

1. **Sparse Spatial Coverage:** Tracking points may not sufficiently cover all moving regions.
2. **Incomplete Motion Detection:** Small or weakly moving parts may lack sufficient valid tracking trajectories.
3. **Loss of Visible Correspondences:** Tracking points become occluded as the object rotates or other motion, limiting continuous motion tracking.

# 6. FEAST

## Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours




Part ID from PartField

↓

FEAST

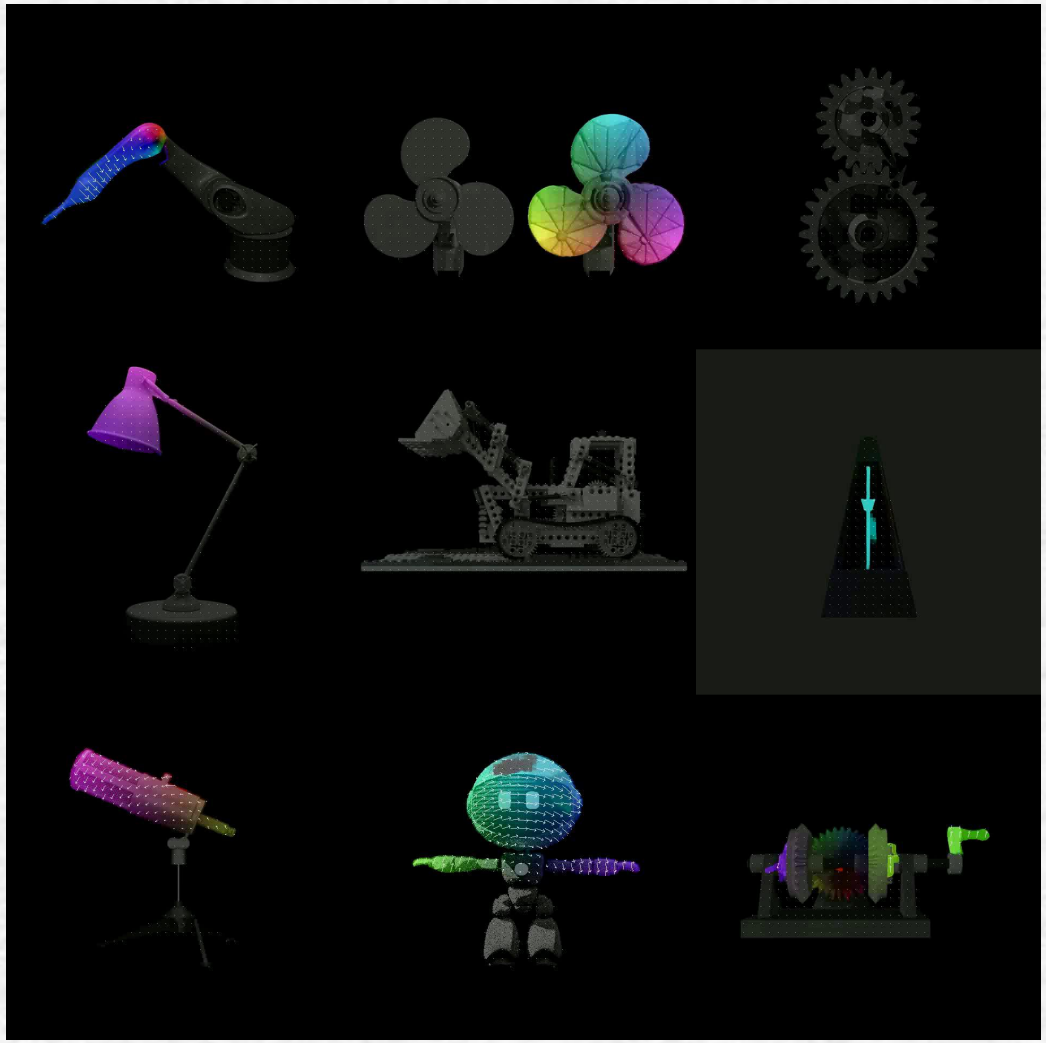
↓

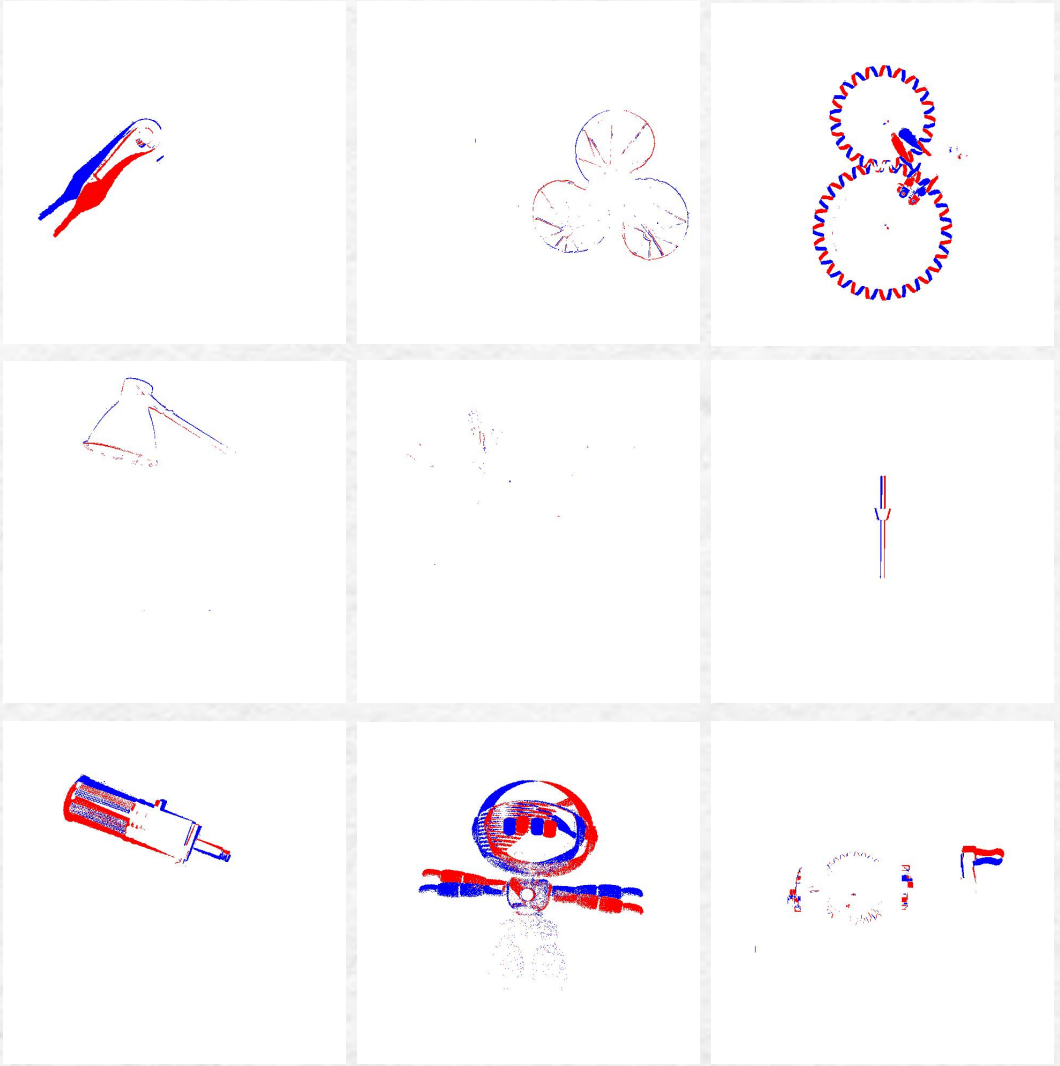
Dynamic Part ID

FEAST

# 6. FEAST

## Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours





# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

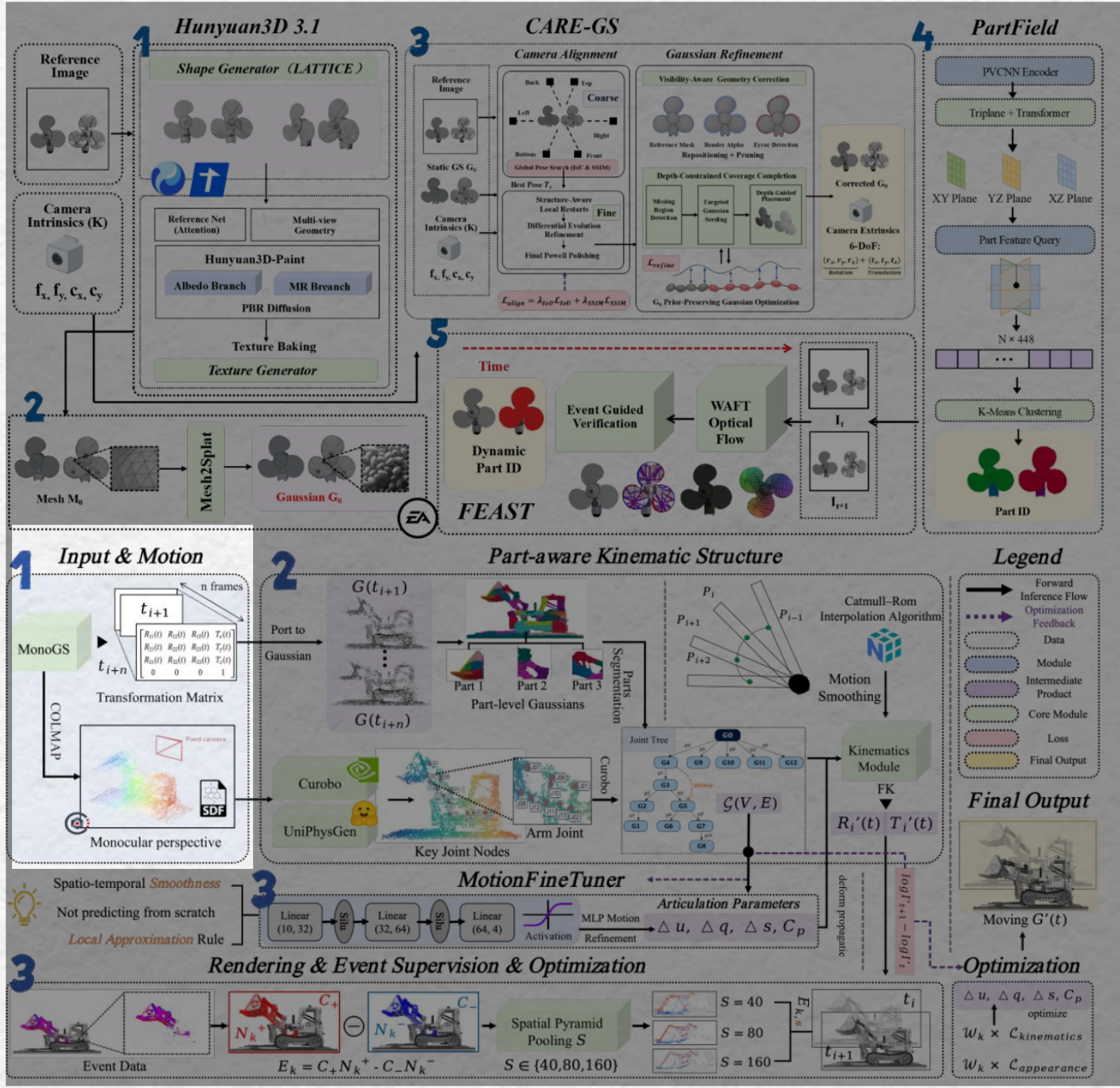
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

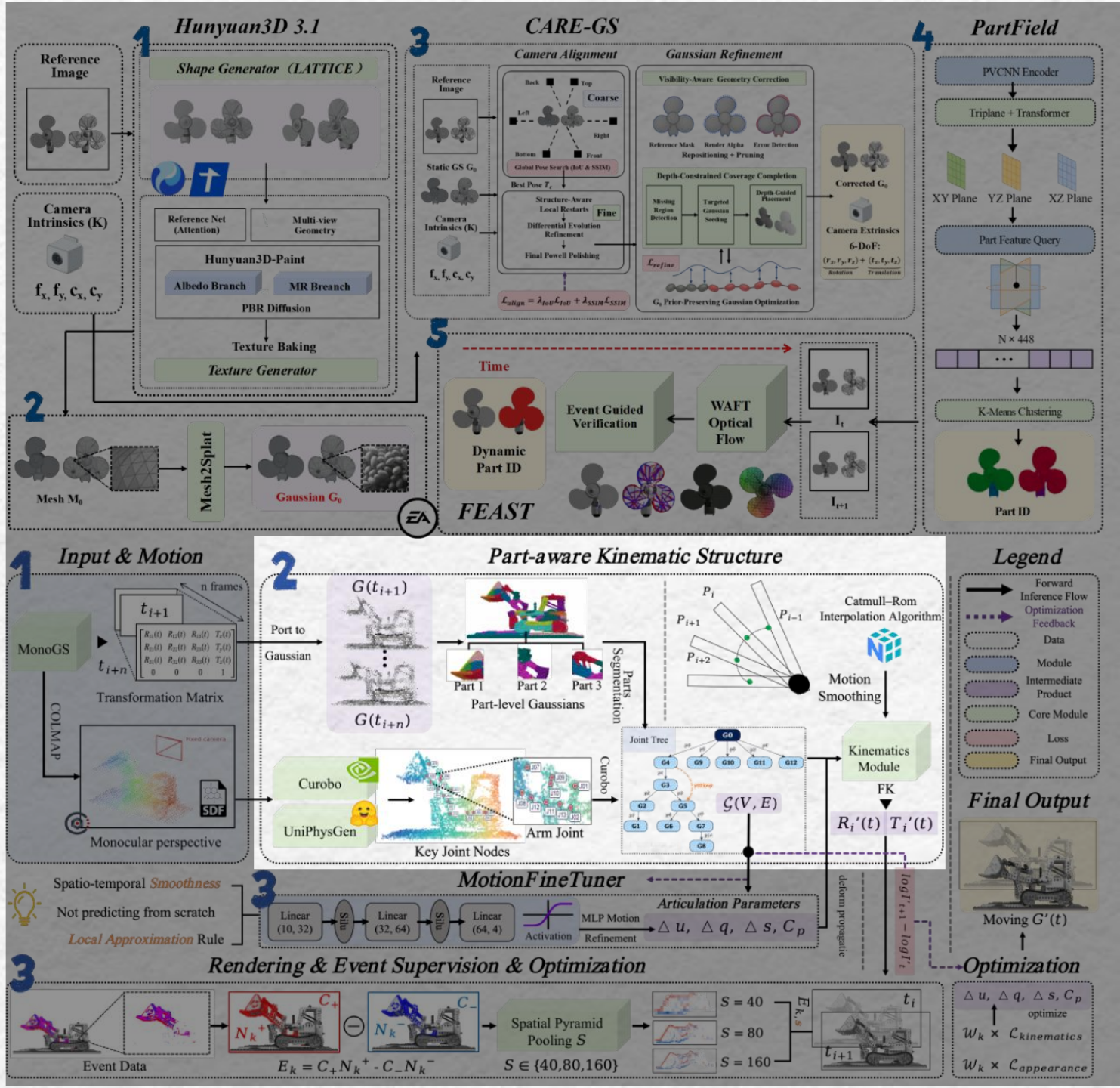
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

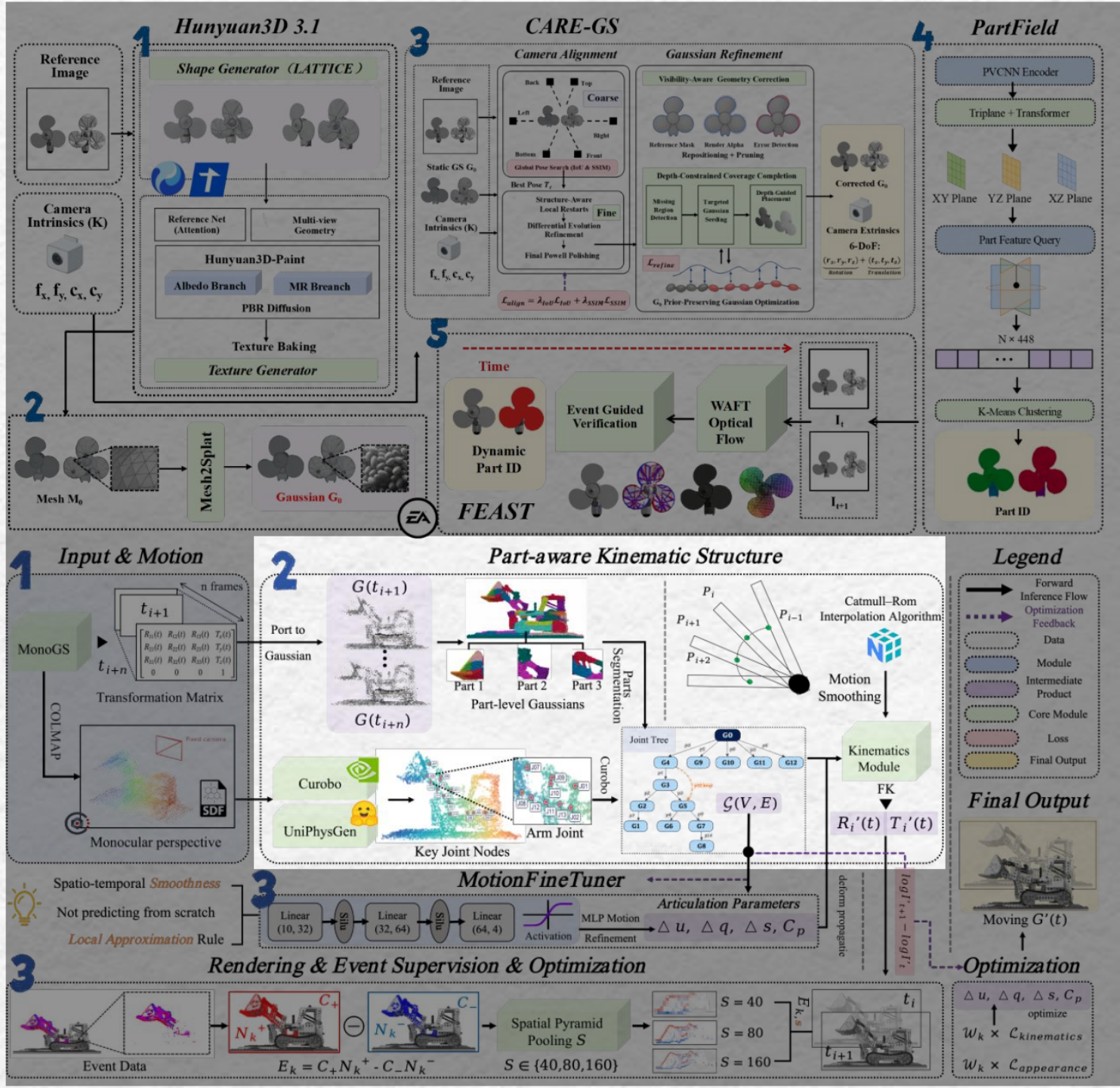
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

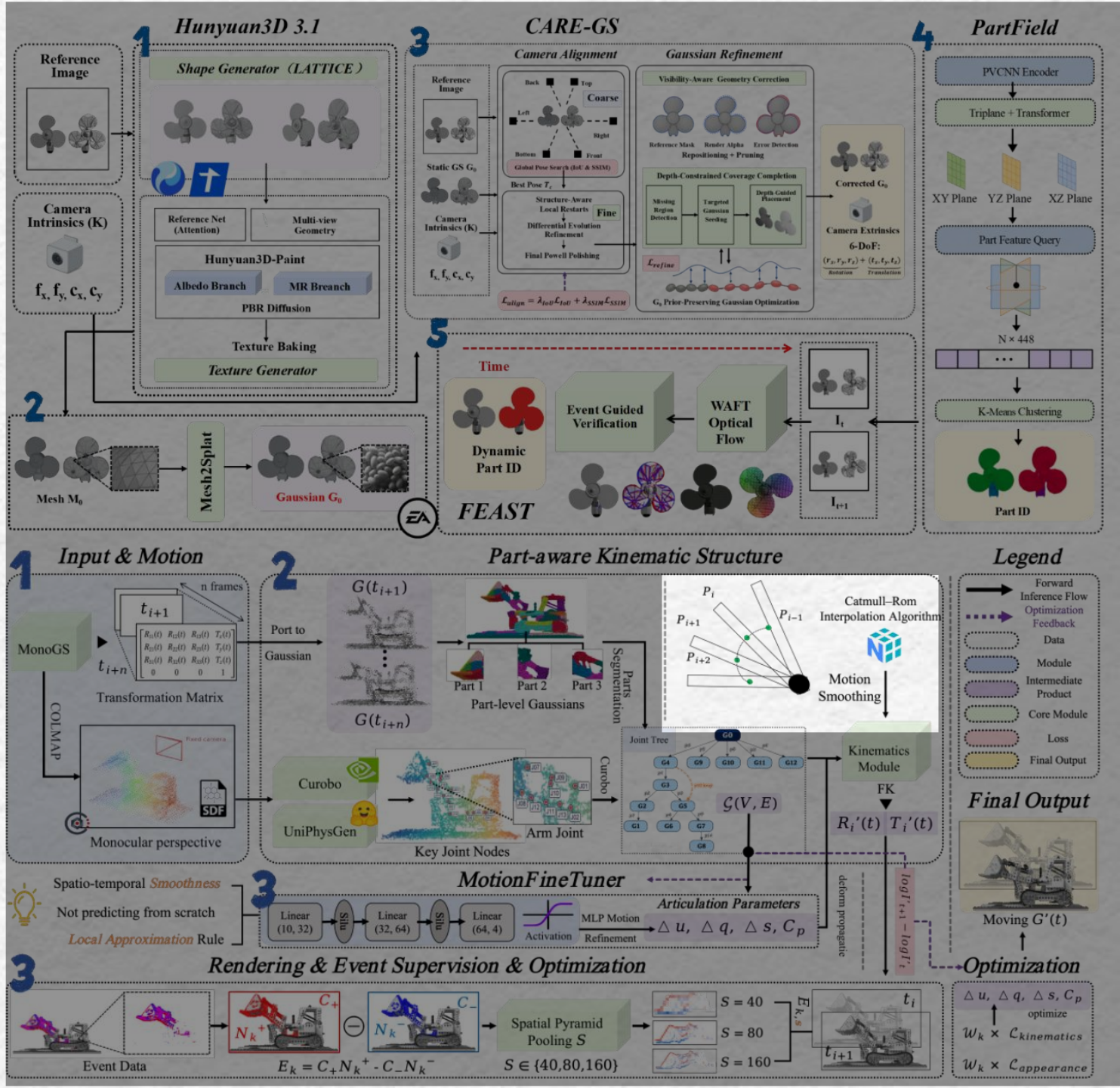
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

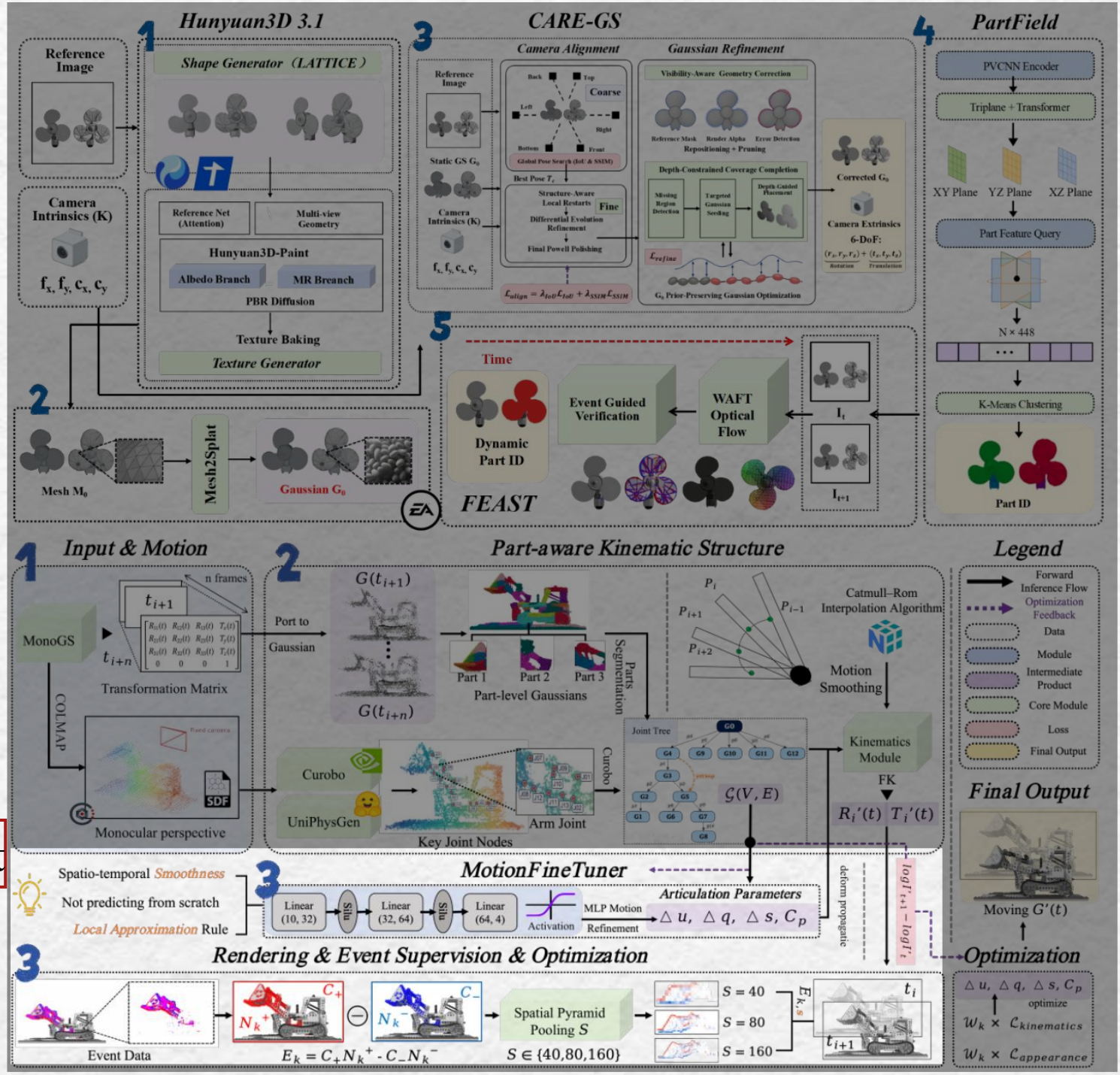
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# Pipeline

Input

→ Hunyuan 3D 3.1

→ Mesh2Splat

→ CARE- GS

→ PartField

→ FEAST

→ MonoGS

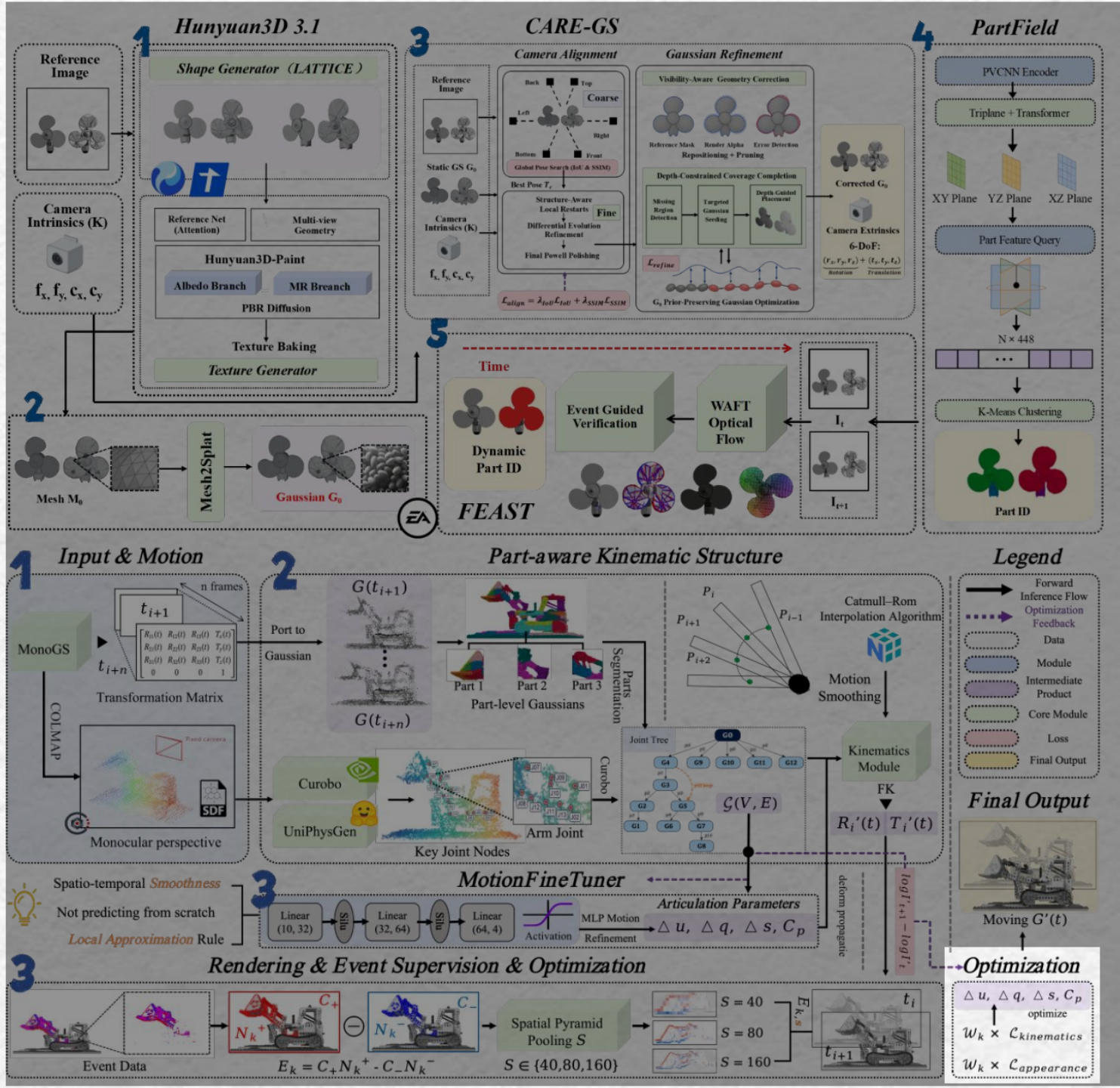
→ Curaobo & UniPhyGen

→ Kimetimic module

→ Catmull-Rom (numpy)

→ MotionFineTuner & Supervised Event

→ Joint Optimization



# 3. Static 3D Gaussian Initialization

Chen Qianqi 2330034001

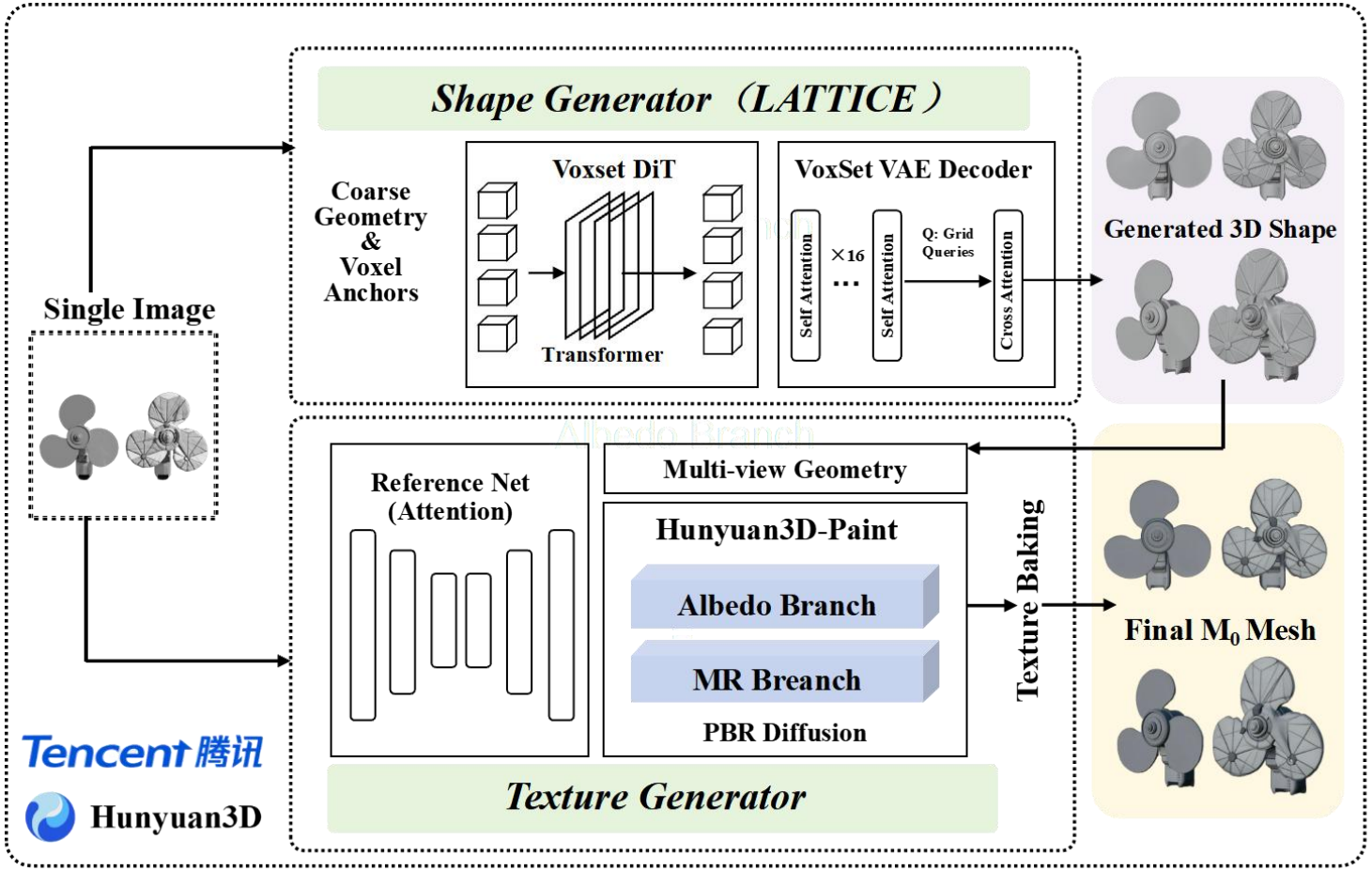
## Why we need Static 3D Gaussian Initialization?

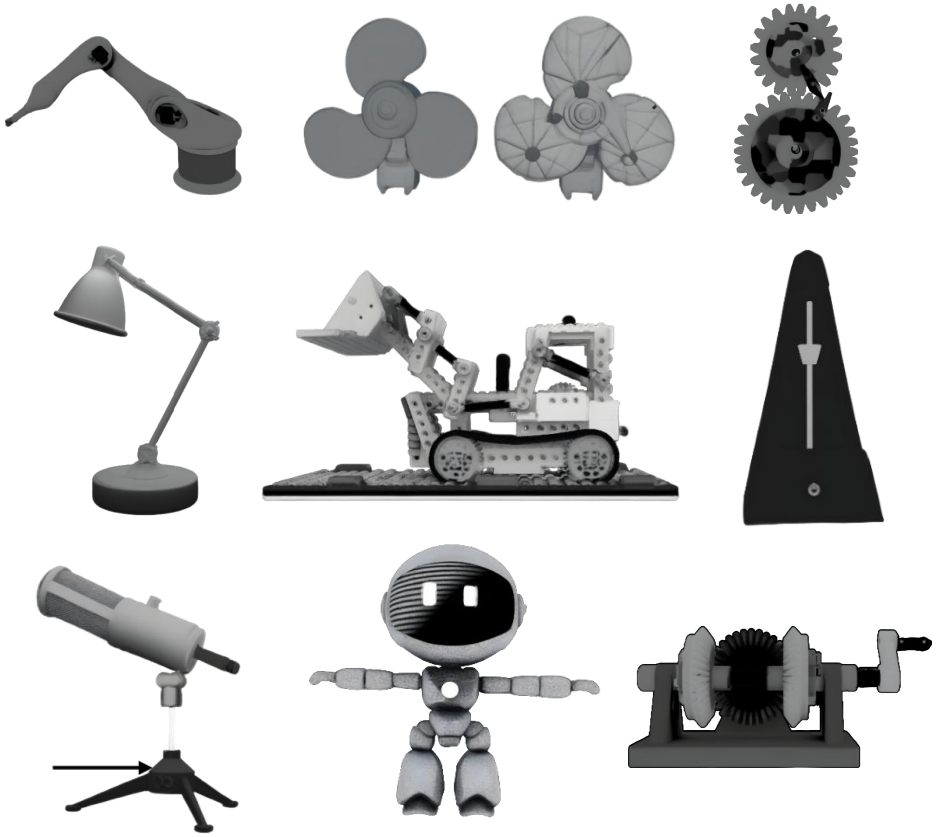
**1. 2D-Only Observations:** RGB videos and event streams provide image-space information rather than explicit 3D geometry.
**2. 3D Geometry Initialization:** Infer an initial 3D representation from a single RGB image using learned geometric priors.
**3. Foundation for 4D Reconstruction:** Establish a canonical Gaussian representation for subsequent 3D motion estimation and deformation.

**2D Observations → Initial 3D Static Model**

# 3. Static 3D Gaussian Initialization

## 3.1 Hunyuan3D 2D Image → Initial 3D Static Mesh

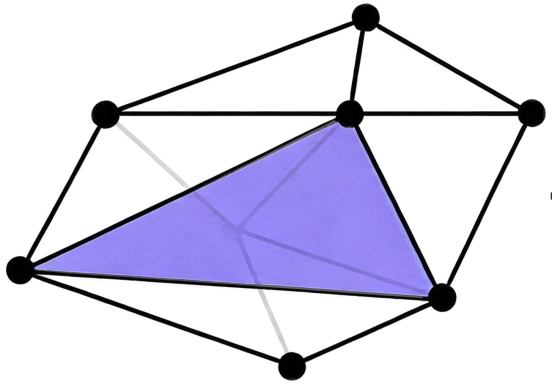


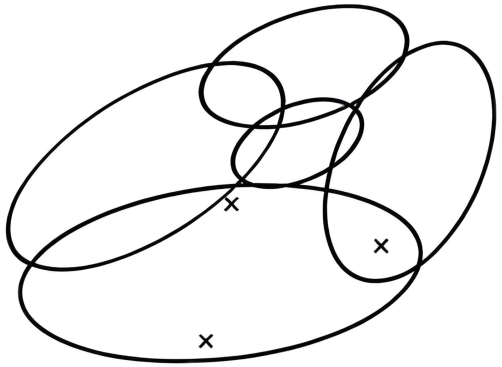
# 3. Static 3D Gaussian Initialization

腾讯混元3D

## 3.2 Mesh2Splat Initial 3D Static Mesh → Initial 3D Static Gaussian



Mesh (Connected Triangular Faces)



GS (Independent Ellipsoids)

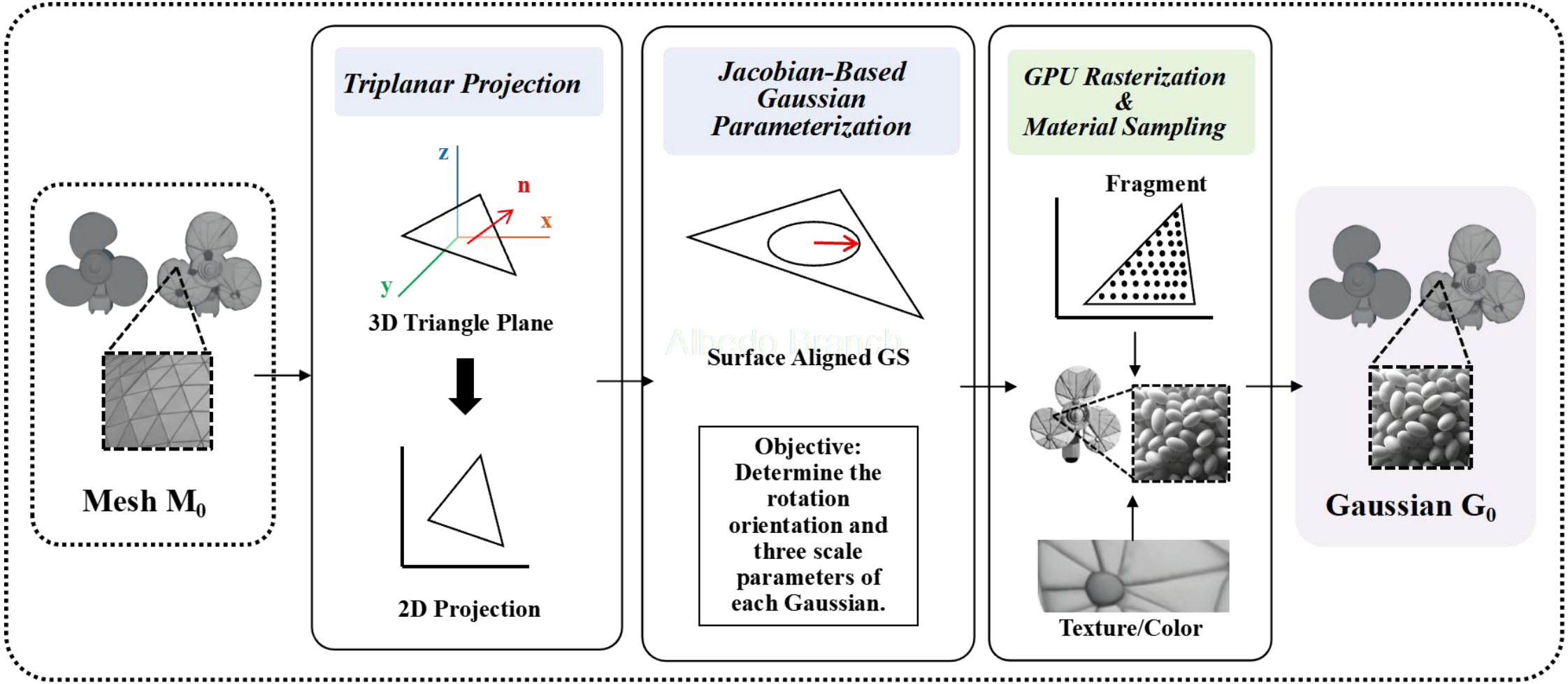
### Why Mesh → Gaussian ?

1. Flexible Gaussian Optimization
2. Efficient Local Refinement
3. Fast Differentiable Rendering

| Mesh | 3D Gaussian Splatting |
|---|---|
| Vertex geometry, topology, and material parameters | Independently optimizable position, rotation, scale, opacity, and color |
| Often requires vertex insertion and mesh remeshing | Direct Gaussian densification and targeted region completion |
| Typically requires specialized differentiable rasterization | Fast, native differentiable splatting |

# 3. Static 3D Gaussian Initialization

## 3.2 Mesh2Splat




Mesh2Splat

# 3. Static 3D Gaussian Initialization

腾讯混元3D

## 3.2 Mesh2Splat

Lower than 0.05% Lower than 0.003

| Model | BBox Relative Error (%) ↓ | NCD ↓ | Normalized Completeness ↓ | Recall@0.2% ↑ |
|---|---|---|---|---|
| Arm | 0.0493% | 0.001817 | 0.000589 | 100.000% |
| Fan | 0.0147% | 0.001881 | 0.000577 | 99.999% |
| Gears | 0.0052% | 0.002195 | 0.000606 | 99.999% |
| Lego | 0.0102% | 0.002967 | 0.000552 | 100.000% |
| Robot | 0.0127% | 0.001857 | 0.000541 | 100.000% |
| lamp | 0.0047% | 0.001913 | 0.000551 | 100.000% |
| metronome | 0.0290% | 0.002131 | 0.000546 | 100.000% |
| micphone | 0.0038% | 0.001474 | 0.000505 | 100.000% |
| rotation | 0.0027% | 0.002431 | 0.000546 | 99.999% |

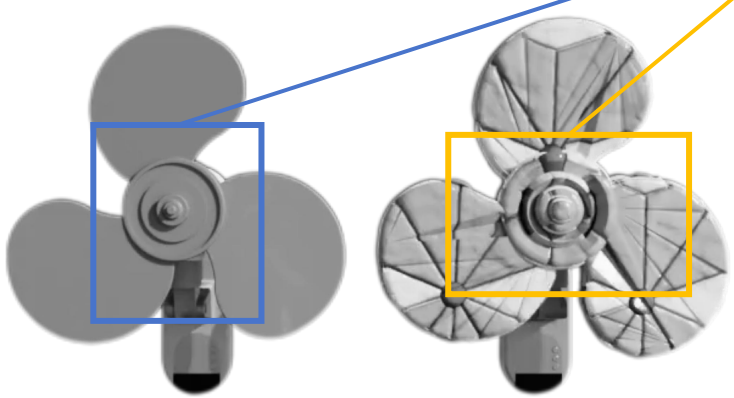
1. BBox Relative Error (%) ↓: Measures the relative size difference between Mesh and GS bounding boxes.
2. NCD ↓: Measures the overall geometric difference between Mesh and GS.
3. Normalized Completeness ↓: Measures how closely GS covers the reference Mesh surface.
4. Recall@0.2% ↑: Percentage of Mesh surface samples covered by GS within a 0.2% distance threshold.

# 3. Static 3D Gaussian Initialization

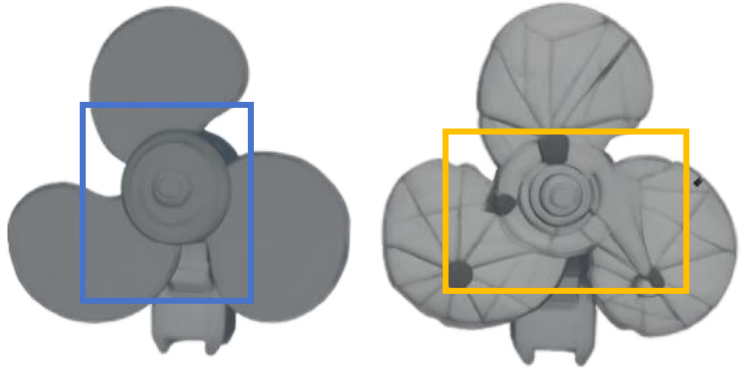
腾讯混元3D

**Limitations of Hunyuan3D + Mesh2Splat Initialization :**

1. Appearance Inconsistency (Color, Texture, Lighting...)
2. Geometric Inconsistency (Silhouette, Structure, Excess Geometry...)
3. Camera Misalignment (Coordinate System Mismatch)



Reference Image



Static 3D Gaussian $G_0$

(Lack Camera Extrinsic)

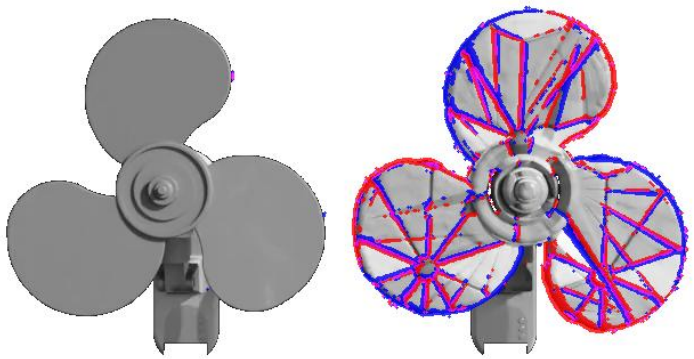
# 4. CARE-GS

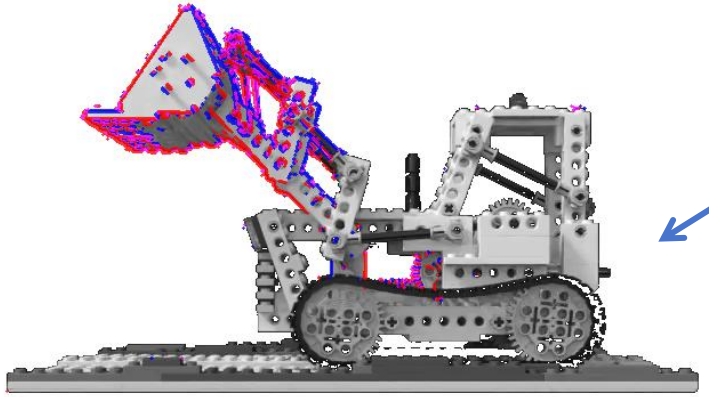
## Camera Alignment, Gaussian Refinement and Continuity Enhancement

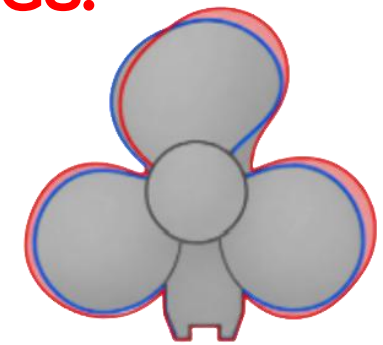
## Why Is Fixed-View Consistency Important for 3DGS Initialization?

1. Reliable Event-to-Gaussian Alignment
2. Accurate 2D–3D Motion Association

If consistency is poor, event boundaries may fail to align with the corresponding 3DGS.







**WE NEED**

**Camera Alignment**

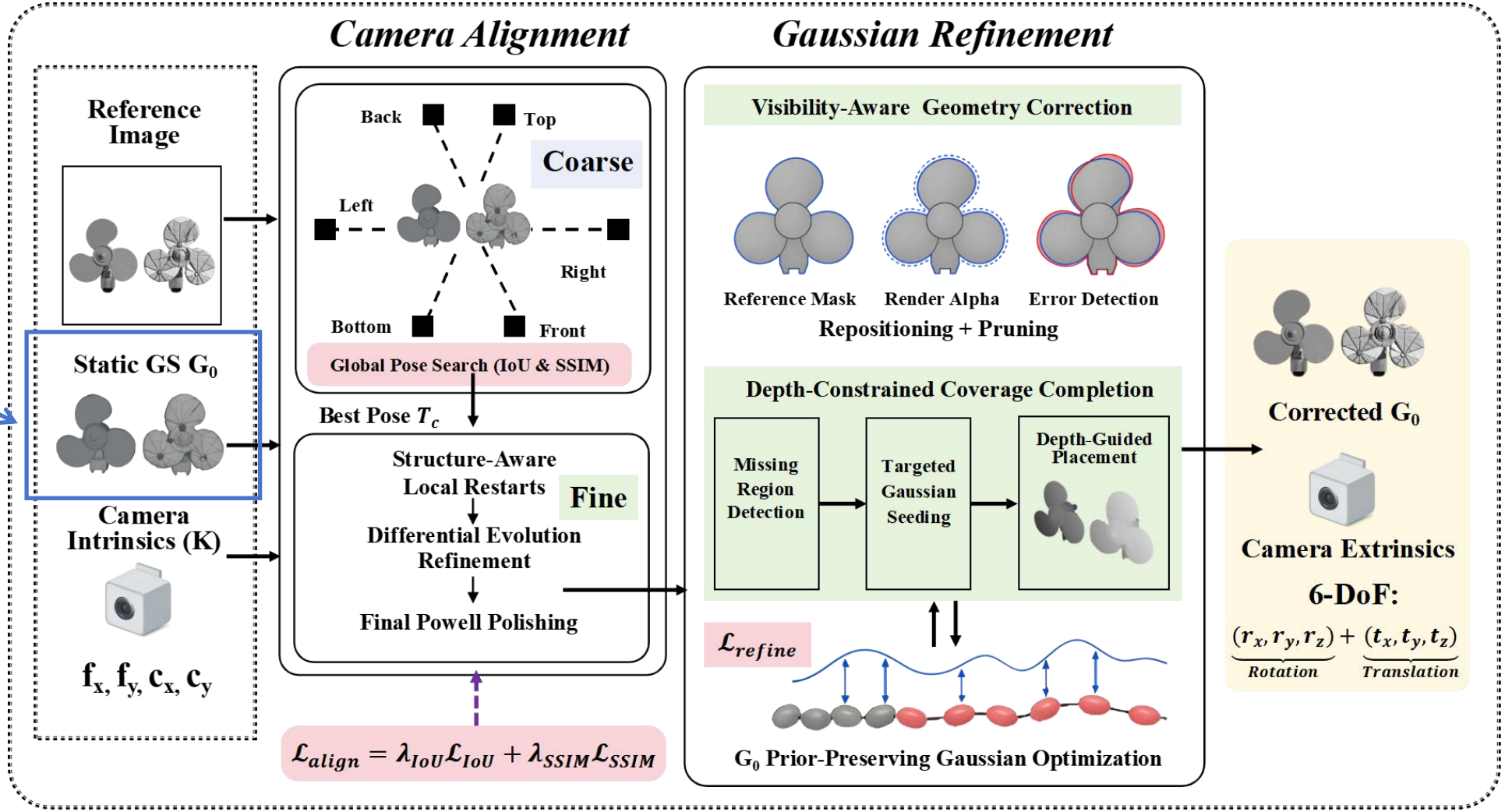
**Gaussian Refinement**

Visualization of Event Streams Overlaid on RGB Images: Fan (Left) and LEGO (Right).

# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Comes from Hunyuan + Mesh2Splat

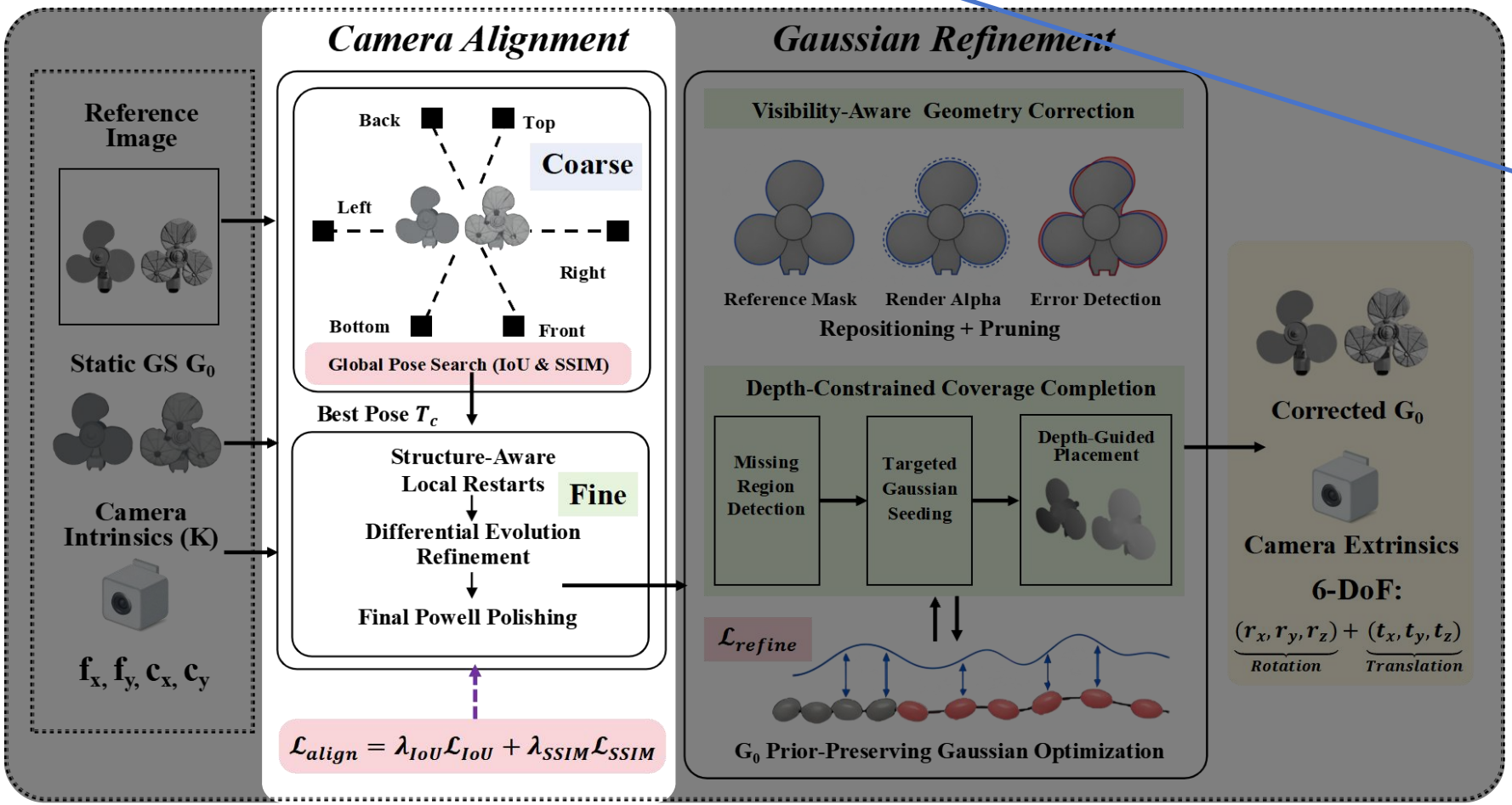


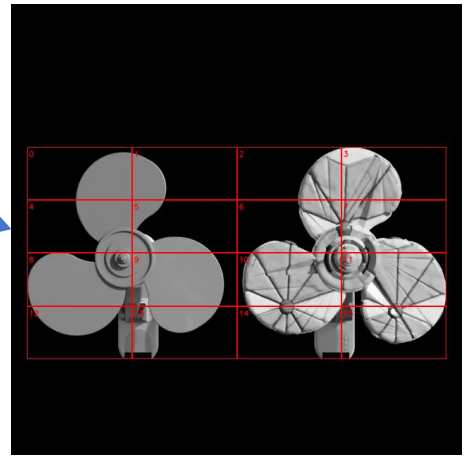

CARE-GS

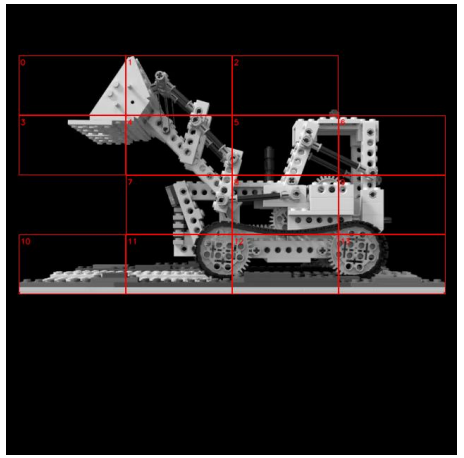
# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

$$\mathcal{L}_{align} = \lambda_1 \mathcal{L}_{IoU}^{global} + \lambda_2 \mathcal{L}_{IoU}^{region} + \lambda_3 \mathcal{L}_{missing} + \lambda_4 \mathcal{L}_{SSIM}^{global} + \lambda_5 \mathcal{L}_{SSIM}^{worst}$$

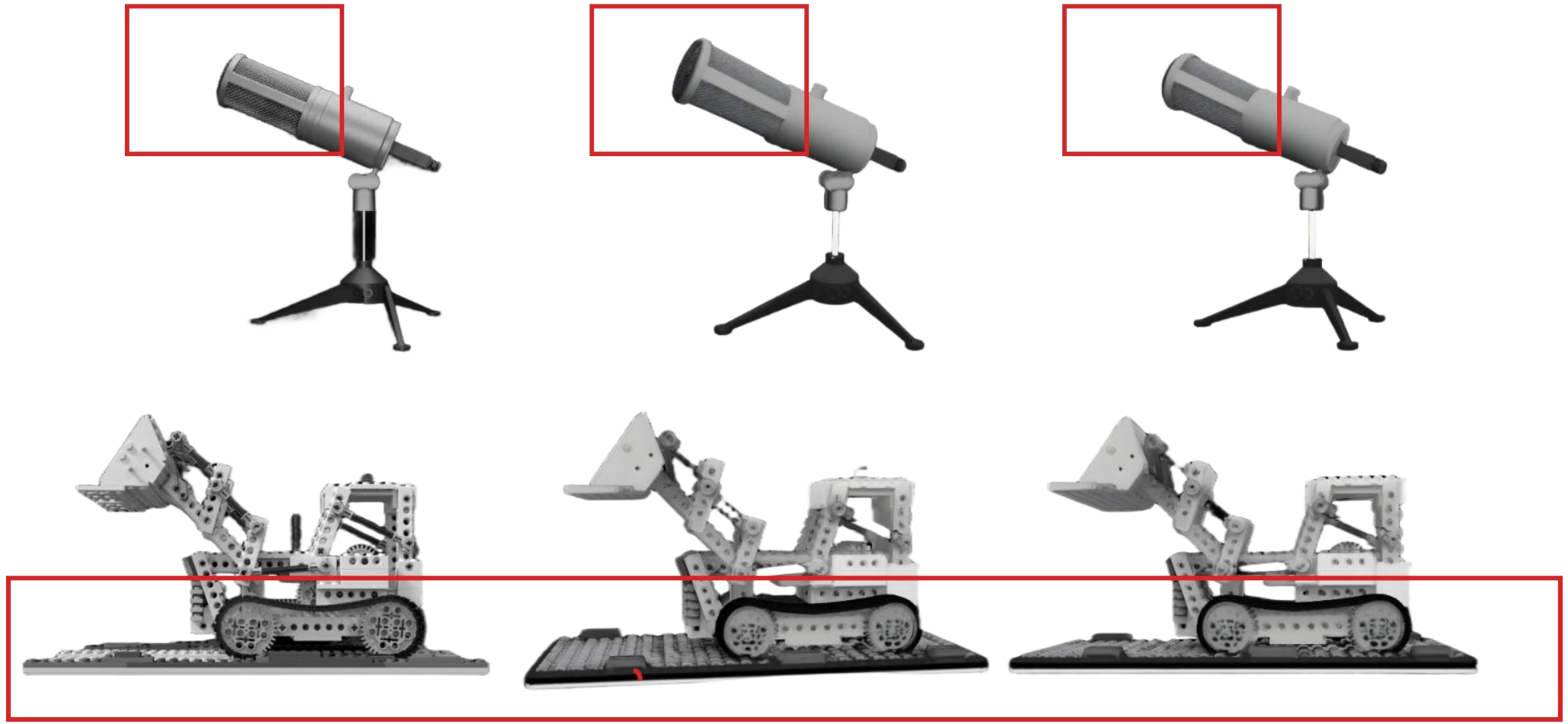




# 4. CARE-GS

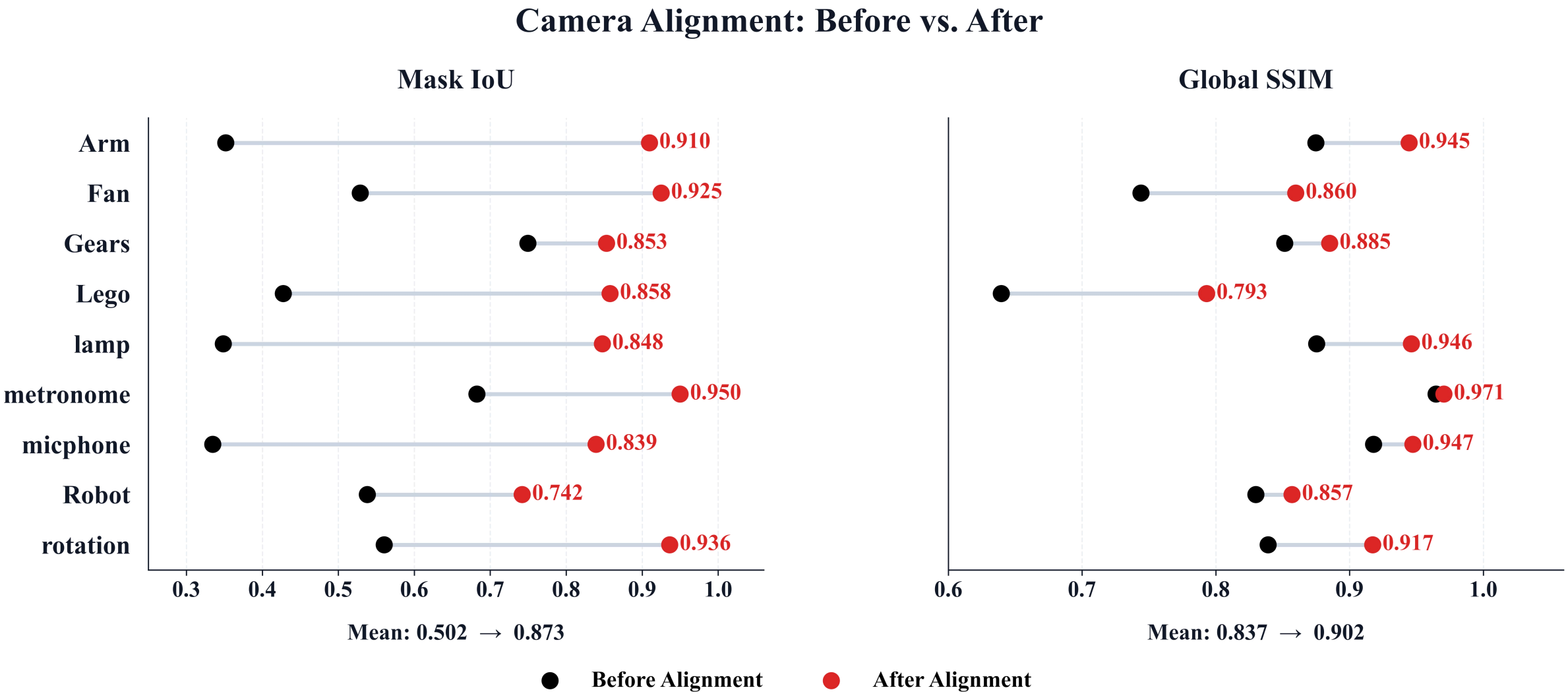
## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement



Reference Image  Course Camera Alignment  Fine Camera Alignment

# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement



# 4. CARE-GS

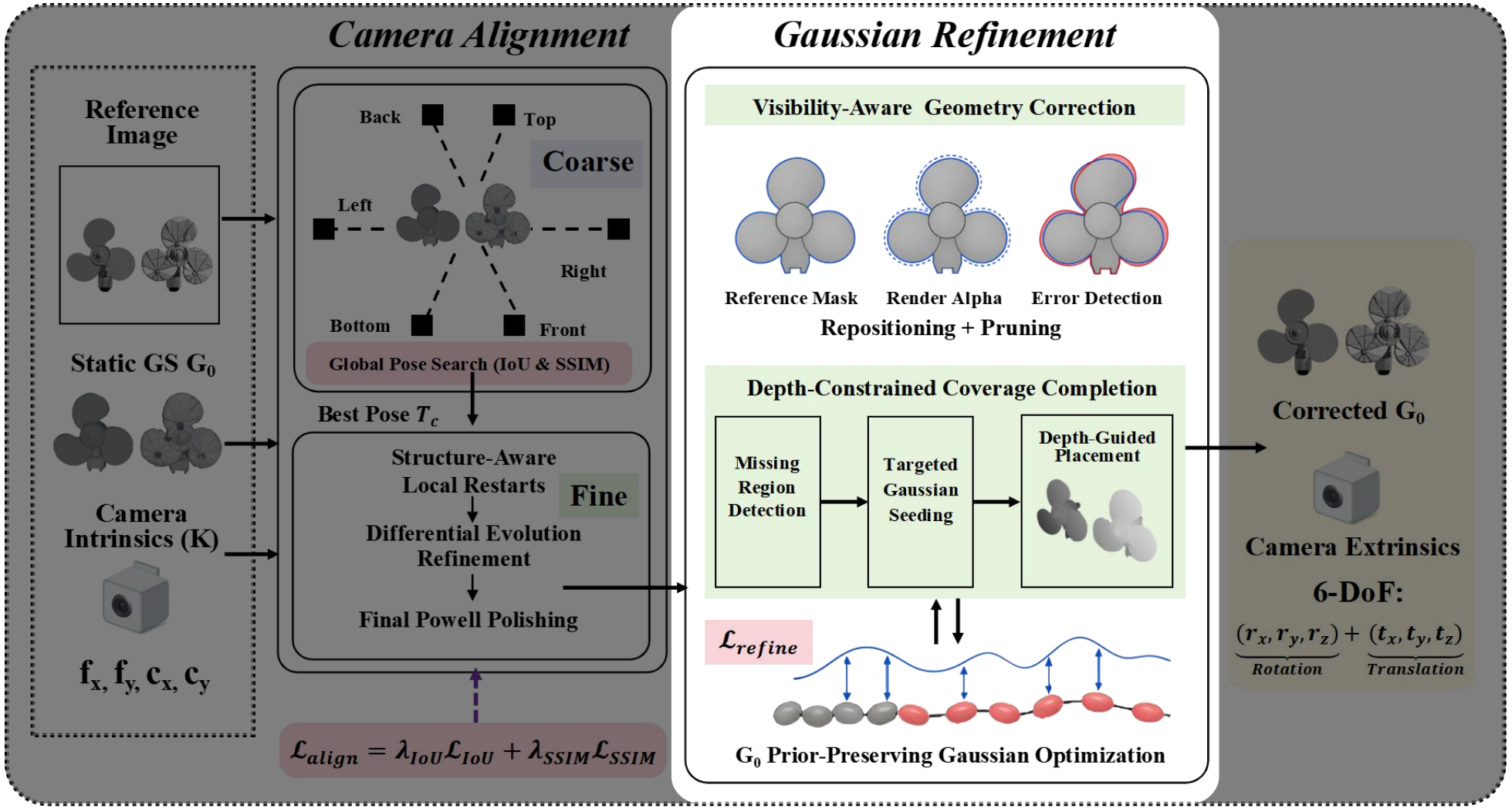
## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

| Model | Mask IoU ↑ | Global SSIM ↑ |
|---|---|---|
| Arm | 0.3518 → 0.9097 (+158.58%) | 0.8748 → 0.9445 (+7.97%) |
| Fan | 0.5289 → 0.9251 (+74.91%) | 0.7441 → 0.8598 (+15.55%) |
| Gears | 0.7496 → 0.8532 (+13.82%) | 0.8515 → 0.8851 (+3.95%) |
| Lego | 0.4276 → 0.8578 (+100.61%) | 0.6396 → 0.7932 (+24.02%) |
| Robot | 0.5382 → 0.7421 (+37.89%) | 0.8300 → 0.8570 (+3.25%) |
| lamp | 0.3486 → 0.8477 (+143.17%) | 0.8754 → 0.9462 (+8.09%) |
| metronome | 0.6825 → 0.9500 (+39.19%) | 0.9647 → 0.9705 (+0.60%) |
| micphone | 0.3348 → 0.8394 (+150.72%) | 0.9180 → 0.9473 (+3.19%) |
| rotation | 0.5604 → 0.9363 (+67.08%) | 0.8391 → 0.9173 (+9.32%) |

**Relatively Low because the initial G0 differs geometrically and visually from the original RGB image.**

# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement



# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

$$\mathcal{L}_{refine} = \lambda_{rgb}\mathcal{L}_{RGB} + \lambda_{mask}\mathcal{L}_{mask} + \lambda_{anchor}\mathcal{L}_{anchor} + \lambda_{depth}\mathcal{L}_{depth} + \lambda_{cov}\mathcal{L}_{coverage} + \lambda_{local}\mathcal{L}_{local} + \lambda_{surface}\mathcal{L}_{surface}$$

| Loss | Purpose |
| --- | --- |
| RGB Reconstruction Loss | Improves visual **appearance and color** consistency. |
| Mask Consistency Loss | Refines object silhouettes and reduces background overflow. |
| Depth Consistency Loss | Preserves **depth consistency** and minimizes geometric distortions. |
| Geometry Preservation Loss | Constrains geometric deviations and **preserves the initial $G_0$ structure**. |

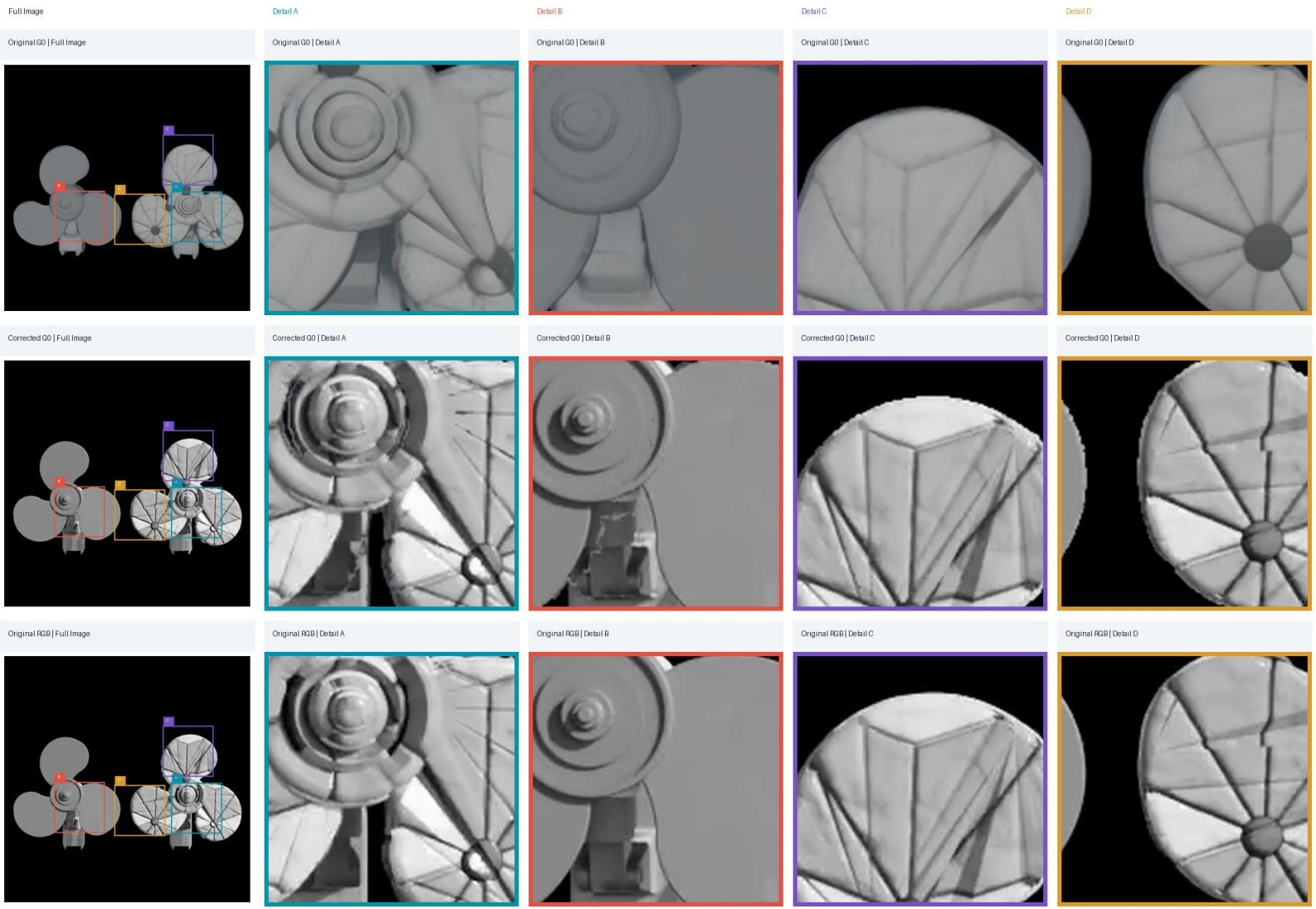
# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Original G0

Corrected G0

Reference Image




**Enhance the Appearance of 3DGS**

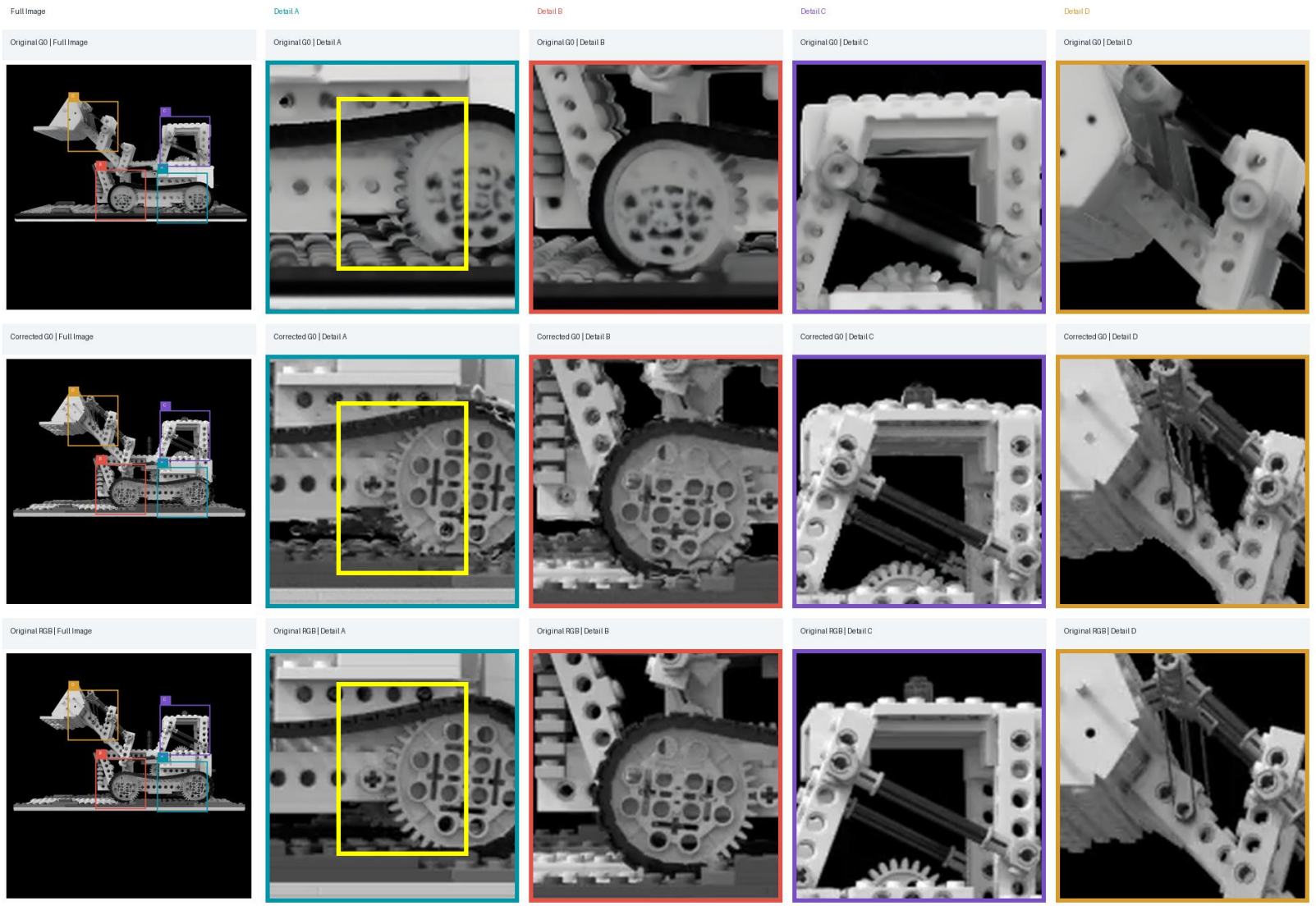
# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Original G0

Corrected G0

Reference Image




**Enhance the Appearance of 3DGS**

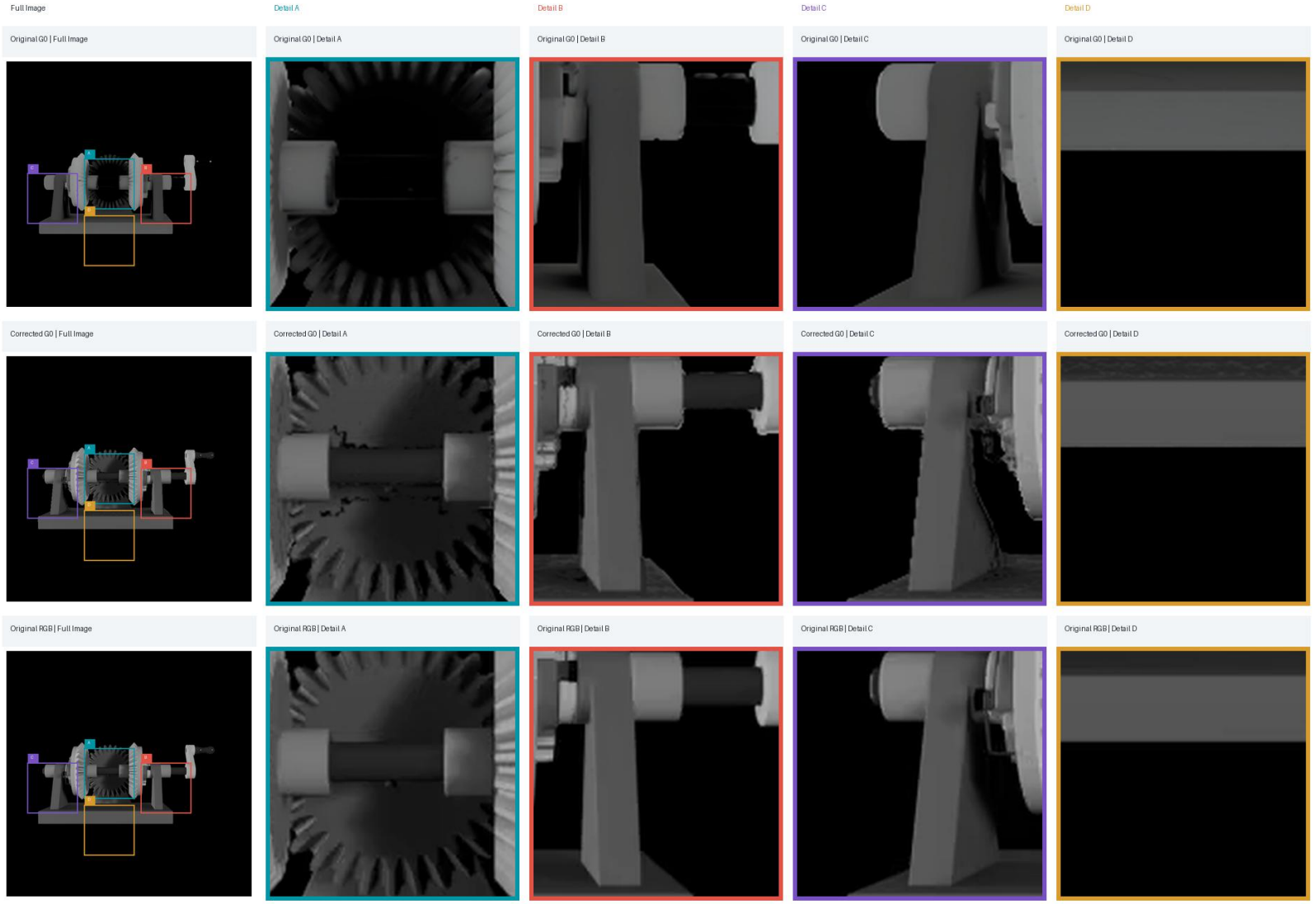
# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Original G0

Corrected G0

Reference Image




**Enhance the Appearance of 3DGS**

# 4. CARE-GS

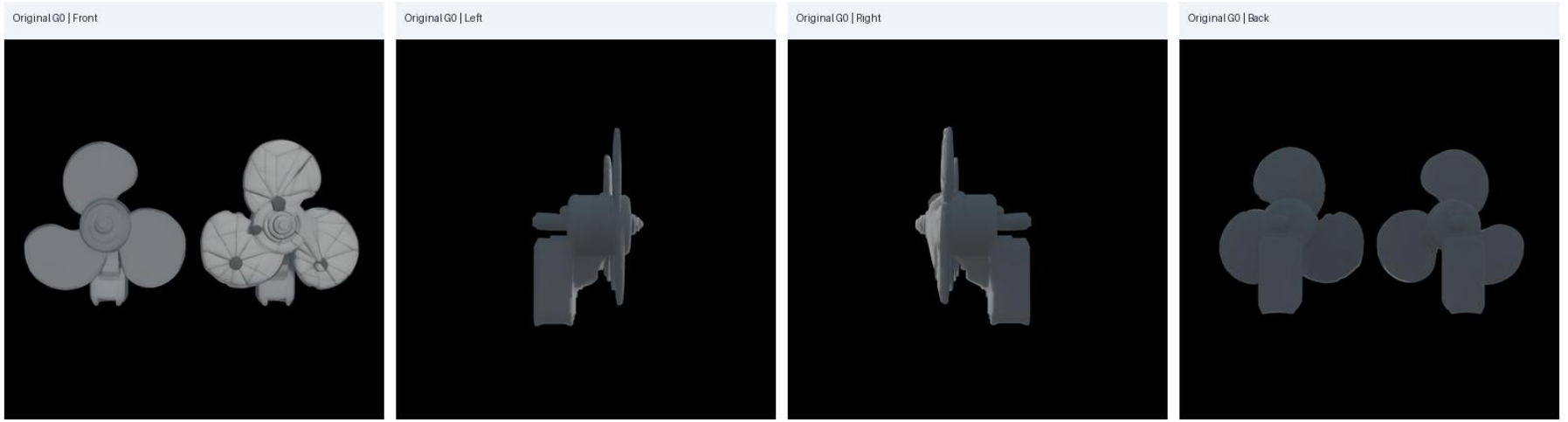
## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

| Model | PSNR (dB) ↑ | SSIM ↑ | LPIPS ↓ |
|---|---|---|---|
| Arm | 18.068 → 29.045 (+60.8%) | 0.9585 → 0.9930 (+3.6%) | 0.0482 → 0.0192 (−60.2%) |
| Fan | 15.465 → 27.715 (+79.2%) | 0.8876 → 0.9904 (+11.6%) | 0.0976 → 0.0204 (−79.1%) |
| Gears | 17.450 → 26.960 (+54.5%) | 0.9258 → 0.9845 (+6.3%) | 0.0616 → 0.0272 (−55.8%) |
| Lego | 12.532 → 22.461 (+79.2%) | 0.8393 → 0.9761 (+16.3%) | 0.1116 → 0.0363 (−67.5%) |
| Robot | 8.978 → 21.310 (+137.4%) | 0.8788 → 0.9835 (+11.9%) | 0.1371 → 0.0214 (−84.4%) |
| lamp | 17.390 → 24.385 (+40.2%) | 0.9693 → 0.9921 (+2.4%) | 0.0445 → 0.0242 (−45.6%) |
| metronome | 21.808 → 32.316 (+48.2%) | 0.9719 → 0.9959 (+2.5%) | 0.0506 → 0.0163 (−67.8%) |
| micphone | 17.806 → 25.041 (+40.6%) | 0.9691 → 0.9953 (+2.7%) | 0.0389 → 0.0131 (−66.3%) |
| rotation | 19.207 → 28.920 (+50.6%) | 0.9325 → 0.9910 (+6.3%) | 0.0553 → 0.0199 (−64.0%) |

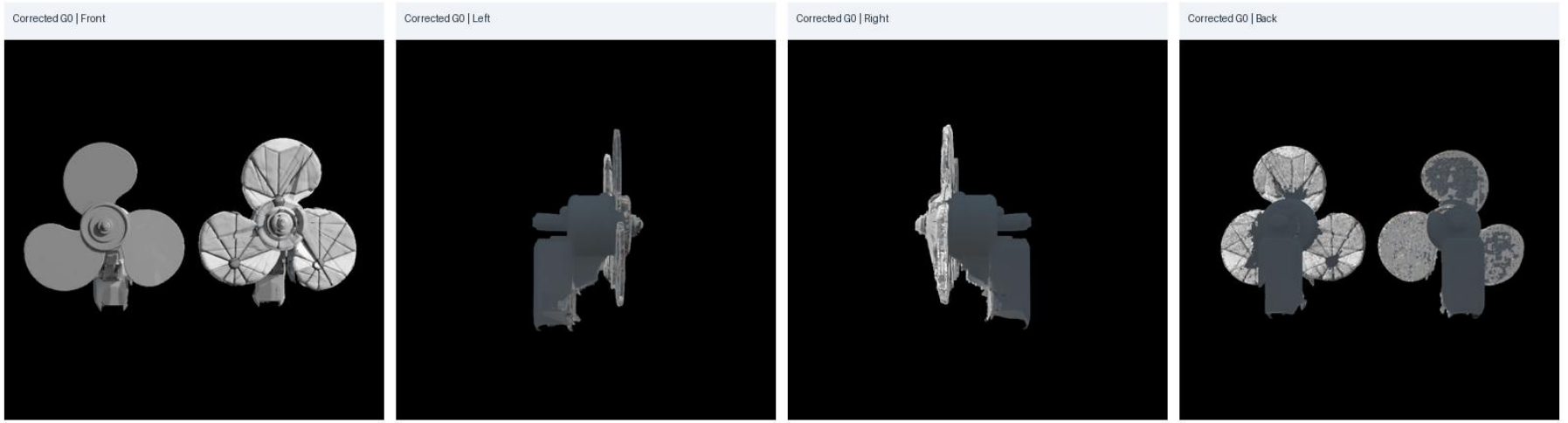
# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Original G0




Corrected G0



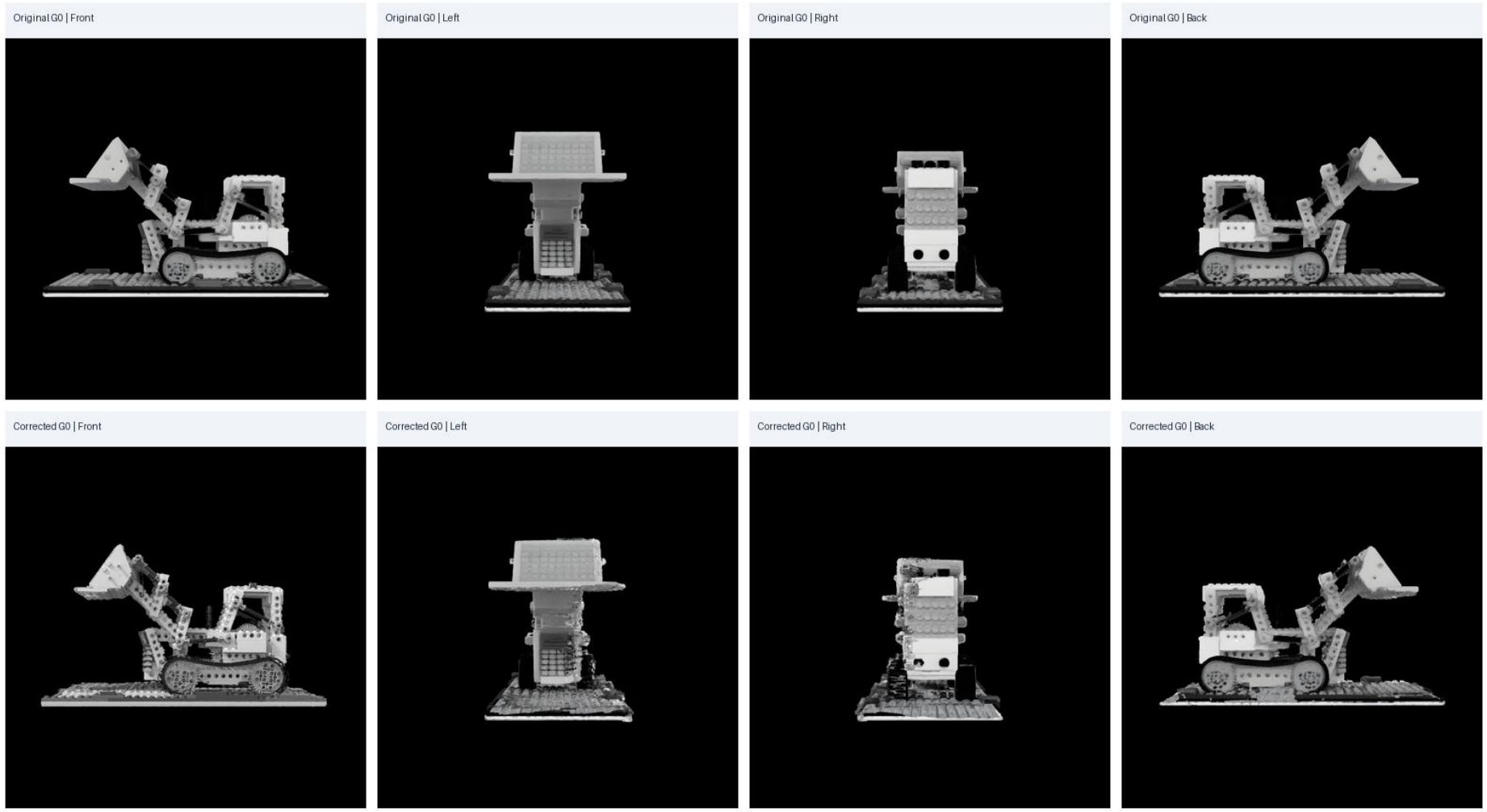

**Preserved Overall Model Integrity and $G_0$ Anchors**

# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Original G0

Corrected G0



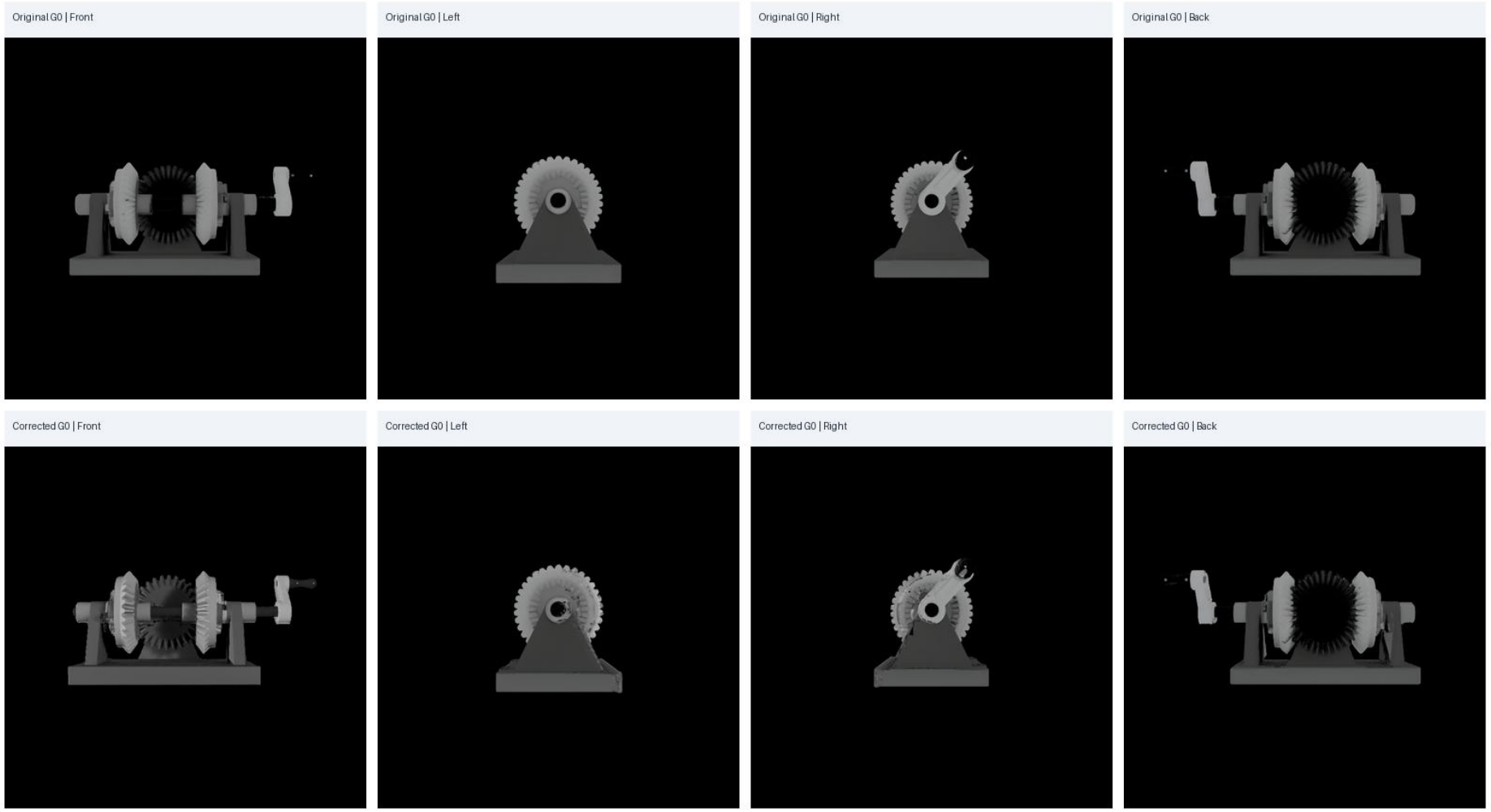

**Preserved Overall Model Integrity and $G_0$ Anchors**

# 4. CARE-GS

## Camera Alignment, Gaussian Reconstruction and Continuity Enhancement

Original G0

Corrected G0




**Preserved Overall Model Integrity and $G_0$ Anchors**

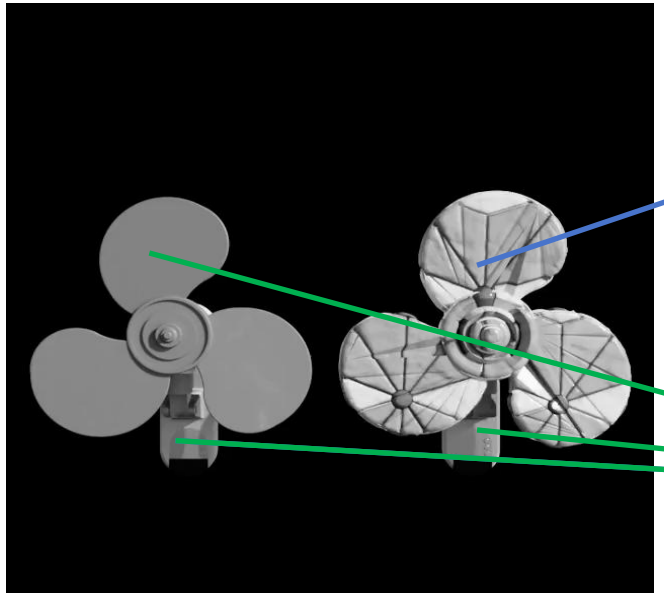
# 6. FEAST

**Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours**

## Why Do We Need Dynamic–Static Separation?

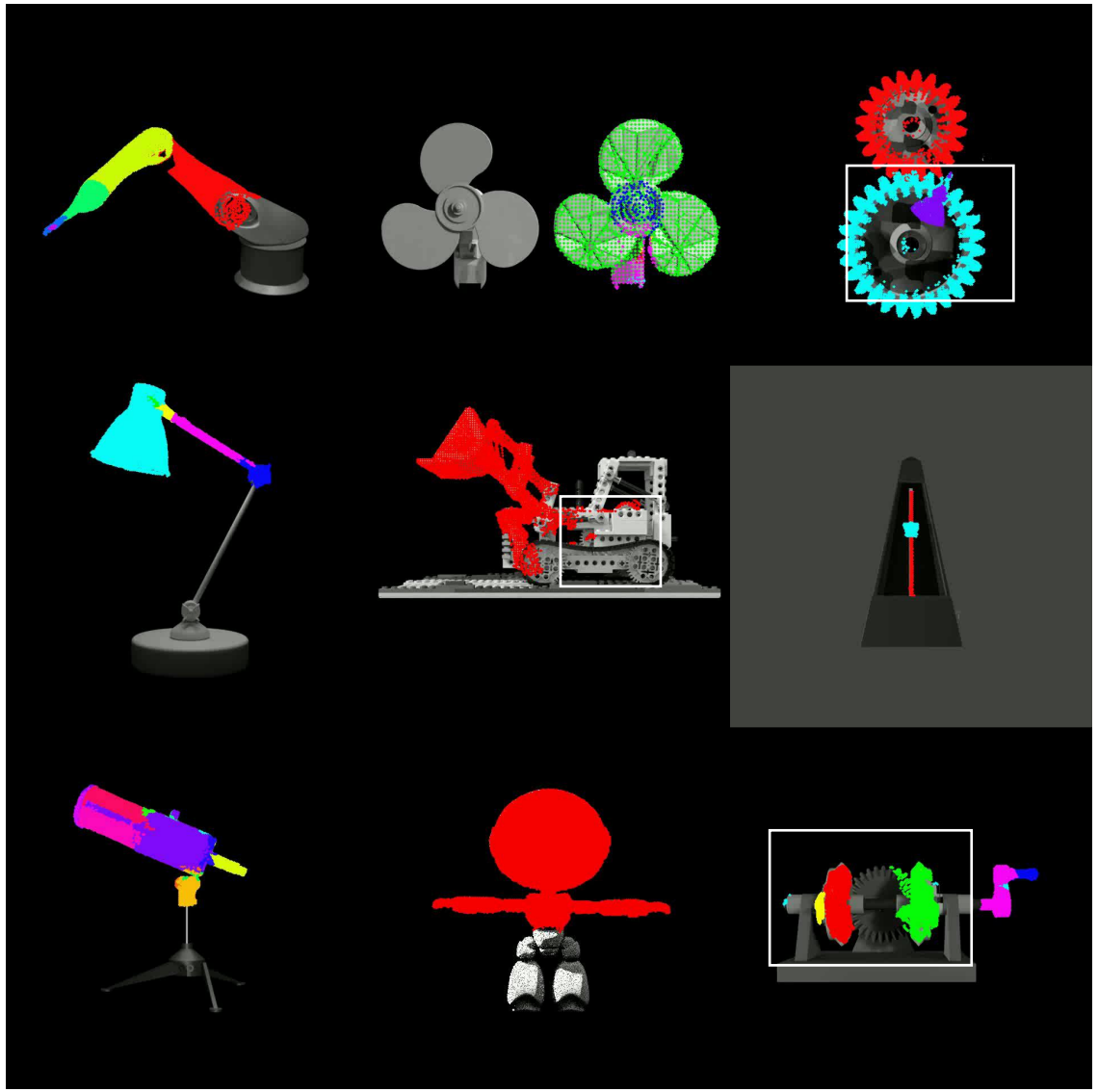
**1. Enable Shared Rigid Motion:** Identify dynamic parts so that Gaussians within each rigid component can share a common SE(3) transformation.

**2. Prevent Unnecessary Deformation:** Keep static regions fixed to reduce motion ambiguity, unnecessary optimization, and geometric distortion.



# 6. FEAST

## Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours



## First Version: Cotracker3

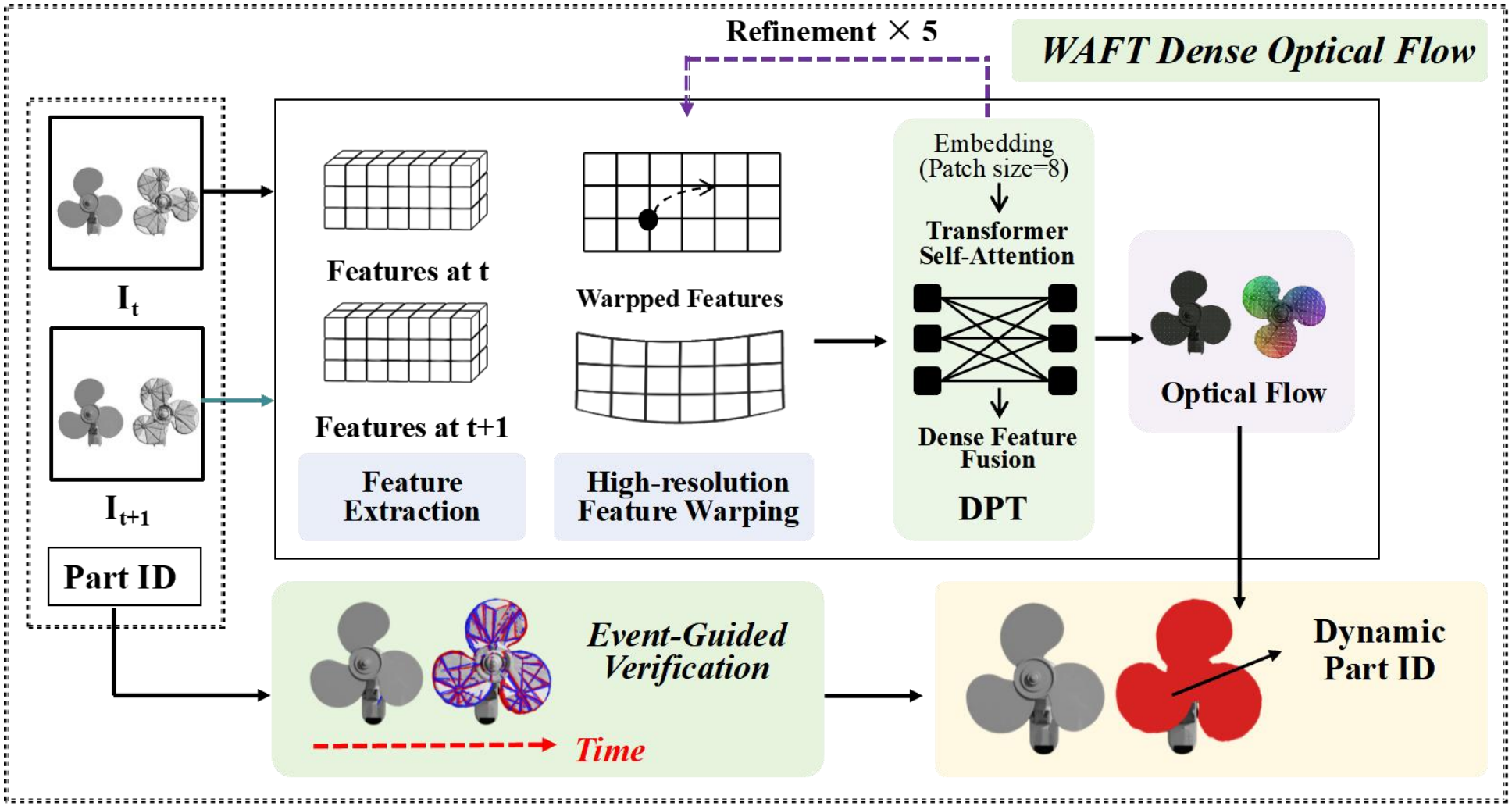
**1. Sparse Spatial Coverage:** Tracking points may not sufficiently cover all moving regions.

**2. Incomplete Motion Detection:** Small or weakly moving parts may lack sufficient valid tracking trajectories.

**3. Loss of Visible Correspondences:** Tracking points become occluded as the object rotates or other motion, limiting continuous motion tracking.

# 6. FEAST

## Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours




FEAST

Part ID from PartField

↓

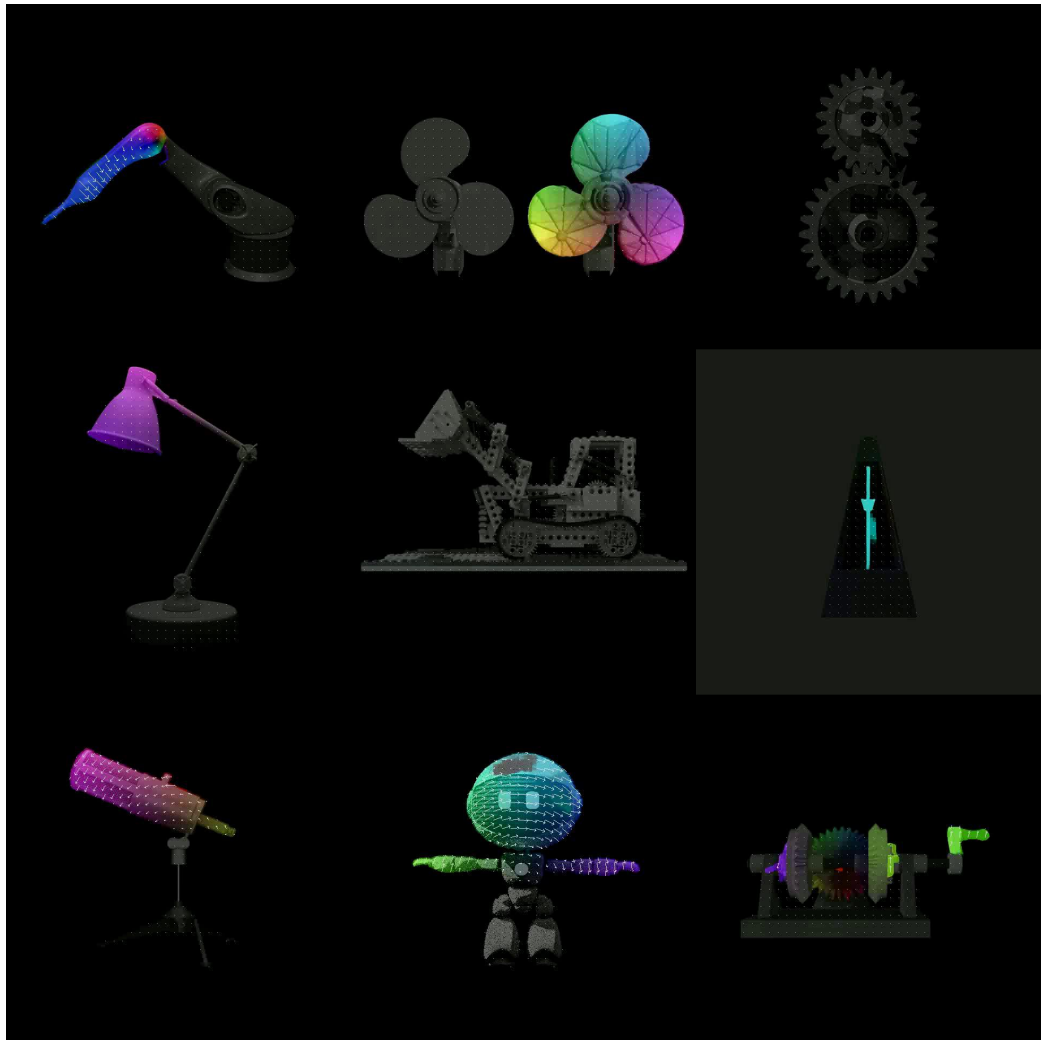
FEAST

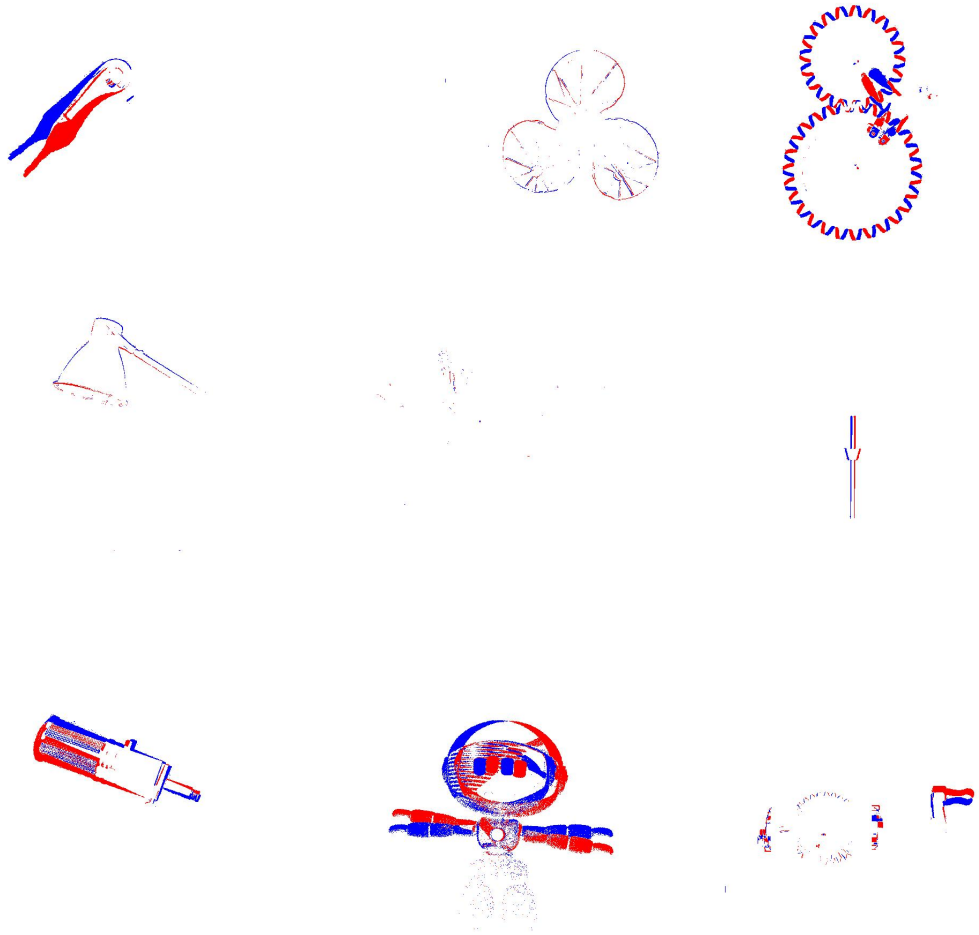
↓

Dynamic Part ID

# 6. FEAST

## Flow and Event-Assisted Dynamic-Static Separation via Temporal Contours

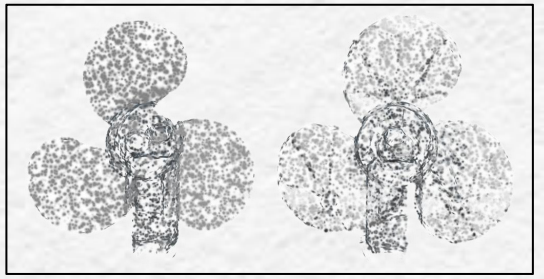


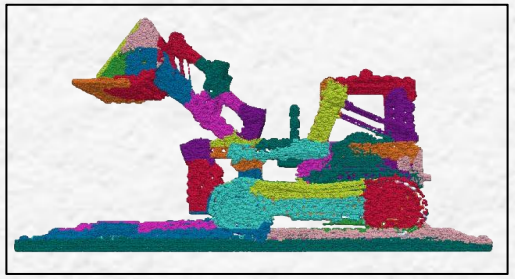


# What we have ?

- After previous work, we can have somethings:

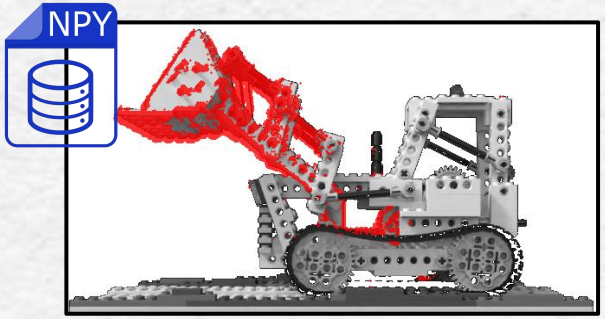
✓ Initial 3D property source → Correcrted Gaussian $\mathcal{G}_0$

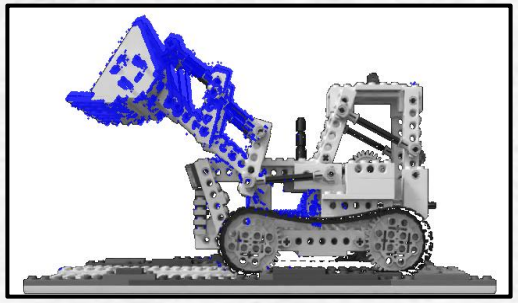




✓ Video Sampled Frames → Supervised Optimization

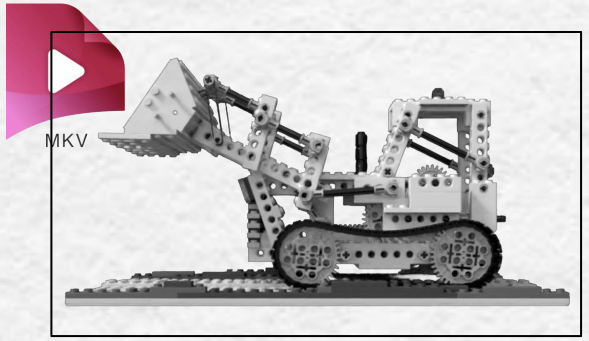
✓ Camera intrinsic/extrinsic → Locating

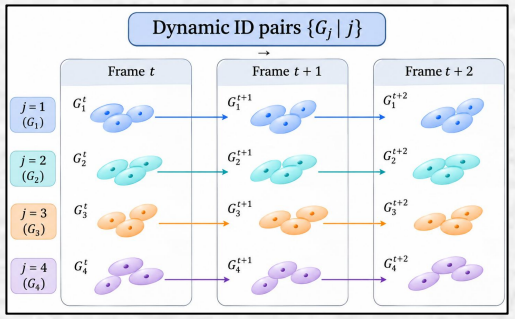


NEW Segemented parts $\{i, G_j\}$

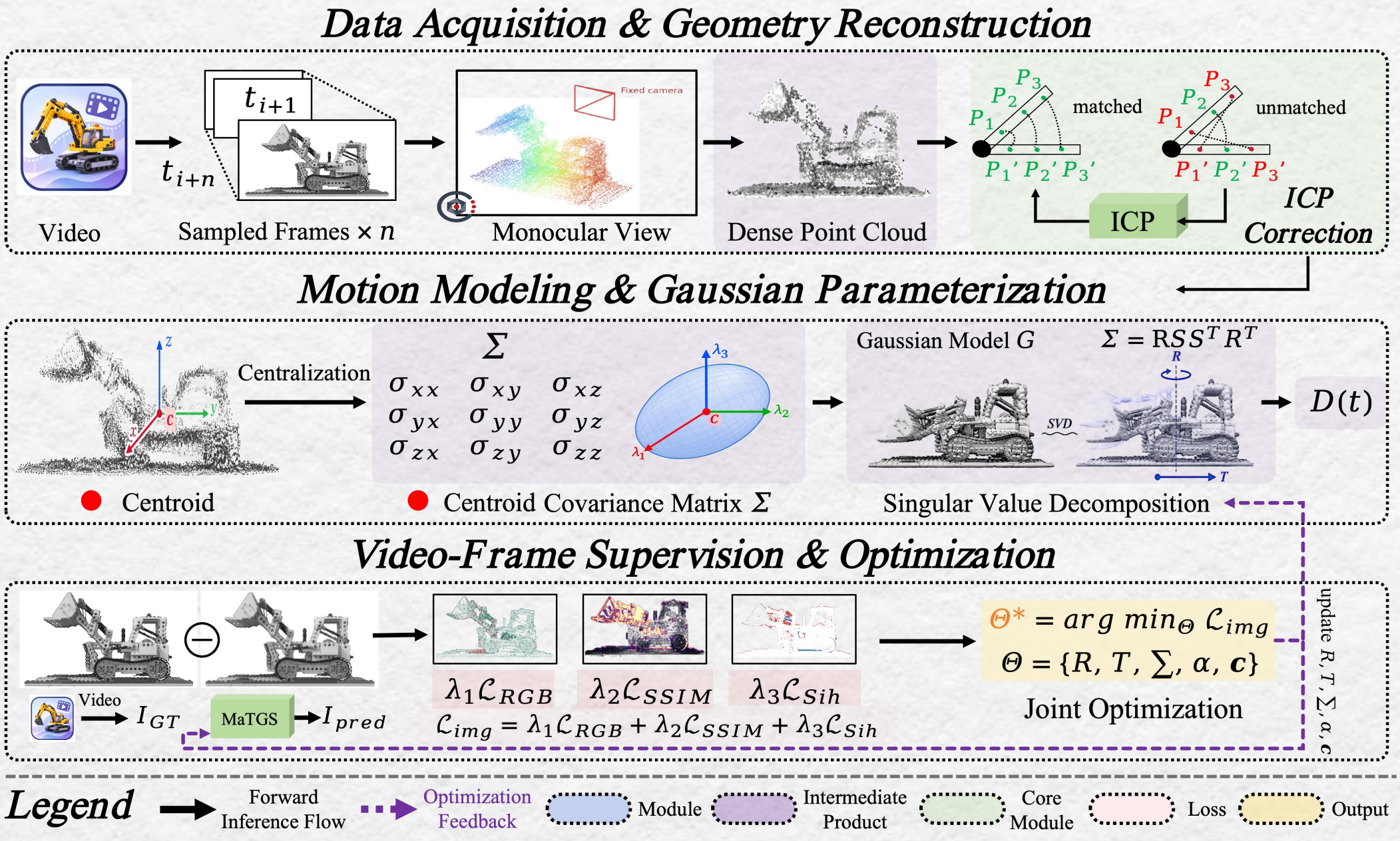
NEW Dynamic ID pairs $\{G_j \mid j\}$




NEW Event Stream $E_t$ $\{\pm 1\}$

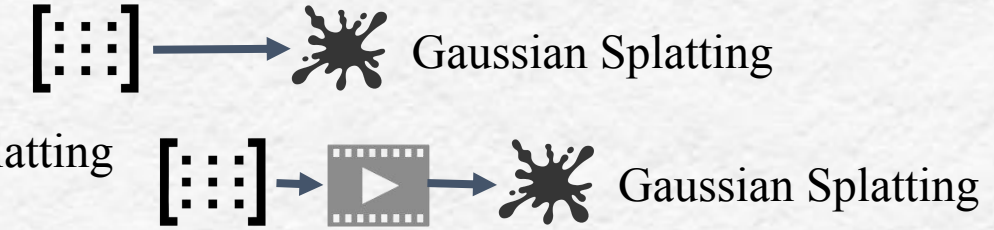
# Architecture of MaTGS & MonoGS



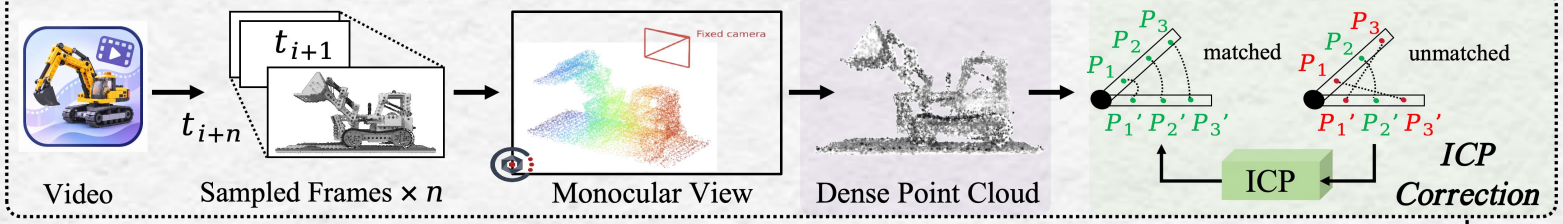
# Problem of MaTGS & MonoGS

***MaTGS***: Matrix-Transformed 3D Gaussian Splatting (MaTGS)

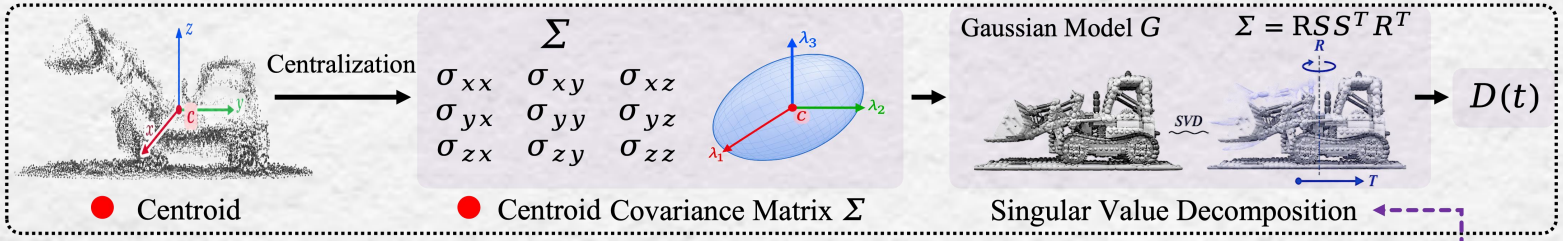
***MonoGS***: Monocularly Supervised Matrix-driven 3D Gaussian Splatting




## *Data Acquisition & Geometry Reconstruction*




## *Motion Modeling & Gaussian Parameterization*




## *Video-Frame Supervision & Optimization*

Initial 3D property source ✓ → Correcrted Gaussian $\mathcal{G}_0$

Video Sampled Frames ✓ → Supervised Optimization

Camera intrinsic/extrinsic ✓ → Locating

- COLMAP + ICP Match
- Kabsch + SVD → $D(t)$
- Gaussian Motion
- Rendering
- Joint Optimization

***Legend*** Forward Inference Flow; Optimization Feedback; Module; Intermediate Product; Core Module; Loss; Output

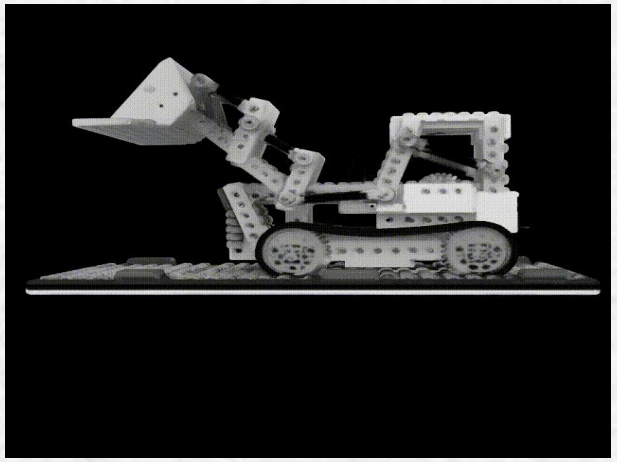
# Problem of MaTGS & MonoGS

## Traditional motion estimation mechanism in *MaTGS*

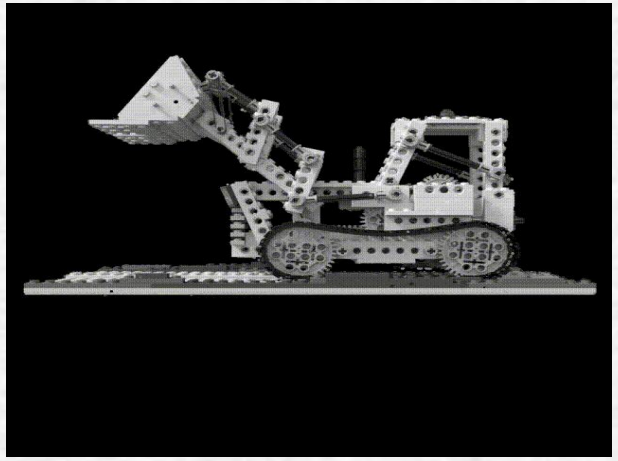
***MaTGS*** primarily employs ICP method combined with the Kabsch algorithm to estimate rigid-body transformation matrices to drive the motion of 3DGS.

COLMAP + ICP Match → Kabsch + SVD → $D(t)$
→ Gaussian Motion → Rendering → Joint Optim

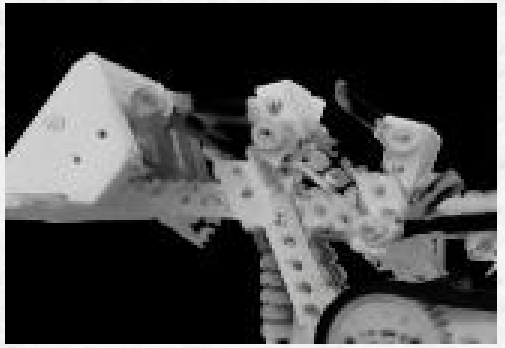
## Problem Preview → The "Broken Arm" Problem



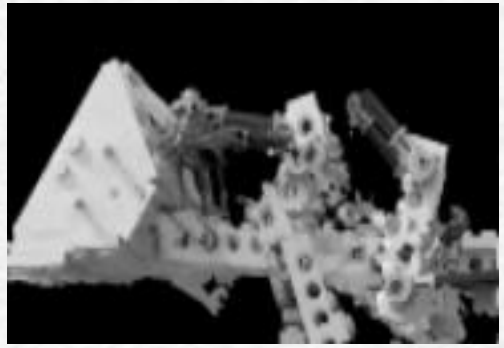
MaTGS



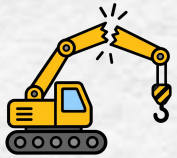
MonoGS



MaTGS



MonoGS



!

Structural **Broken Arm**

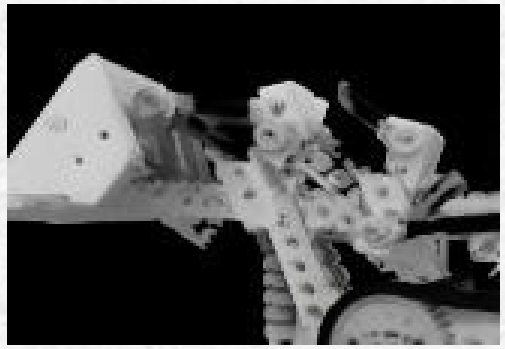
Poor visual effect

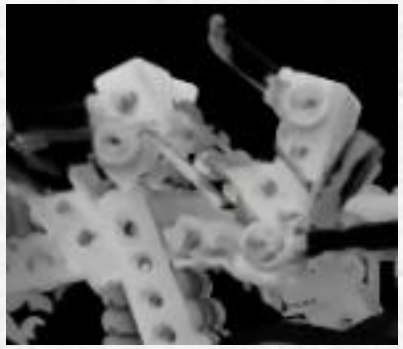
Structural **Broken Arm**

Better visual effect

# "Broken Arm" of MaTGS & MonoGS

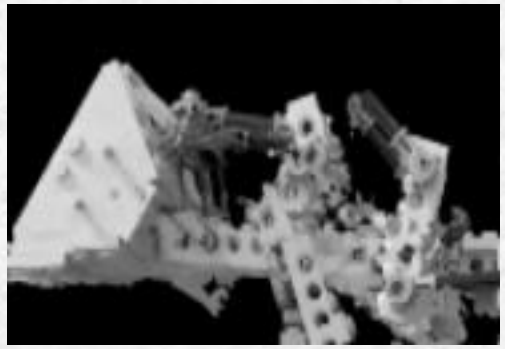
MaTGS





Add Video
Visual Supervision

MonoGS



From MaTGS → MonoGS

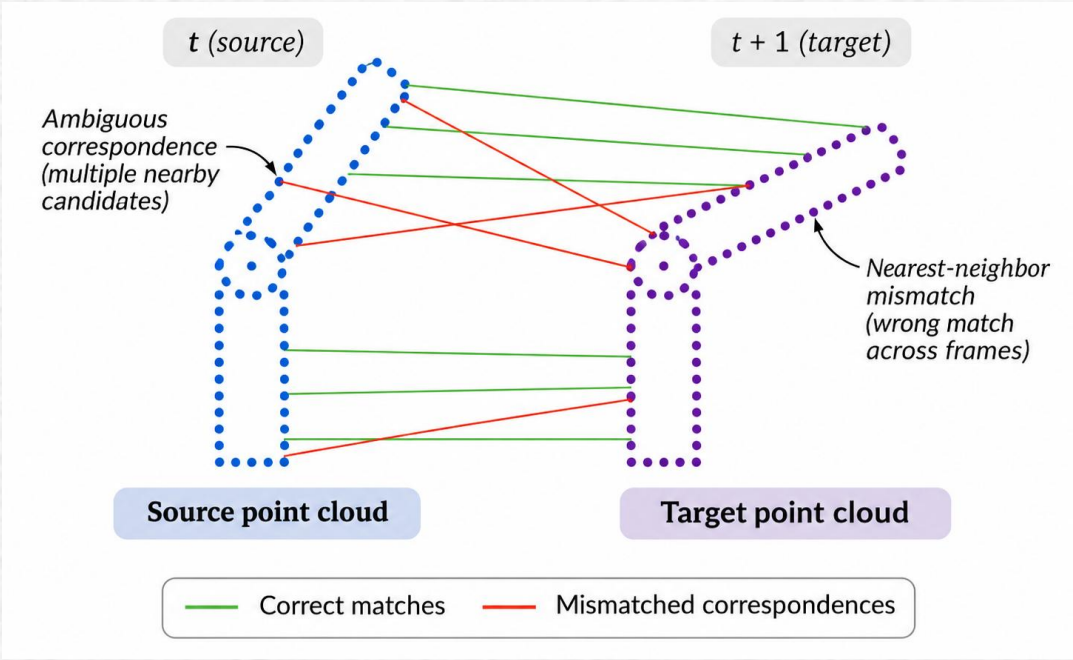
***Visual Effect:*** Better ✓

***Structural Broken Arm:*** Still Same ! ✗

## Why?

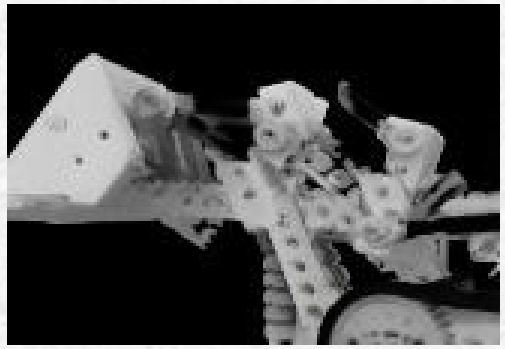
(1) Point cloud correspondence errors (ICP)

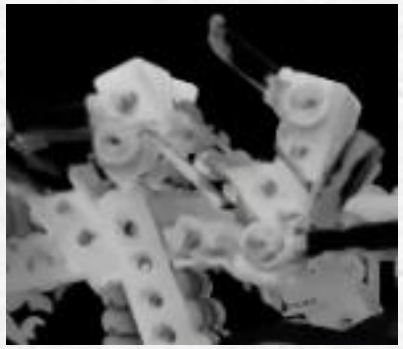
- nearest-neighbor search
- **occlusion and partial overlap** mismatches
- no guarantee corresponding points belong to the **same physical component.**
- degraded rotation and translation estimation



# "Broken Arm" of MaTGS & MonoGS

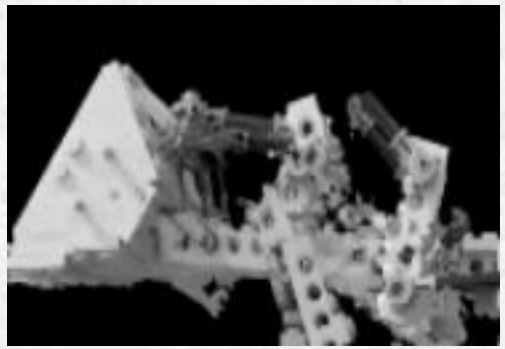
MaTGS





Add Video
Visual Supervision

MonoGS



From MaTGS → MonoGS

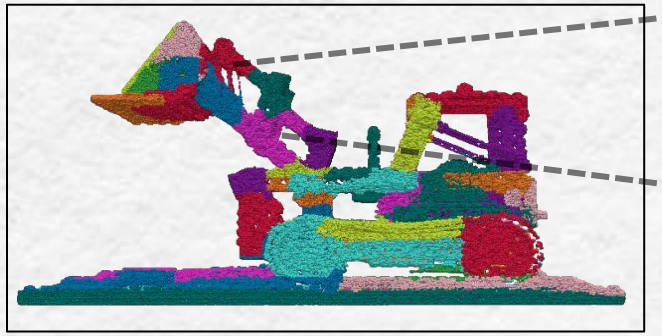
***Visual Effect:*** Better ✓

***Structural Broken Arm:*** Still Same ! ✗

## Why?

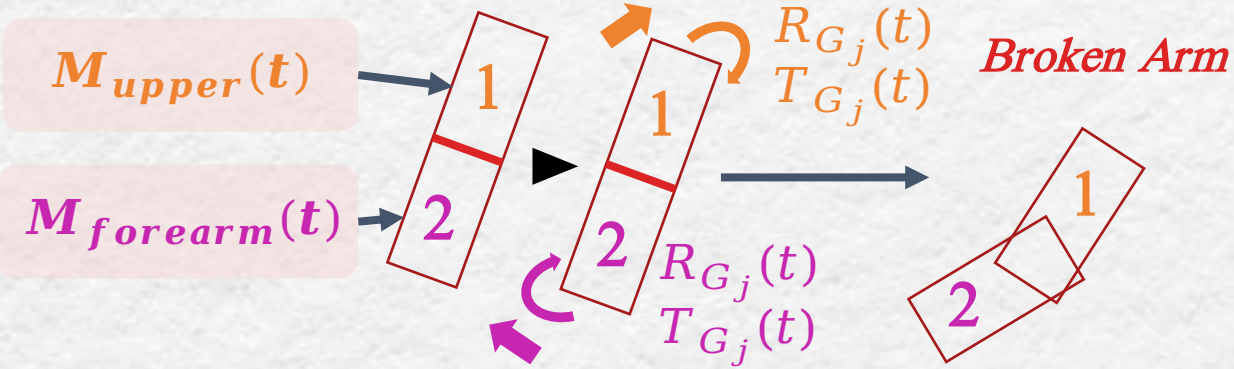
(2) Different components are subjected to independent rigid-body transformations

- original **spatial connectivity** between them is **not necessarily preserved**.
- especially for **Wrong Parts-Decomposition**



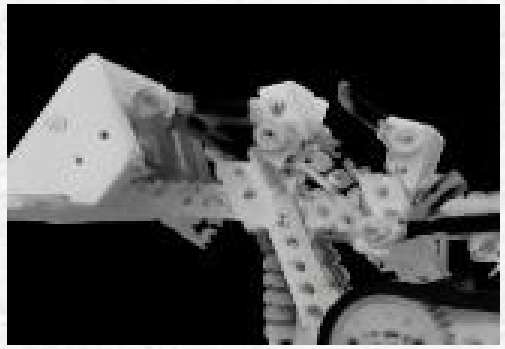
Wrongly segmented !

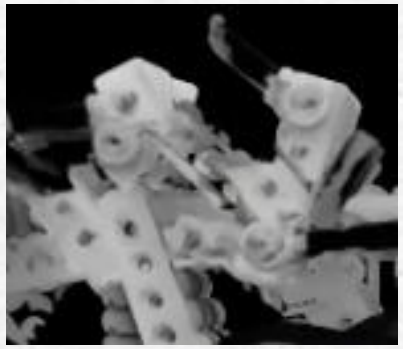
Proper



# "Broken Arm" of MaTGS & MonoGS

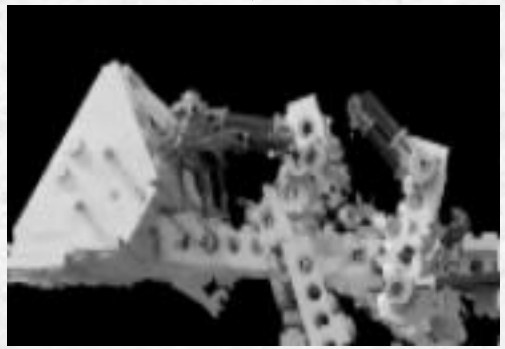
MaTGS





Add Video
Visual Supervision

MonoGS



From MaTGS → MonoGS

***Visual Effect:*** Better ✓

***Structural Broken Arm:*** Still Same ! ✗

## Why?

(3) Missing parent-child joint constraints

- e.g: Forearm should be connected to the upper arm via the elbow joint;

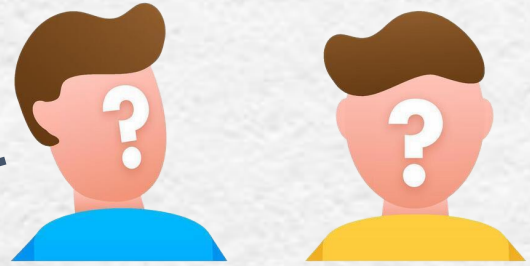
- upper arm movement depends not only on its **own rotation** but also on the **overall motion transmitted from the forearm.**

Constraint Should be Built !

# "Broken Arm" Root ! → Transformation Matrix ≠ Joint Constraint

| / | ✗ Transformation Matrix | Joint Constraint (We Need !) ✓ |
|---|---|---|
| Core Function | Describes rigid-body transformation of a component **independently** | Describes the connection relationships between components **inter-individual** |
| Key Parameters | Rotation (R), Translation (T) | Parent, Pivot, Axis, **Relative** Transform |
| Solved Problem | How much **a component** has moved or rotated ? | Where a component should pivot, and how it **connects and moves** |
| Scope of Constraint | A single component | Multiple interconnected components |
| if Absence ? | Errors in motion estimation | Disjointed component space; failure of connection relationships |

SUMMARY



"I won't affect you! "

One for One (141)

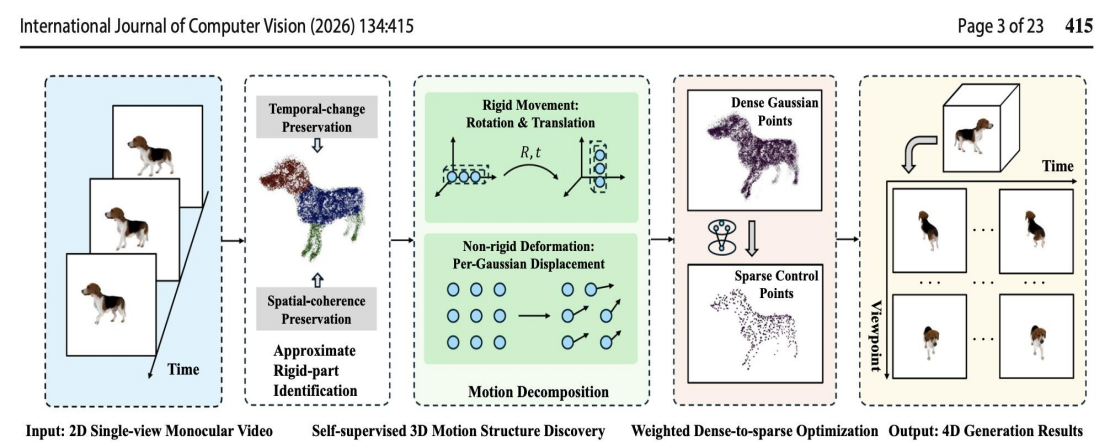
build the connection (ours)

One to One (121)

# To Solve 'Broken Arm' Problem

we borrow and optimize others methods and ideas...

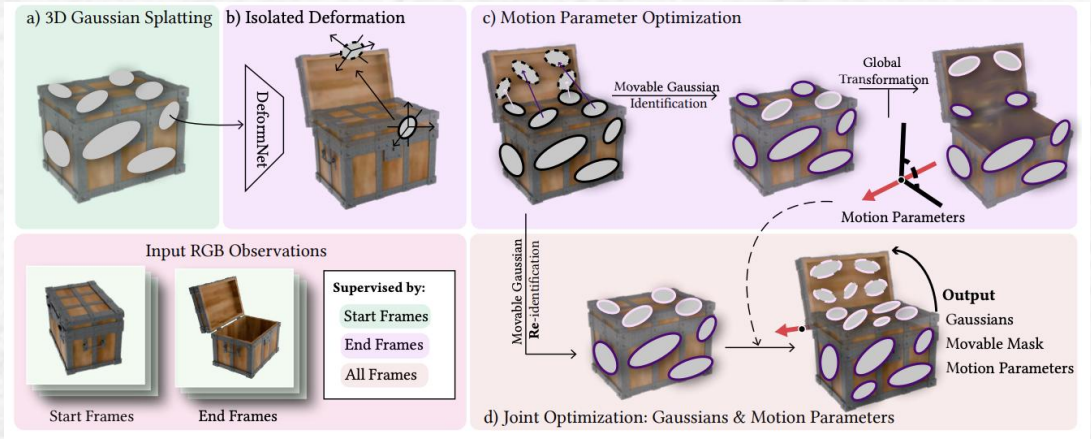
## MotionStruct4D: Discovering Motion Structure of Gaussian Splatting for Video-to-4D Generation （2026）




Rigid Motion Decomposition ✓ | Weighted Kabsch | Rigid Part Discovery

https://link.springer.com/article/10.1007/s11263-026-02992-z

## ArticulatedGS: Self-supervised Digital Twin Modeling of Articulated Objects using 3D Gaussian Splatting （2025）




Modeling of Articulated Object ✓ | Motion Parameter Estimation | Part-level Reconstruction ✓

https://arxiv.org/pdf/2503.08135

## Part-aware Modeling of Articulated Objects using 3D Gaussian Splatting

https://arxiv.org/pdf/2506.17212

Part-aware GS Representation ✓ | Disentangled Transformation ✓ | Contact Constraint ✓

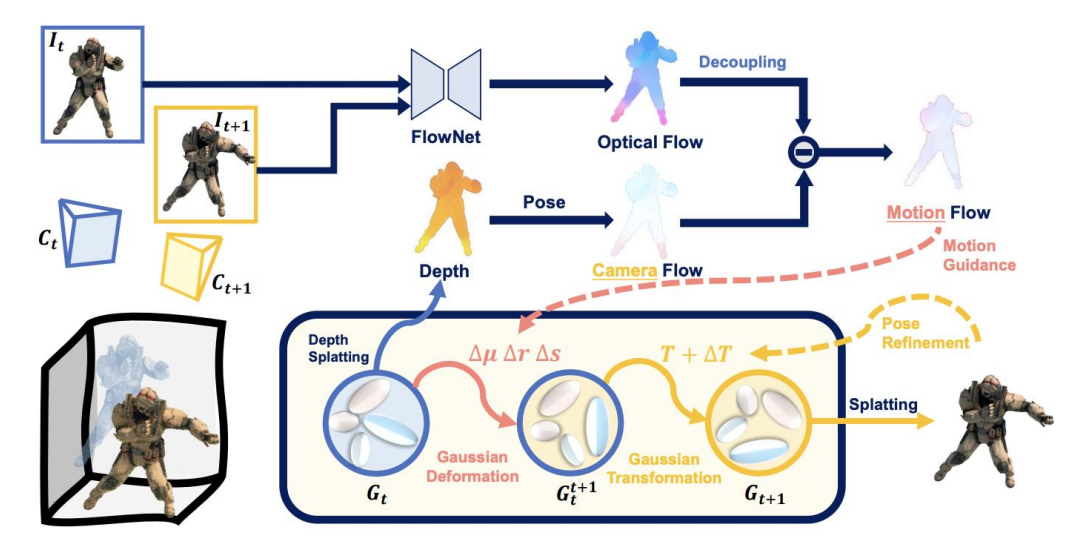
## HAIF-GS: Hierarchical and Induced Flow-Guided Gaussian Splatting for Dynamic Scene

https://arxiv.org/abs/2506.09518

Anchor-driven Deformation ✓ | Hierarchical Anchor Propagation | Contact Constraints

# More Related Work for MaTGS & MonoGS

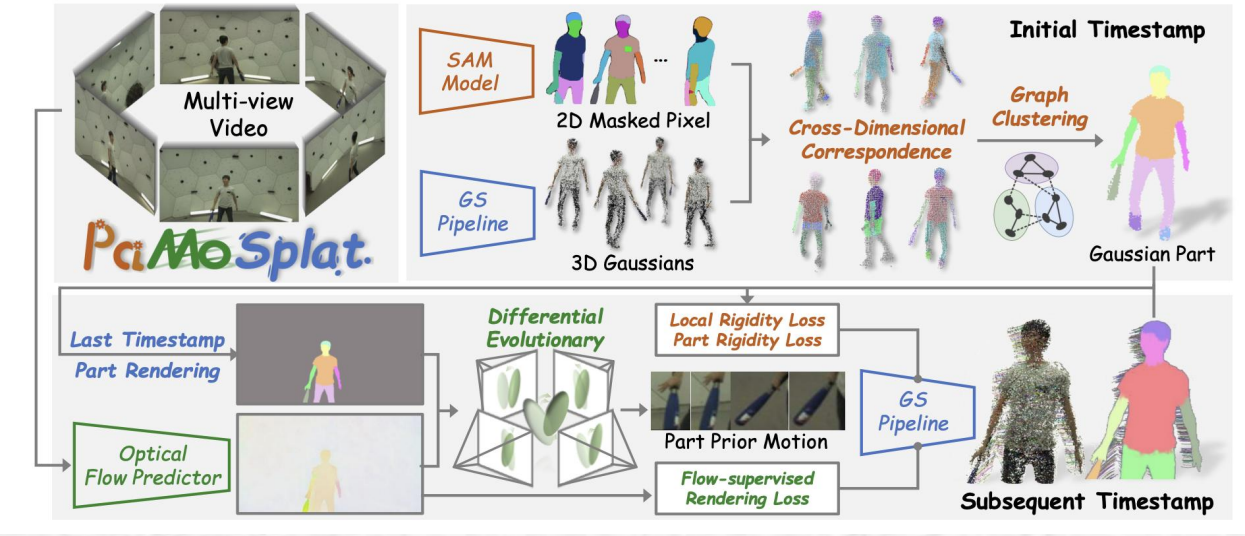
MotionGS: Exploring Explicit Motion Guidance for Deformable 3D Gaussian Splatting （2025）




Rigid Motion Decomposition ✓ | Motion-guided GS Deformation | Optical Flow Decoupling ✓

https://arxiv.org/pdf/2410.07707

PaMoSplat: Part-Aware Motion-Guided Gaussian Splatting forDynamic Scene Reconstruction （2026）




Part-aware Segmentation | Rigid Motion Estimation | Supervised Rendering Loss ✓

https://arxiv.org/pdf/2605.10307

Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis

https://arxiv.org/abs/2308.09713

Persistent Gaussians | 6-DOF Tracking ✓ | Local Rigidity Constraints ✓

SplArt: Articulation Estimation and Part-Level Reconstruction with 3D Gaussian Splatting

https://arxiv.org/abs/2506.03594

Gaussian-level Mobility Parameter | Part Segmentation ✓ | Articulation Estimation

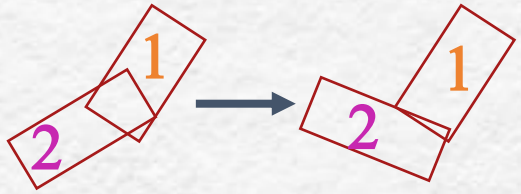
# Our Focus

## Part-level Segmentation Modeling

- Part Segmentation
- Part-level Reconstruction
- Part-aware GS Representation

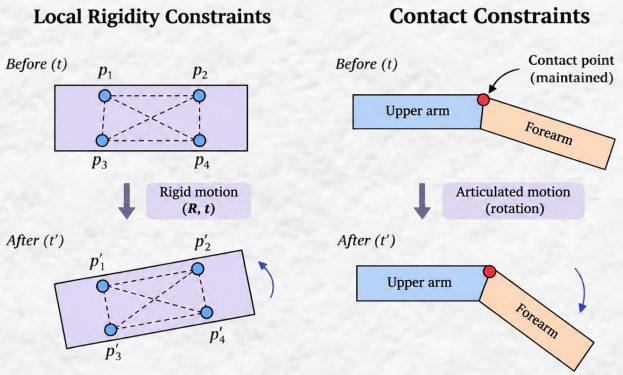
## Rigid & Articulated Motion

- Rigid Motion Decomposition
- 6-DOF Tracking
- Disentangled Transformation
- Articulated Object Modeling



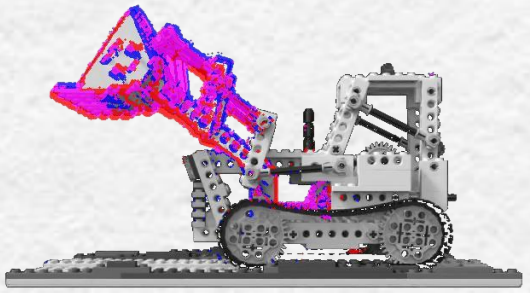

## Motion Constraint & Deformation

- Local Rigidity Constraints
- Contact Constraints




## Motion Supervision & Optimization

- Event Stream
- Supervised Rendering Loss



# Our Architecture (MERCY)

## 1 Input & Motion

MonoGS

$t_{i+1}$

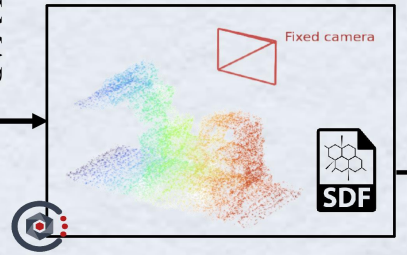
$t_{i+n}$

n frames

$$\begin{bmatrix} R_{11}(t) & R_{12}(t) & R_{13}(t) & T_x(t) \\ R_{21}(t) & R_{22}(t) & R_{23}(t) & T_y(t) \\ R_{31}(t) & R_{32}(t) & R_{33}(t) & T_z(t) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformation Matrix

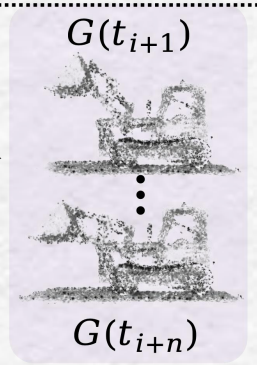
COLMAP

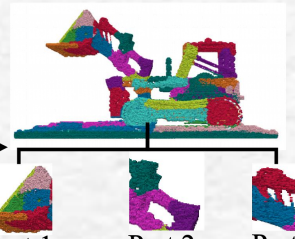



Monocular perspective

## 2 Part-aware Kinematic Structure

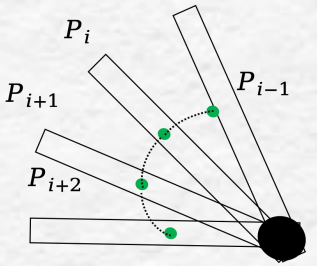
Port to Gaussian




Part-level Gaussians

Parts Segmentation

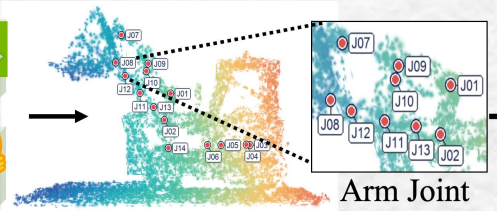



Catmull–Rom Interpolation Algorithm

Motion Smoothing

Curobo

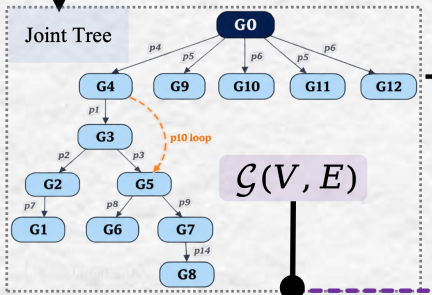
UniPhysGen




Key Joint Nodes

Arm Joint

Curobo




$\mathcal{G}(V, E)$

Kinematics Module

FK

$R_i'(t)$ $T_i'(t)$

## 3 MotionFineTuner

Spatio-temporal *Smoothness*

Not predicting from scratch

*Local Approximation* Rule

Linear (10, 32) → Silu → Linear (32, 64) → Silu → Linear (64, 4) → Activation

MLP Motion Refinement

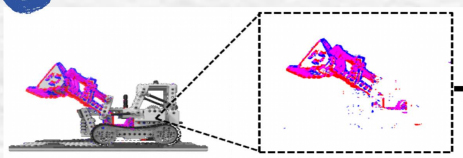
*Articulation Parameters*

$\triangle u,\ \triangle q,\ \triangle s,\ C_p$

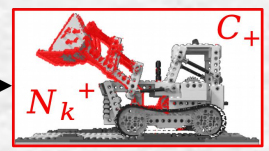
deform propagatic

$logI'_{i+1} - logI'_i$

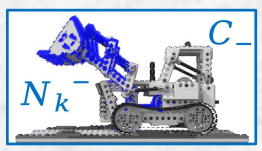
## 3 Rendering & Event Supervision & Optimization



Event Data



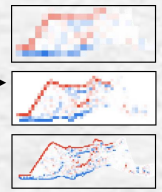

$\ominus$



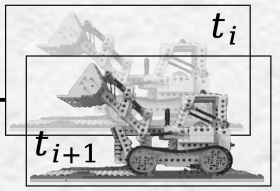

$E_k = C_+ N_k^+ - C_- N_k^-$

Spatial Pyramid Pooling $S$

$S \in \{40,80,160\}$



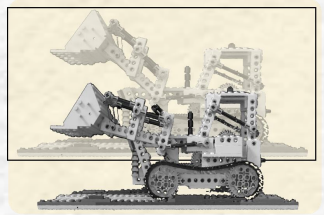

$E_{k,s}$




## Legend

- Forward Inference Flow
- Optimization Feedback
- Data
- Module
- Intermediate Product
- Core Module
- Loss
- Final Output

## Final Output



Moving $G'(t)$

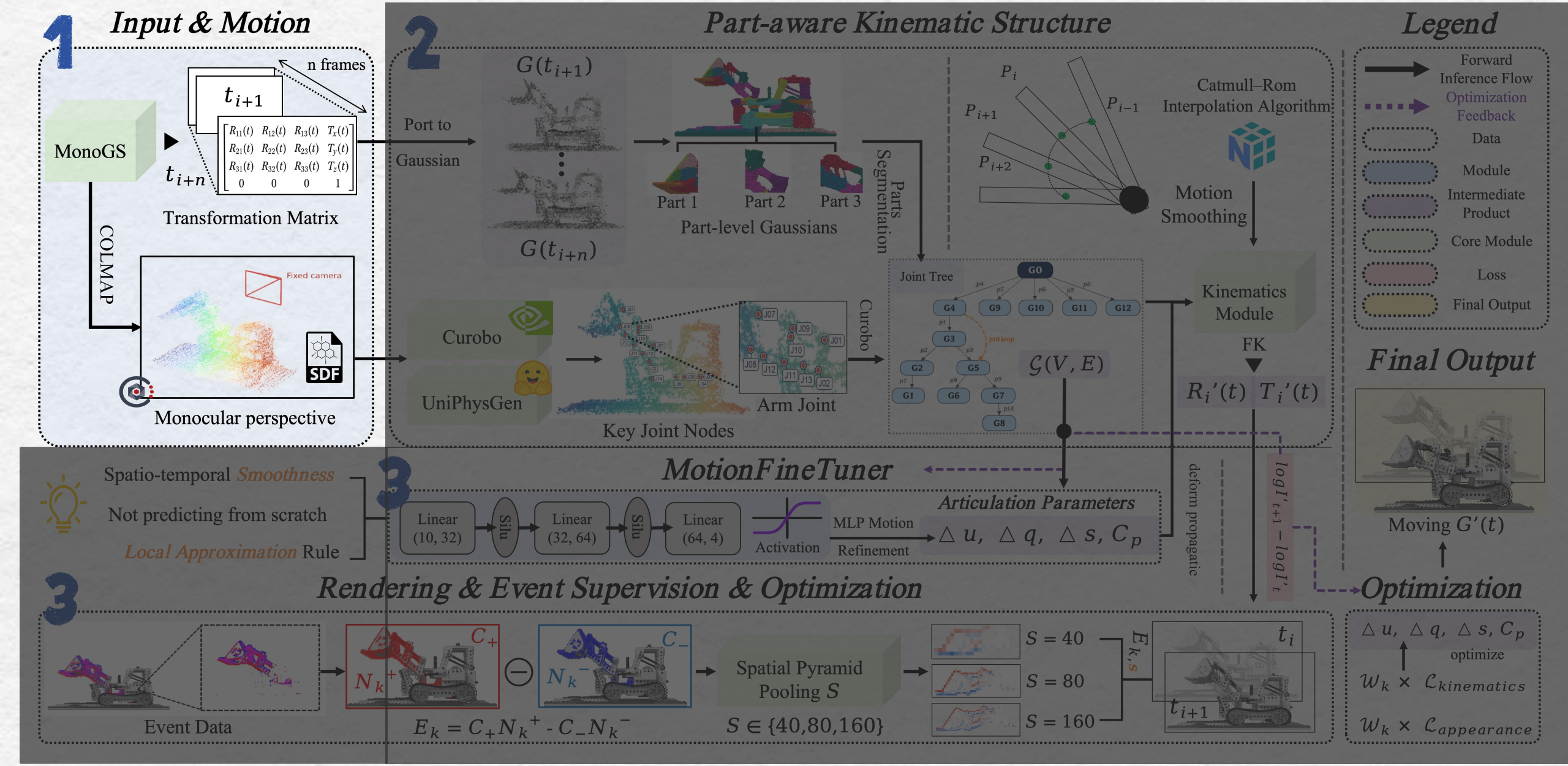
## Optimization

$\triangle u,\ \triangle q,\ \triangle s,\ C_p$ optimize

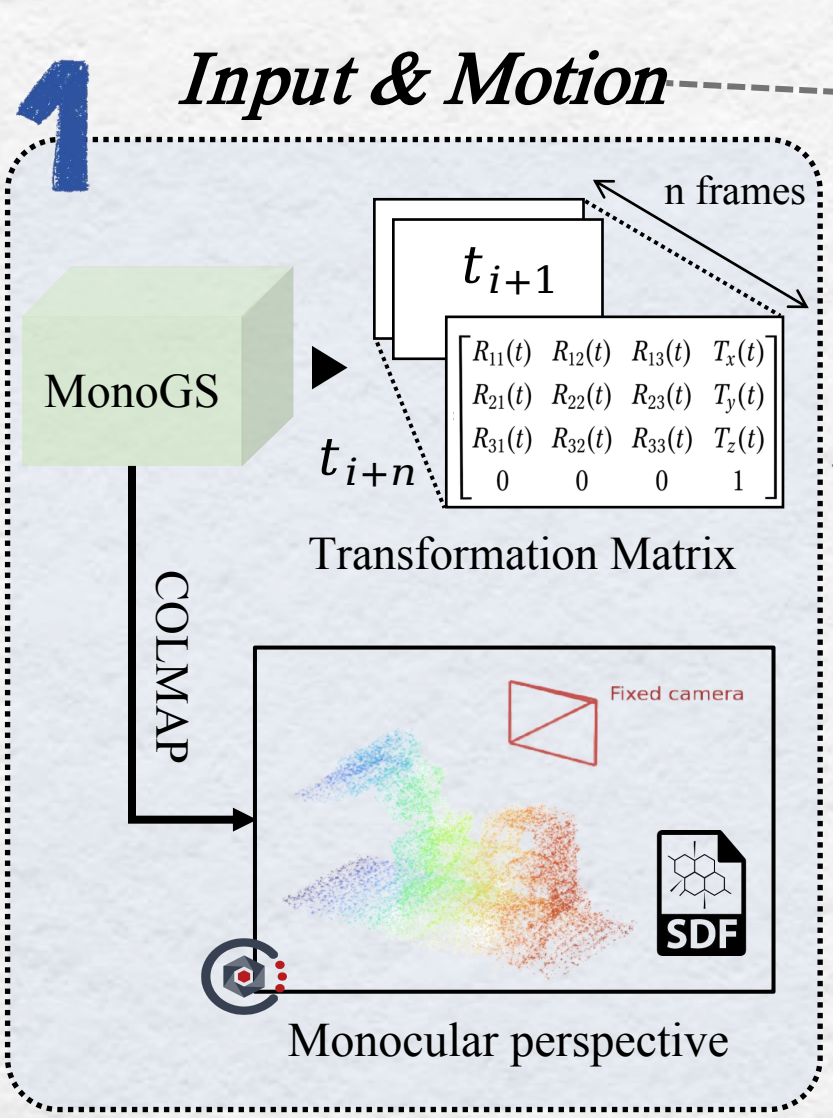
$w_k \times \mathcal{L}_{kinematics}$

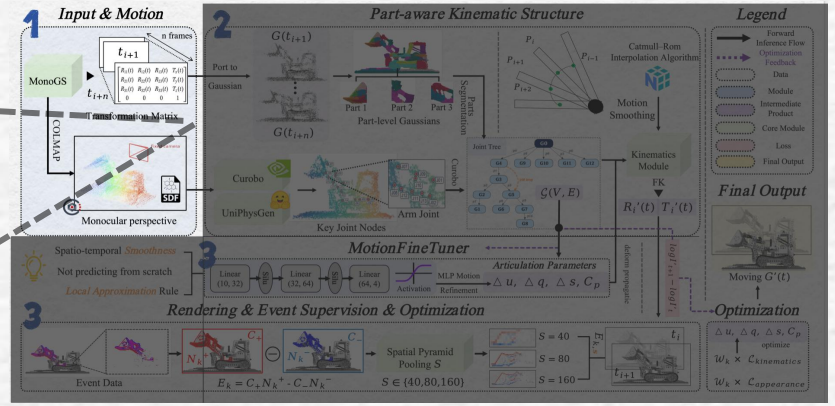
$w_k \times \mathcal{L}_{appearance}$

# Part I: Mercy initial Input



# Part I: Mercy initial Input Preparation



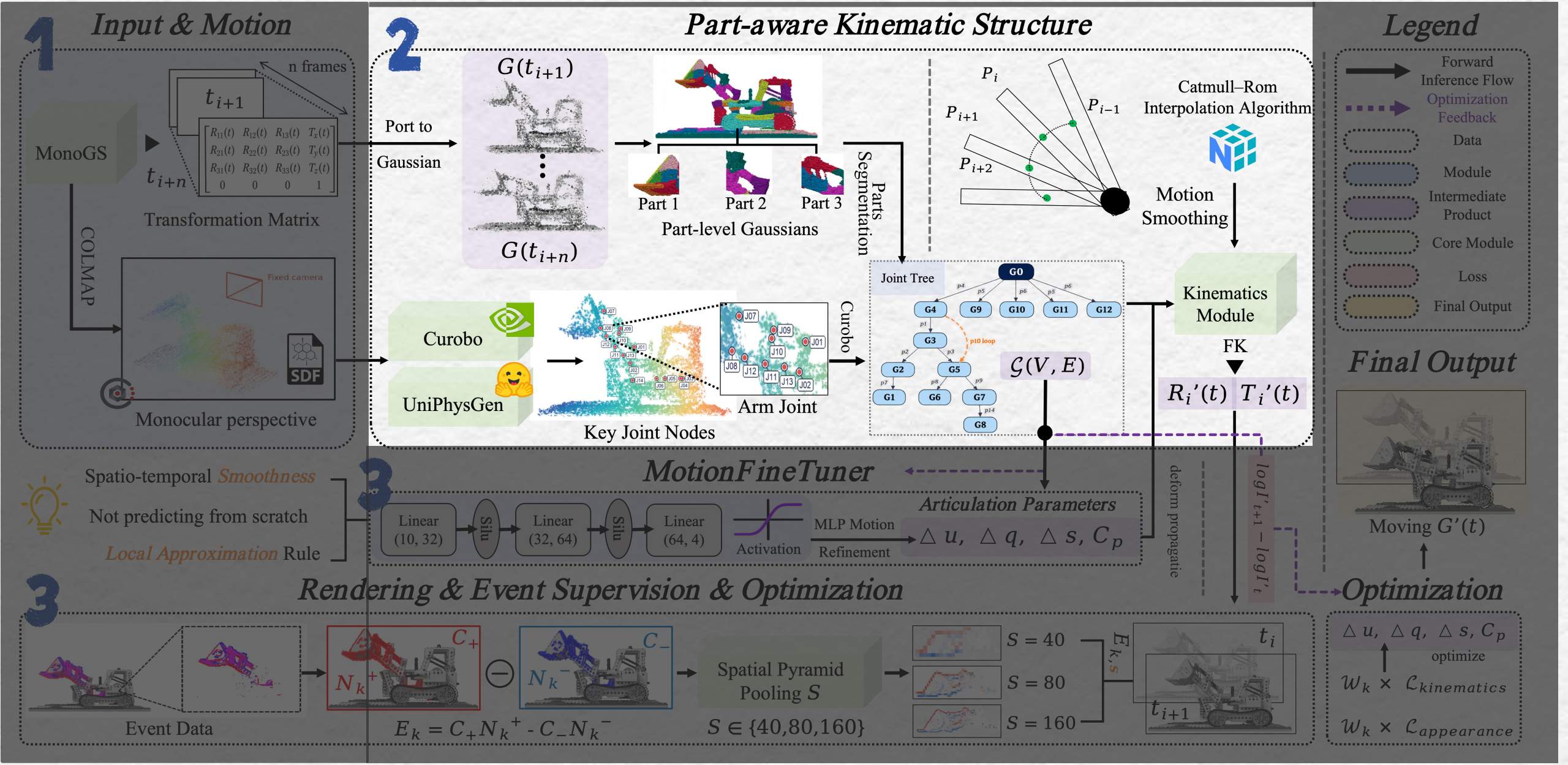

## 1. Input & Motion:

- MonoGS - input module, **monocular video**
- Video is processed by **COLMAP** to establish geometric foundation for subsequent 3D reconstruction

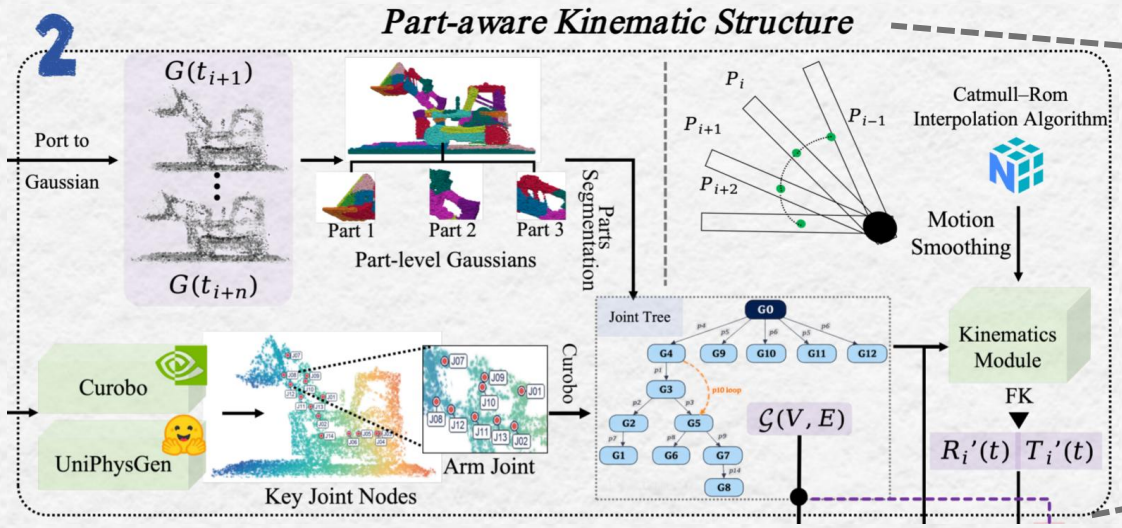
## 2. Multi-frame Sampling:

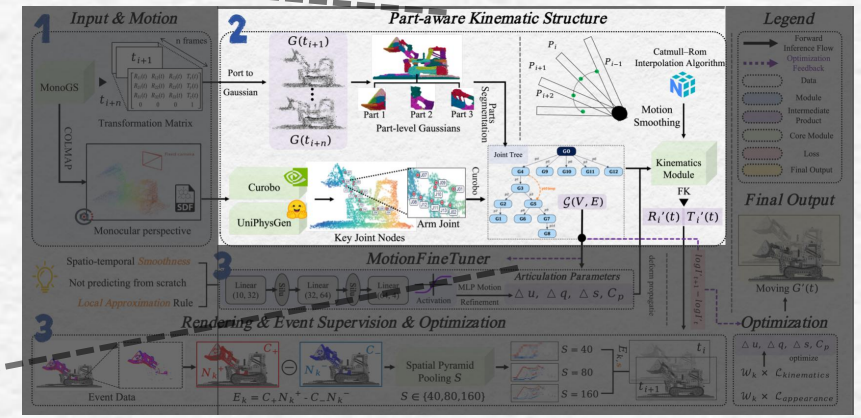
- Spatiotemporal **Motion Sequences** (Motion Matrices)
- Transformation Matrix Estimation

# Part II: Part-aware Kinematic Structure



# Part II: Part-aware Kinematic Structure




## 1. Part-level Gaussian Segmentation

- Each moment $\boldsymbol{t}$, the Gaussian representation partitioned into **distinct components** ($i^{th}$ component — $G_j^i$).

## 2. Joint Node Extraction:

- Use open-source CuRobo & UniPhysGen → key joints pivot

## 3. Joint Tree Construction

- Based on step2 → $key\ joints\ pivot$ → Joint Tree $\mathcal{G}(V, E)$

## 4. Motion Smoothing

- Catmull–Rom **Interpolation Algorithm**
- Continuous, smooth **motion paths**

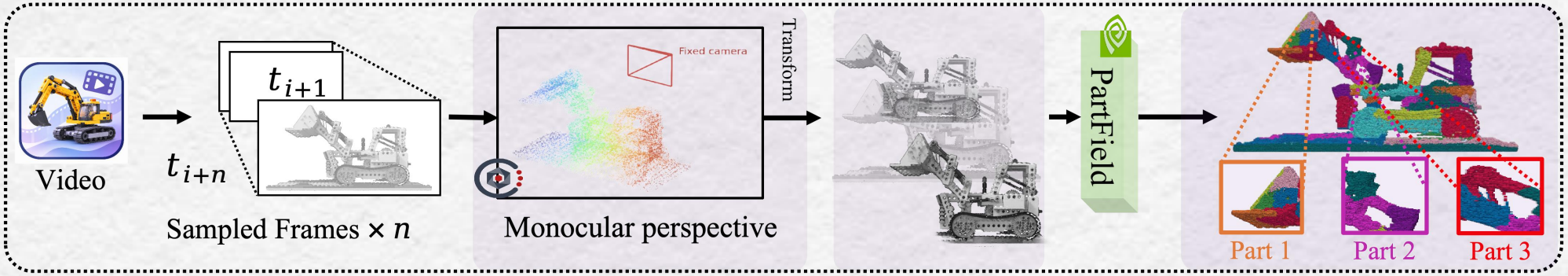
  → reduce abrupt motion changes

## 5. Forward Kinematics Reconstruct:

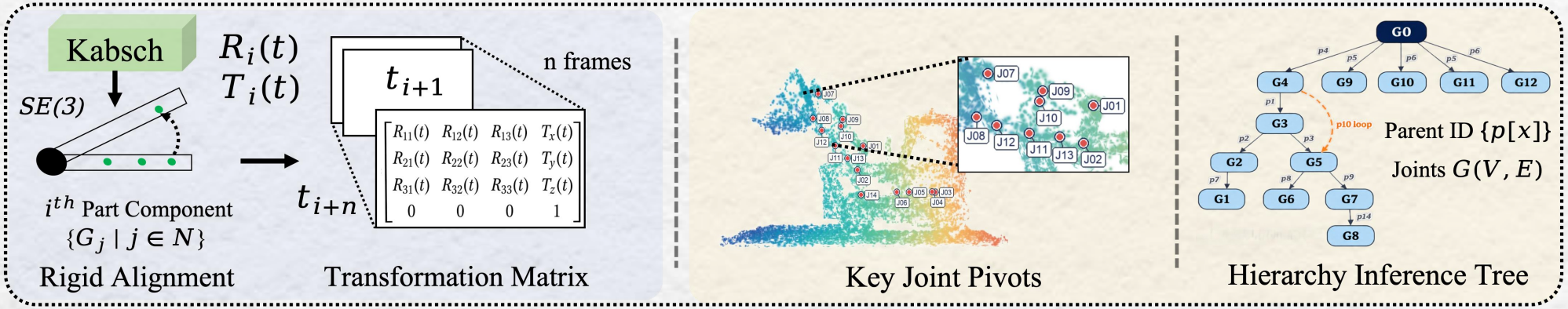
- get Joint Node + Tree Construction
- get smooth **motion path**
- **FK module** → subcomponent pose

# Part II: Part-aware Kinematic Structure

## Data Acquisition & Reconstruction




## Motion Estimation & Kinematic Structure Inference




Part-level GS Segmentation

↓

Monocular 3D Reconstruction

↓

Initial Base Movement

↓

Key Joint Pivot / Tree

**Legend**

Forward Inference Flow

Optimization Feedback

Module

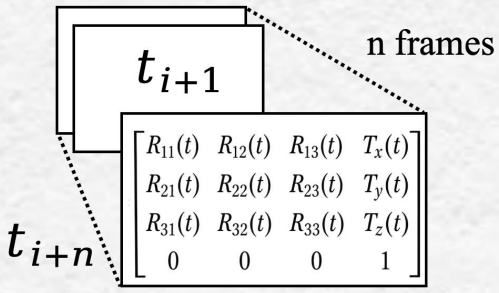
Intermediate Product

Core Module

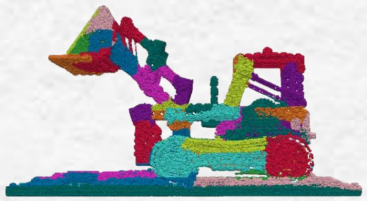
Output

# Part II: Part-aware Kinematic Structure

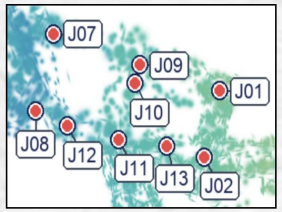
## *Input & Motion*




Transformation Matrix



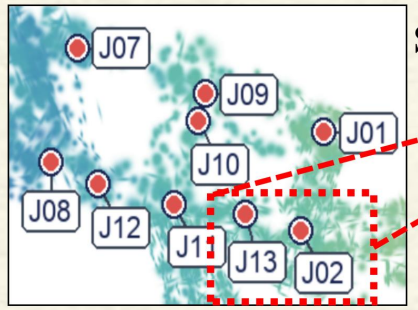
Separated Part Components




Joint Pivots

Joint Parent
$G(V, E)$

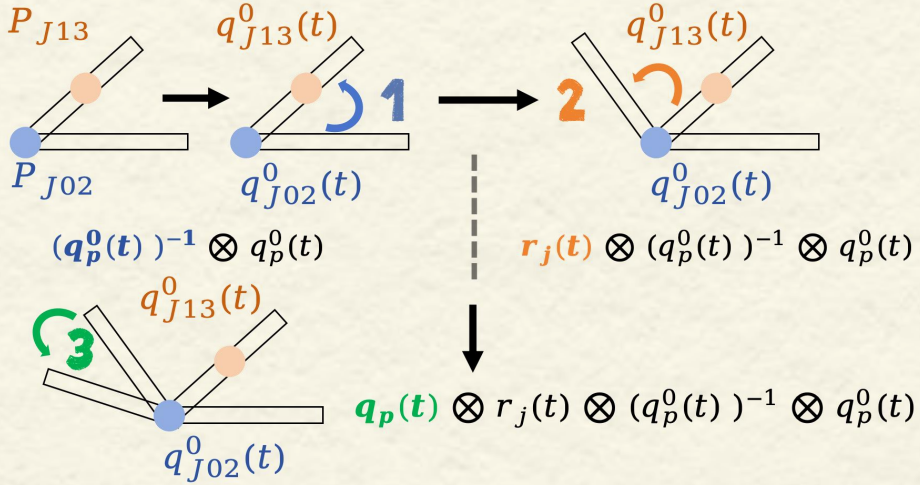
## *Parent-Child Kinematic Constraint*




Selected Joint Pair: *J02* → *J13*

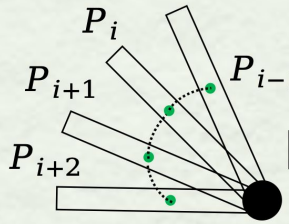
*P* *J02* ..... *j* *J13*

*P: parent node*
*j: child node*




Joint Kinematic Constraint Propagate

Catmull–Rom Interpolation




$R_i'(t)$ $T_i'(t)$

Temporal Smooth

Source & Motion Initial Input

Key Joint Pivot / Tree

Parent–Child Hierarchy Infer

Hierarchical Kinematic Model

Interpolation

**Legend**

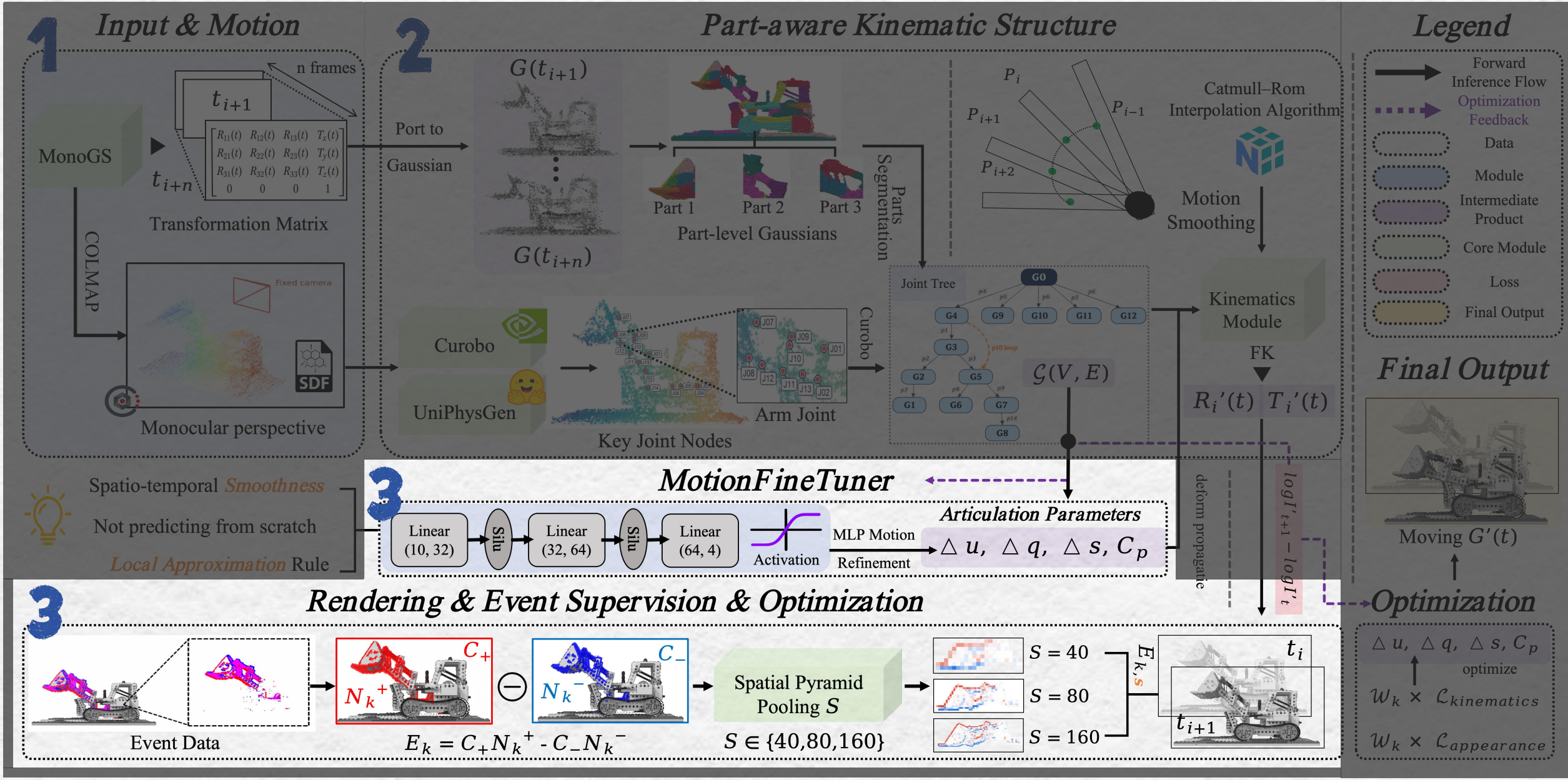
Forward Inference Flow

Joint Kinematic Constraint Propagate

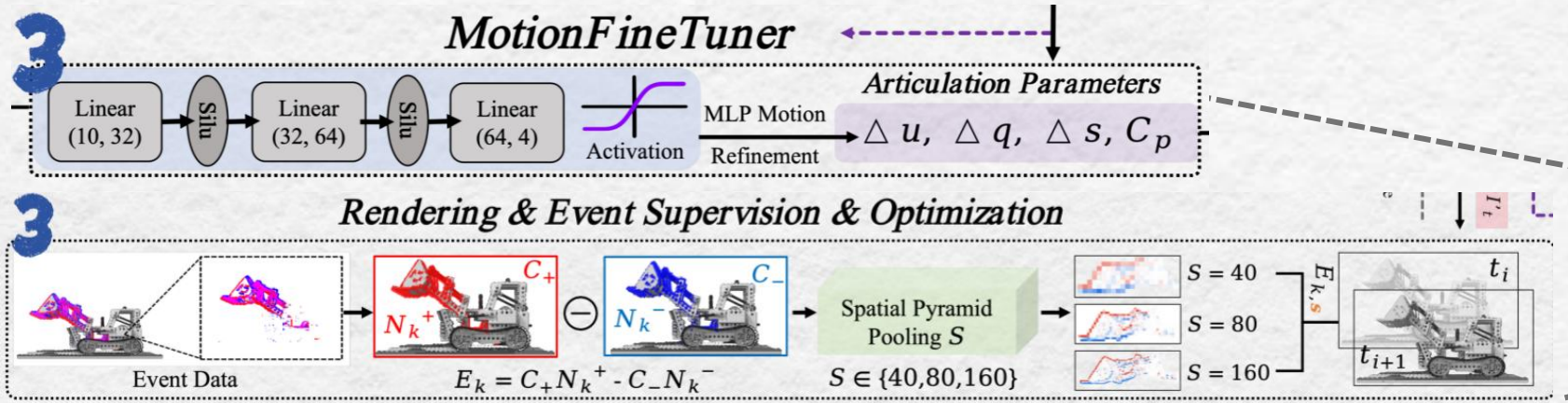
Temporal Smooth

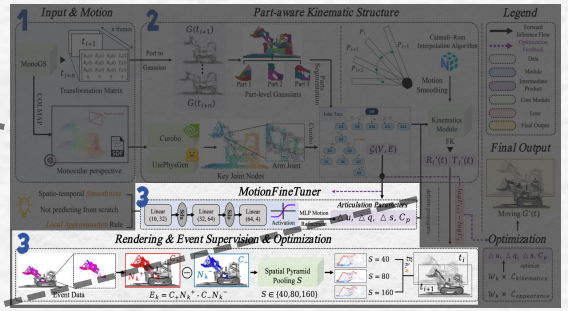
Final Output

# Part III: Motion FineTuner & Rendering, Event Supervision



# Part III: Motion FineTuner & Rendering, Event Supervision



## 1. MLP Motion Refinement

- **Frozen** static component
- Optimize **joint motion, rotation**, and spatial transformation

## 2. Event Data Representation

- **Weighted combination** of ± events
- **event-based** supervisory signals

## 3. Spatial Pyramid Pooling

- Extract **multi-scale** motion features,
- broader outlines & local details.

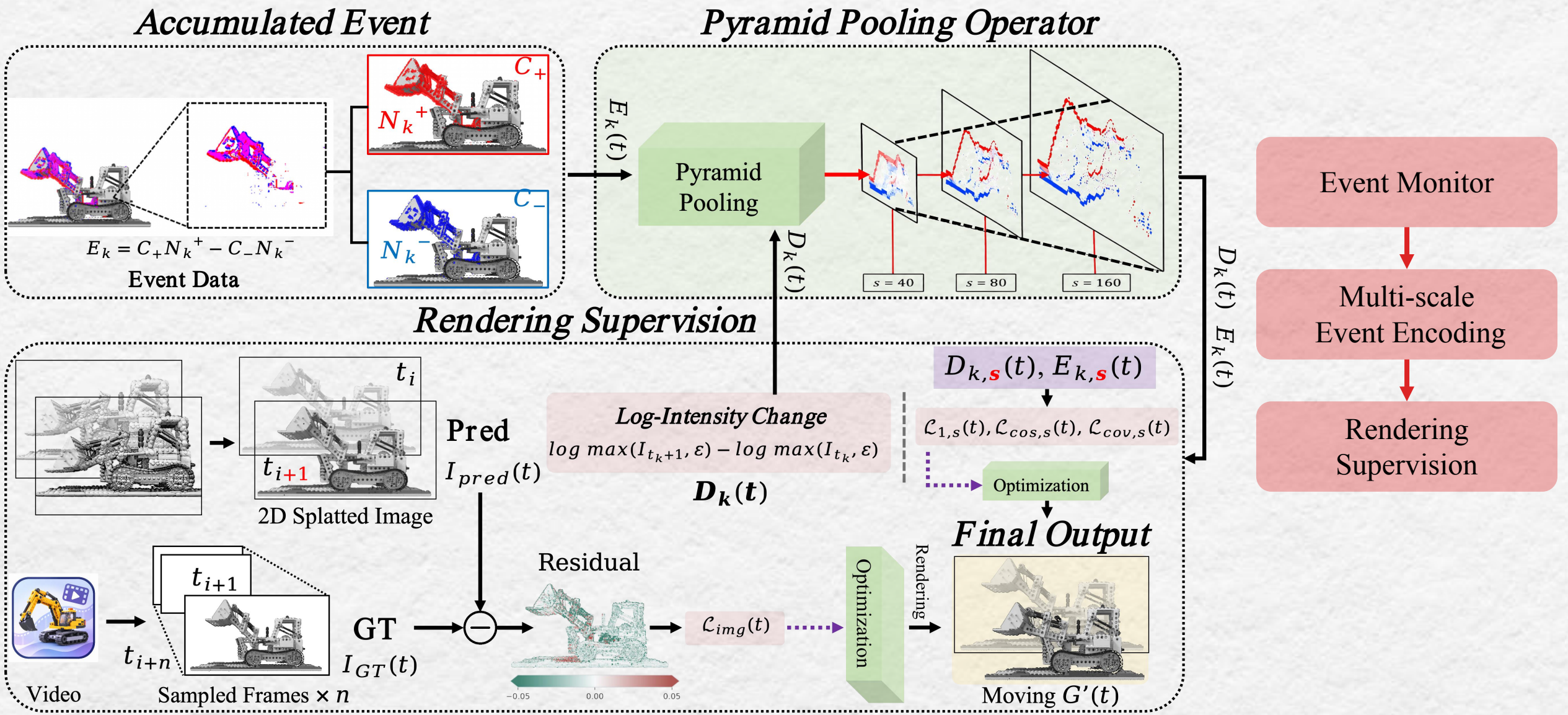
## 4. Rendering & Event Supervision

- **Optimized motion** params to GS Model
- **Rendering** $I'(t_i),\ I'(t_{i+1})$
- Event Supervision Loss

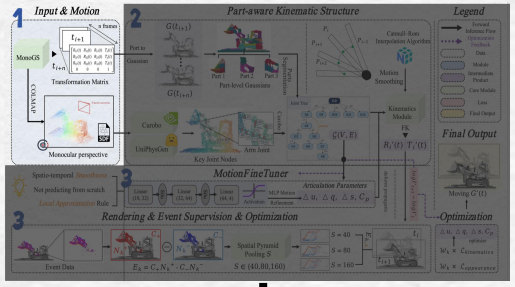
## 5. Joint Motion Optimization

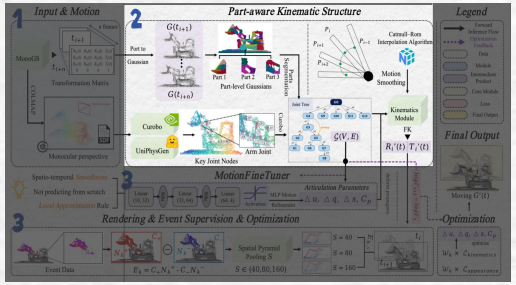
- MLP predicted **refined movement**
- Joint optimization + refined motion param estimation with supervision

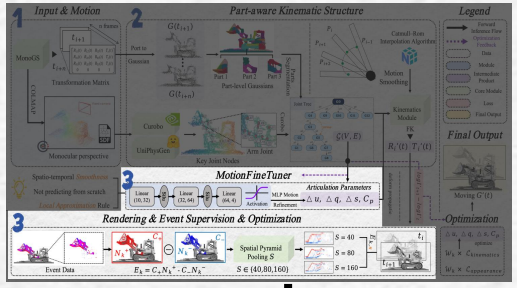
# Part III: Motion FineTuner & Rendering, Event Supervision




**Legend** → Forward Inference Flow; ⇢ Optimization Feedback; Intermediate Product; Core Module; Loss; Output







1 Input & Motion

MonoGS

COLMAP

n frames

$t_{i+1}$

$t_{i+n}$

$\begin{bmatrix} R_{11}(t) & R_{12}(t) & R_{13}(t) & T_x(t) \\ R_{21}(t) & R_{22}(t) & R_{23}(t) & T_y(t) \\ R_{31}(t) & R_{32}(t) & R_{33}(t) & T_z(t) \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Transformation Matrix

Fixed camera

SDF

Monocular perspective

2 Part-aware Kinematic Structure

Port to Gaussian

$G(t_{i+1})$

$G(t_{i+n})$

Part 1 Part 2 Part 3

Part-level Gaussians

Parts Segmentation

$P_i$ $P_{i+1}$ $P_{i+2}$ $P_{i-1}$

Catmull–Rom Interpolation Algorithm

Motion Smoothing

Curobo

UniPhysGen

Key Joint Nodes

Arm Joint

Curobo

Joint Tree

G0 G1 G2 G3 G4 G5 G6 G7 G8 G9 G10 G11 G12

p10 loop

$\mathcal{G}(V, E)$

Kinematics Module

FK

$R_i'(t)$ $T_i'(t)$

3 MotionFineTuner

Spatio-temporal *Smoothness*

Not predicting from scratch

*Local Approximation* Rule

Linear (10, 32) Silu Linear (32, 64) Silu Linear (64, 4)

Activation

MLP Motion Refinement

*Articulation Parameters*

$\triangle u,\ \triangle q,\ \triangle s,\ C_p$

deform propagatic

$logI'_{t+1} - logI'_t$

3 Rendering & Event Supervision & Optimization

Event Data

$C_+$ $N_k^+$ $\ominus$ $C_-$ $N_k^-$

$E_k = C_+ N_k^+ - C_- N_k^-$

Spatial Pyramid Pooling $S$

$S \in \{40,80,160\}$

$S = 40$

$S = 80$

$S = 160$

$E_{k,s}$

$t_i$ $t_{i+1}$

Legend

Forward Inference Flow

Optimization Feedback

Data

Module

Intermediate Product

Core Module

Loss

Final Output

Final Output

Moving $G'(t)$

Optimization

$\triangle u,\ \triangle q,\ \triangle s,\ C_p$

optimize

$w_k \times \mathcal{L}_{kinematics}$

$w_k \times \mathcal{L}_{appearance}$

# Input

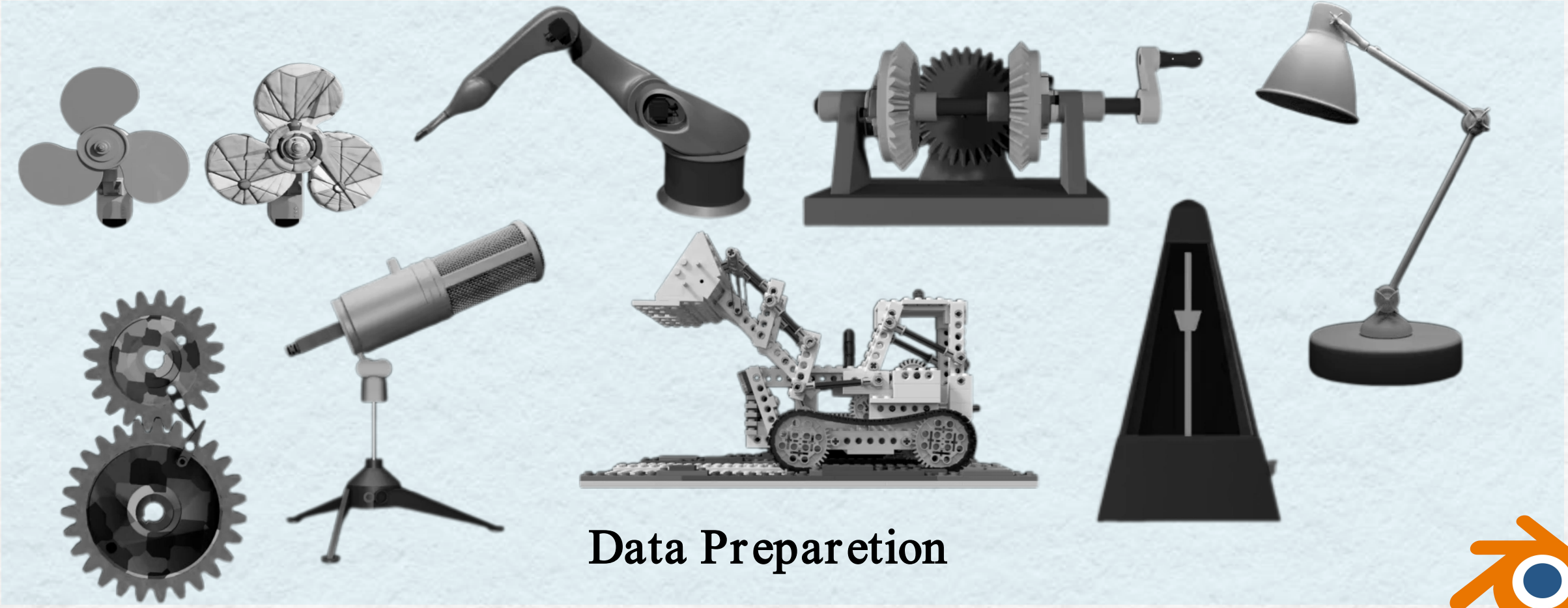
6 types of input

MERCY

3D Property Rendered Video

# Output

# Result & Evaluation

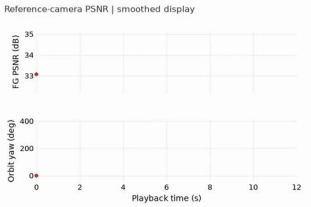


Data Preparetion

# Result & Evaluation

## Lego

3.50 cycles / 12s | 0.972x source




Video 00.00s | Source f=000

FG PSNR: 31.24 dB

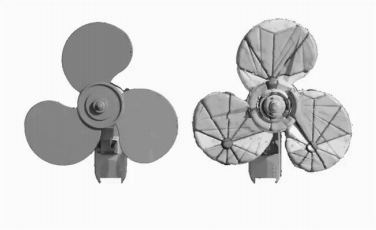
Orbit yaw: 0.0 deg

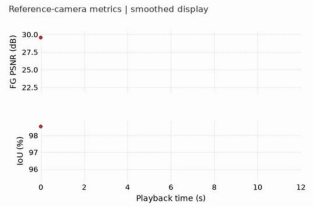
Source t=0.000s | sample t=0.000s

Reference-camera PSNR; fitting/selection included

## Fan

4.00 round trips / 12s | 1.111x source






Video 00.00s | Source f=000

FG PSNR: 32.88 dB IoU: 98.69%

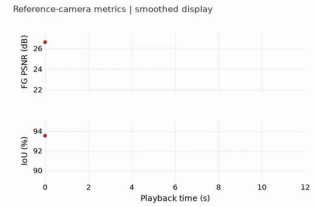
LPIPS: 0.0084

Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

## Arm

3.00 cycles / 12s | 0.217x source




Video 00.00s | Source f=000

FG PSNR: 27.91 dB IoU: 94.68%

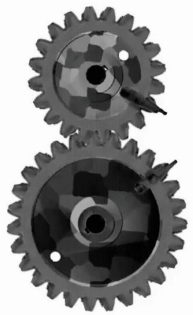
LPIPS: 0.0098

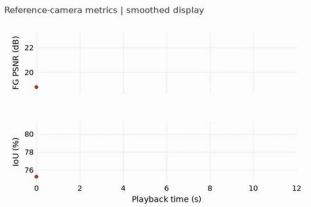
Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

## Gears

3.50 gear turns / 12s | 0.058x source






Video 00.00s | Source f=003

FG PSNR: 18.67 dB IoU: 75.03%

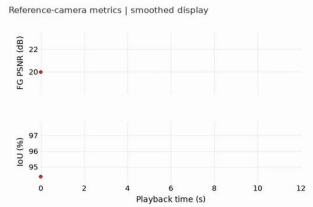
LPIPS: 0.0443

Source t=0.100s | sample t=0.100s

Nearest measured sample; fitting/selection included

## Lamp

3.00 round trips / 12s | 0.817x source




Video 00.00s | Source f=000

FG PSNR: 19.55 dB IoU: 93.91%

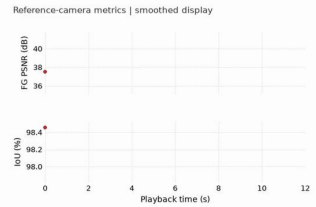
LPIPS: 0.0303

Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

## Metronome

3.25 cycles / 12s | 0.451x source




Video 00.00s | Source f=000

FG PSNR: 37.26 dB IoU: 98.47%

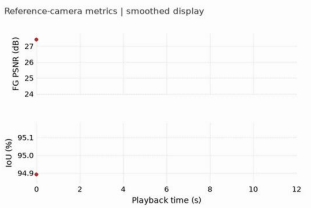
Missing foreground: 0.00%

Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

## Microphone

3.50 cycles / 12s | 0.243x source




Video 00.00s | Source f=000

FG PSNR: 27.61 dB IoU: 94.84%

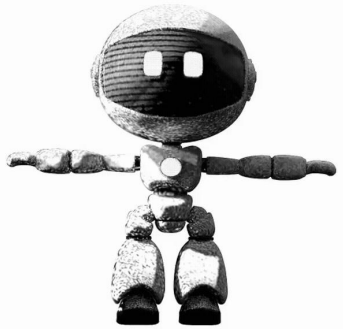
LPIPS: 0.0125

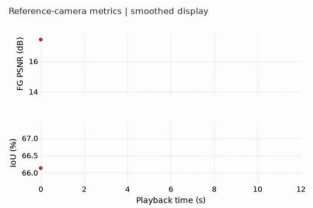
Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

## Robot

4.00 cycles / 12s | 0.133x source






Video 00.00s | Source f=000

FG PSNR: 17.37 dB IoU: 66.24%

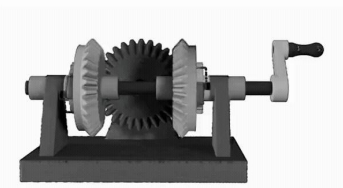
Missing foreground: 0.06%

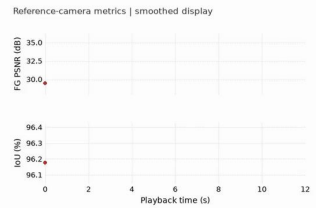
Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

## Rotation

3.50 round trips / 12s | 0.758x source






Video 00.00s | Source f=000

FG PSNR: 29.28 dB IoU: 96.27%

LPIPS: 0.0055

Source t=0.000s | sample t=0.000s

Nearest measured sample; fitting/selection included

# Result 1: Image reconstruction quality

**Table 1: Image reconstruction quality.**

| Object | PSNR (dB) ↑ | | | SSIM ↑ | | | LPIPS ↓ | | | IoU ↑ | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy |
| Lego | 17.63 | 26.10 | 31.84 | 0.7905 | 0.9398 | 0.9772 | 0.1431 | 0.0450 | 0.0223 | 0.8327 | 0.9187 | 0.9335 |
| Fan | 23.91 | 24.24 | 29.76 | 0.9106 | 0.9135 | 0.9731 | 0.0473 | 0.0437 | 0.0170 | 0.9571 | 0.9657 | 0.9657 |
| Lamp | 23.44 | 25.22 | 30.64 | 0.9382 | 0.9604 | 0.9717 | 0.0646 | 0.0496 | 0.0292 | 0.8163 | 0.8756 | 0.9663 |
| Metronome | 32.54 | 37.08 | 43.25 | 0.9700 | 0.9792 | 0.9950 | 0.0401 | 0.0320 | 0.0134 | 0.9577 | 0.9588 | 0.9619 |
| Microphone | 25.61 | 30.56 | 34.38 | 0.9428 | 0.9688 | 0.9837 | 0.0663 | 0.0305 | 0.0187 | 0.8334 | 0.9203 | 0.9235 |
| Arm | 26.36 | 25.39 | 33.14 | 0.9373 | 0.9366 | 0.9786 | 0.0483 | 0.0700 | 0.0153 | 0.8662 | 0.8556 | 0.9220 |
| Robot | 17.01 | 17.46 | 23.07 | 0.8503 | 0.8540 | 0.9294 | 0.1024 | 0.1650 | 0.0375 | 0.6223 | 0.6307 | 0.6658 |
| Gears | 23.44 | 21.81 | 27.93 | 0.8742 | 0.8644 | 0.9439 | 0.0817 | 0.0976 | 0.0281 | 0.8100 | 0.8050 | 0.8147 |
| Rotation | 24.84 | 24.16 | 38.78 | 0.8999 | 0.8946 | 0.9900 | 0.0766 | 0.0799 | 0.0071 | 0.8809 | 0.8713 | 0.9624 |

*Notes.* PSNR and SSIM are full-image metrics. Mercy columns are shaded light blue.

- Mercy **more consistent** reconstruction performance
- consistently **achieved highest values for PSNR, SSIM, IoU**
- **reducing LPIPS** — underscoring advantagesof image reconstruction quality and structural consistency

# Result 2: Foreground quality and coverage

Table 2: Foreground quality and coverage.

| Object | FG-PSNR (dB) ↑ | | | FG-SSIM ↑ | | | Missing (%) ↓ | | |
|---|---|---|---|---|---|---|---|---|---|
| | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy |
| Lego | 12.75 | 21.71 | 27.31 | 0.2221 | 0.7901 | 0.9239 | 3.43 | 2.27 | 1.61 |
| Fan | 18.57 | 18.62 | 24.02 | 0.6956 | 0.6965 | 0.9233 | 2.17 | 2.15 | 2.15 |
| Lamp | 17.10 | 18.34 | 21.49 | 0.6699 | 0.7677 | 0.7897 | 3.95 | 5.08 | 1.73 |
| Metronome | 22.87 | 28.03 | 34.71 | 0.7306 | 0.8219 | 0.9720 | 1.03 | 1.71 | 1.49 |
| Microphone | 16.01 | 21.12 | 25.50 | 0.4997 | 0.6717 | 0.8388 | 3.24 | 3.73 | 3.60 |
| Arm | 17.79 | 16.65 | 24.80 | 0.5153 | 0.5209 | 0.8660 | 7.09 | 7.07 | 1.83 |
| Robot | 8.62 | 8.95 | 15.86 | 0.0764 | 0.0909 | 0.5651 | 8.07 | 5.84 | 0.04 |
| Gears | 18.23 | 16.28 | 23.37 | 0.4878 | 0.4324 | 0.8330 | 10.28 | 6.68 | 4.55 |
| Rotation | 18.34 | 17.16 | 31.29 | 0.5003 | 0.4647 | 0.9523 | 3.90 | 4.15 | 0.08 |

- Mercy improvements in foreground reconstruction quality (FG-PSNR, FG-SSIM)
- Consistently lower missing rates across "Missing" metrics
- superior ability to accurately recover foreground details of moving components.

# Result 4: Foreground quality and coverage

**Table 4: Runtime efficiency and model size.**

| Object | Mean latency (ms) ↓ | | | P50 latency (ms) ↓ | | | P95 latency (ms) ↓ | | | Gaussians (count) | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy |
| Lego | 15.53 | 14.33 | 32.16 | 13.28 | 14.27 | 32.15 | 28.58 | 15.07 | 33.33 | 1479653 | 1841816 | 1813271 |
| Fan | 13.94 | 13.44 | 13.73 | 13.32 | 13.22 | 13.53 | 14.23 | 14.22 | 14.45 | 906401 | 906401 | 906401 |
| Lamp | 10.26 | 18.41 | 9.96 | 9.04 | 16.89 | 8.38 | 19.95 | 31.95 | 23.31 | 426697 | 853403 | 426697 |
| Metronome | 7.55 | 45.06 | 44.93 | 7.55 | 43.32 | 43.30 | 8.26 | 51.23 | 51.20 | 645262 | 1143789 | 1143789 |
| Microphone | 6.60 | 16.36 | 16.47 | 6.60 | 16.27 | 16.30 | 7.69 | 18.36 | 18.25 | 227244 | 454420 | 454420 |
| Arm | 14.15 | 6.76 | 11.26 | 13.91 | 6.70 | 11.02 | 14.67 | 7.37 | 12.02 | 576117 | 288103 | 538488 |
| Robot | 18.73 | 9.93 | 17.58 | 18.96 | 9.85 | 17.51 | 20.13 | 11.06 | 18.72 | 645791 | 358899 | 737171 |
| Gears | 15.46 | 7.90 | 8.33 | 15.19 | 7.88 | 8.33 | 17.48 | 8.49 | 8.88 | 828050 | 467436 | 301739 |
| Rotation | 32.69 | 16.37 | 20.79 | 33.70 | 16.23 | 22.48 | 38.08 | 17.32 | 27.56 | 1567112 | 1155218 | 1016971 |

- Mercy lower inference latency in scenarios such as "Lamp," "Gears," and "Rotation," while simultaneously reducing model size by decreasing the number of Gaussians.
- latency overhead in more complex scenes like "LEGO" and "Metronome"
- Trade-off between reconstruction quality, operational efficiency, and model size

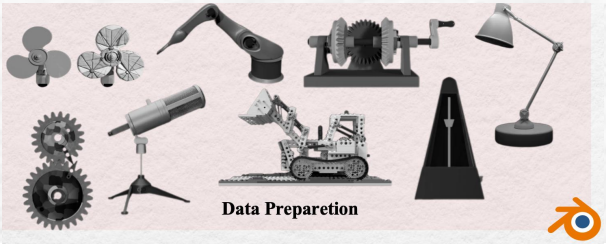
# Result 5: Event Consistency

**Table 5: Event consistency.**

| Object | Event relative L1 ↓ | | | Event cosine ↑ | | | Polarity agreement (%) ↑ | | | No-event change ↓ | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy | MaTGS | MonoGS | Mercy |
| Lego | 1.4955 | 1.3808 | 0.6943 | 0.2841 | 0.4406 | 0.8383 | 41.10 | 56.90 | 81.87 | 0.00760 | 0.00687 | 0.00213 |
| Fan | 0.7570 | 0.6923 | 0.4144 | 0.7236 | 0.7620 | 0.8755 | 73.61 | 73.70 | 87.35 | 0.01863 | 0.01500 | 0.00760 |
| Lamp | 1.1255 | 0.8672 | 0.5258 | 0.5235 | 0.6889 | 0.8511 | 70.03 | 77.83 | 84.01 | 0.01361 | 0.00691 | 0.00248 |
| Metronome | 0.4153 | 0.4326 | 0.3268 | 0.9088 | 0.8773 | 0.9288 | 71.41 | 70.51 | 72.62 | 0.00049 | 0.00091 | 0.00061 |
| Microphone | 0.6753 | 0.4651 | 0.3626 | 0.7342 | 0.8577 | 0.9016 | 81.33 | 82.29 | 86.74 | 0.00955 | 0.00304 | 0.00241 |
| Arm | 0.5054 | 0.5267 | 0.3742 | 0.8084 | 0.8169 | 0.8869 | 66.47 | 63.65 | 76.08 | 0.00838 | 0.00610 | 0.00896 |
| Robot | 0.9585 | 0.9551 | 0.5719 | 0.5503 | 0.5948 | 0.8220 | 55.65 | 57.89 | 74.68 | 0.04210 | 0.04667 | 0.03721 |
| Gears | 1.6359 | 1.4840 | 1.1225 | -0.0467 | 0.1533 | 0.2630 | 42.25 | 42.68 | 61.36 | 0.02413 | 0.03055 | 0.00959 |
| Rotation | 2.4357 | 1.8521 | 1.1946 | 0.3052 | 0.3370 | 0.8980 | 41.47 | 33.59 | 86.42 | 0.01786 | 0.01178 | 0.00490 |

- Mercy stronger event consistency: lower Event L1
- Improvements in Event Cosine similarity and Polarity Agreement
- "No-event Change" is effectively suppressed.
- Mercy possesses more accurate event representations and a more stable capacity

# ALL 9 Objects Metrics



PSNR ↑

error in detail reconstruction reduce

SSIM ↑

Higher structural similarity

LPIPS ↓

Lower perceptual error

IoU ↑

Higher geometric overlap

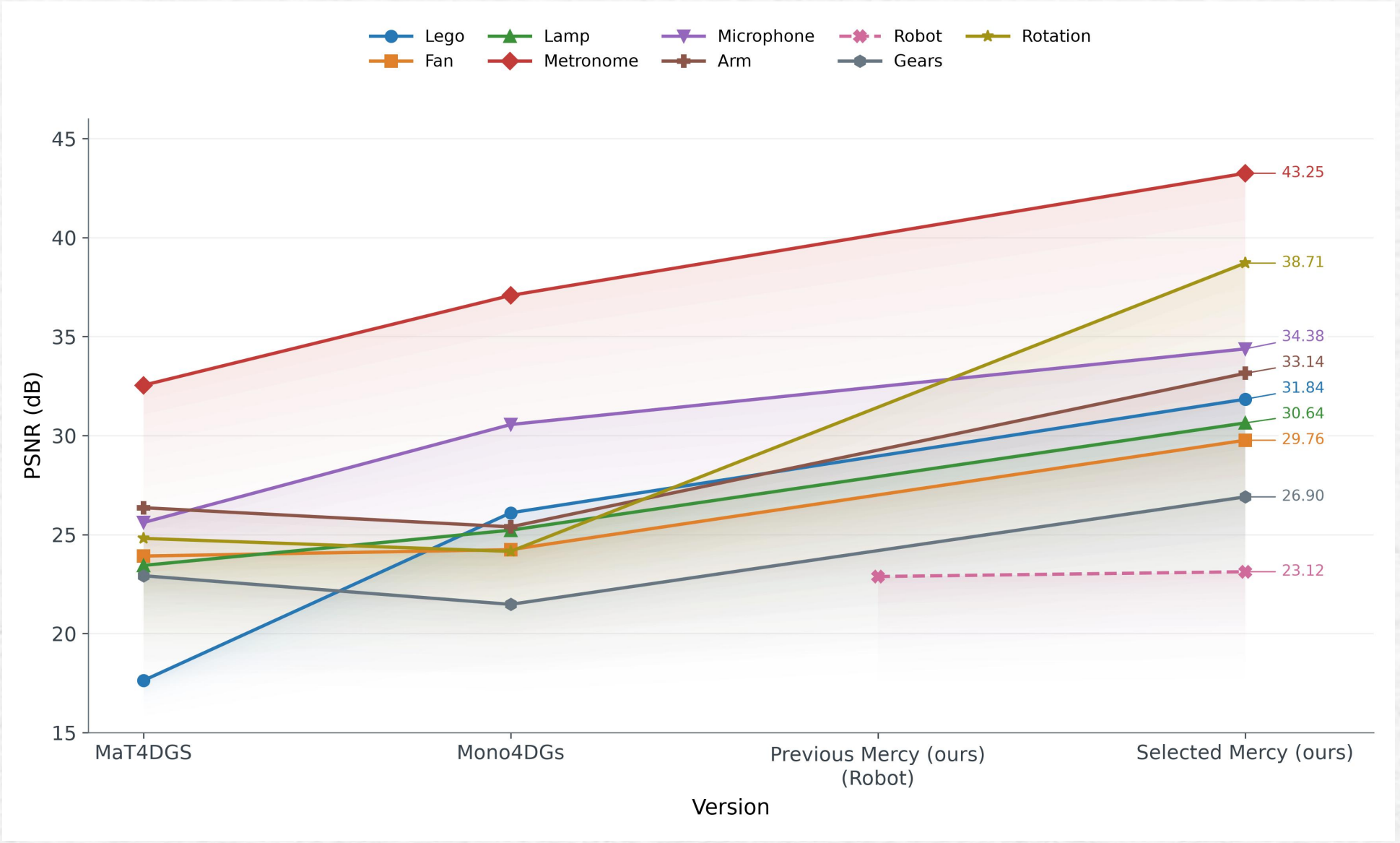
FG PSNR ↑

Higher foreground detail

Missing ↓

Lower geometric missing rate

# ALL 9 Objects Metrics



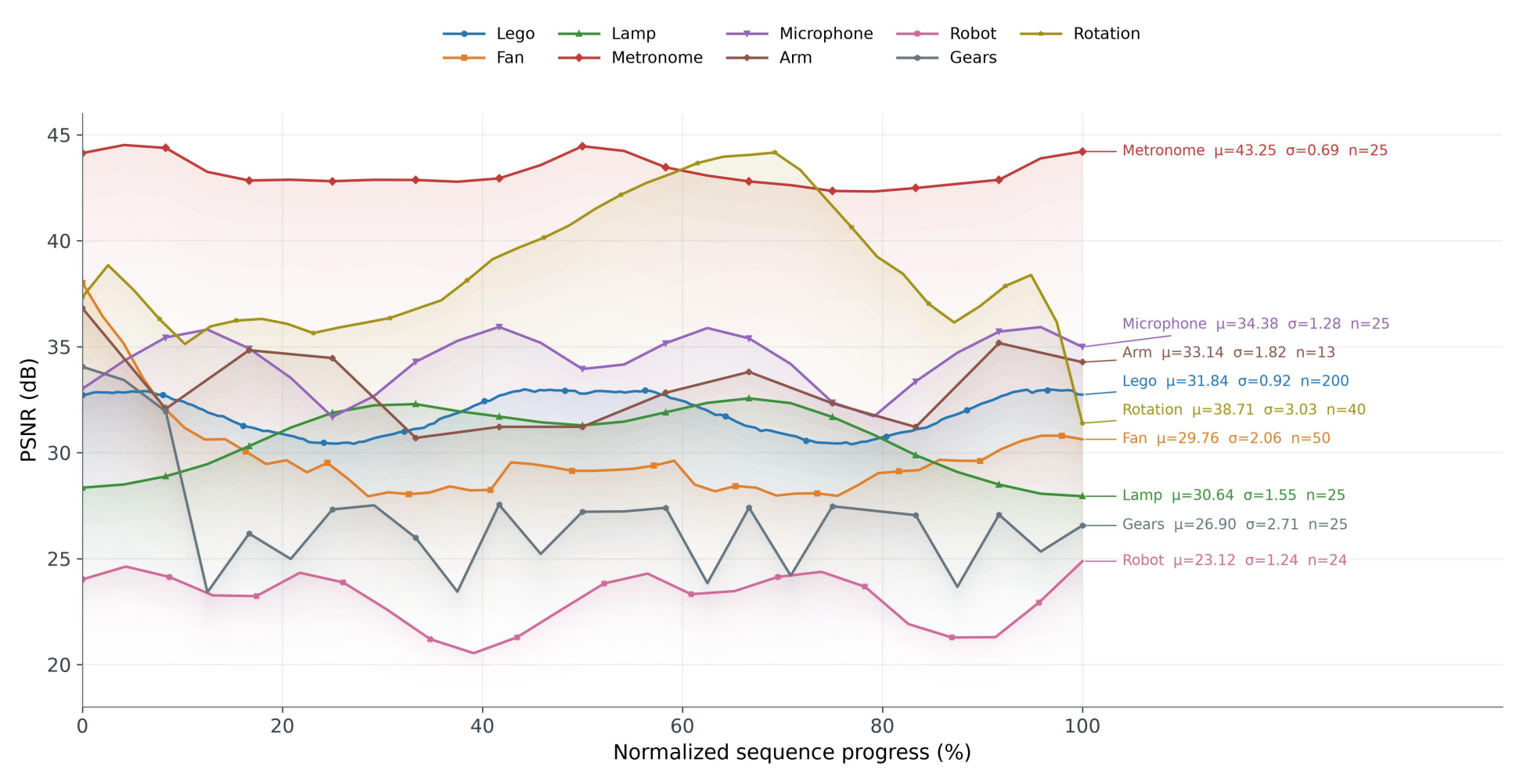

## PSNR ↑

Mercy achieving highest PSNR across all

## Stability

Consistent Cross-object Improvement ↑

# ALL 9 Objects Metrics




## High Temporal Stability ↑

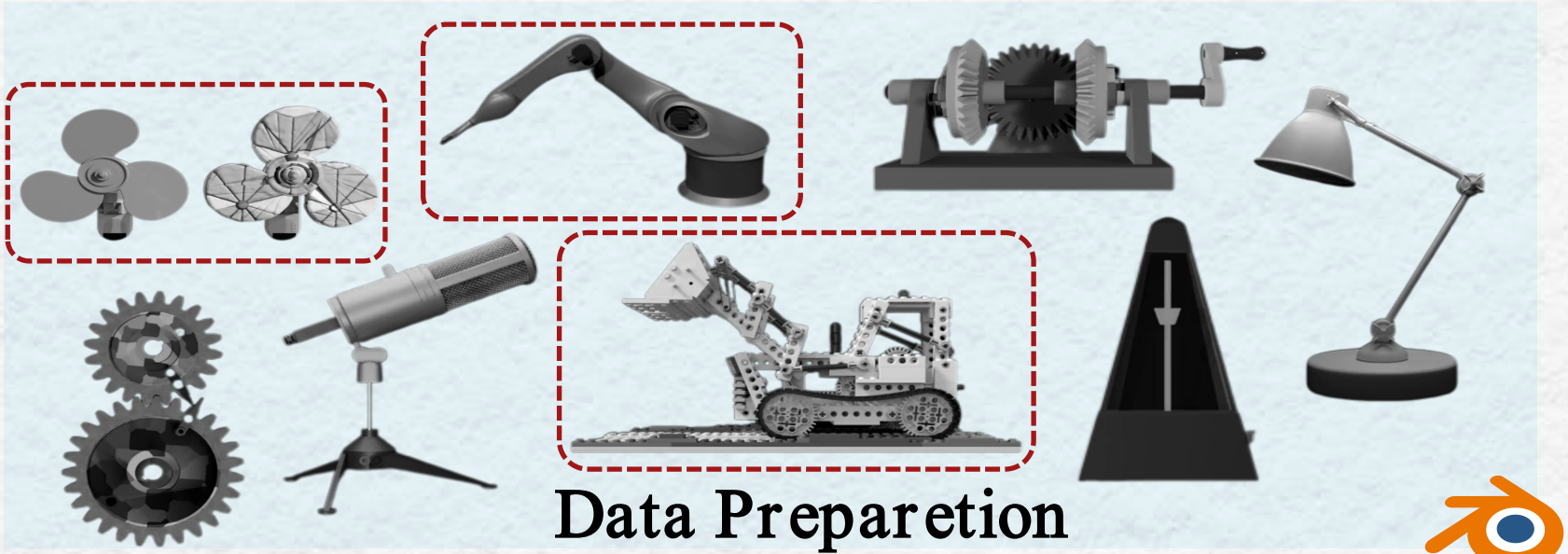
Mercy achieving highest PSNR across all

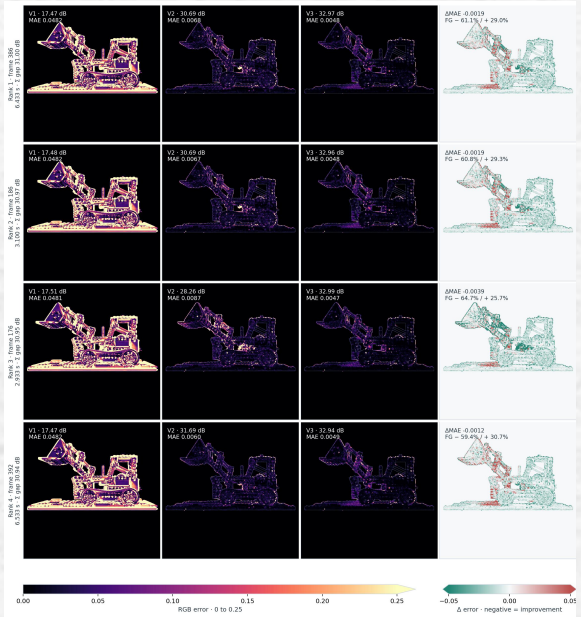
## Consistent

Consistent Cross-frame Reconstruction ↑

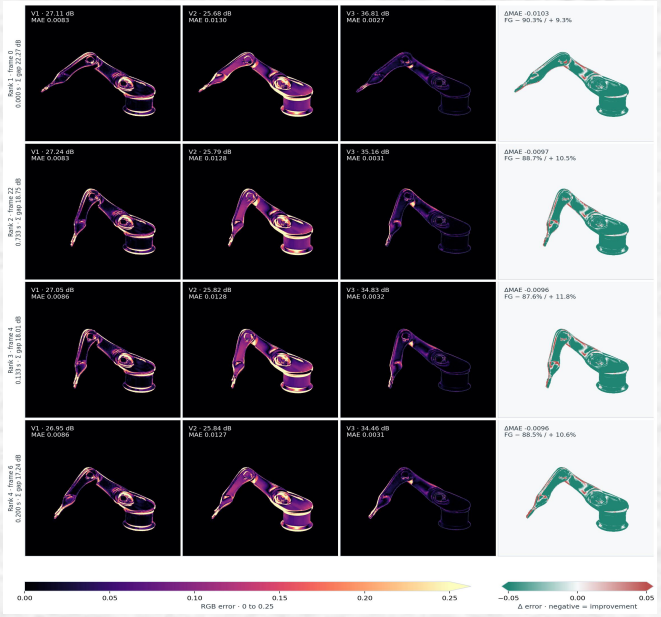
# Look Into Detailed Objects

Sampled 3 independent objects

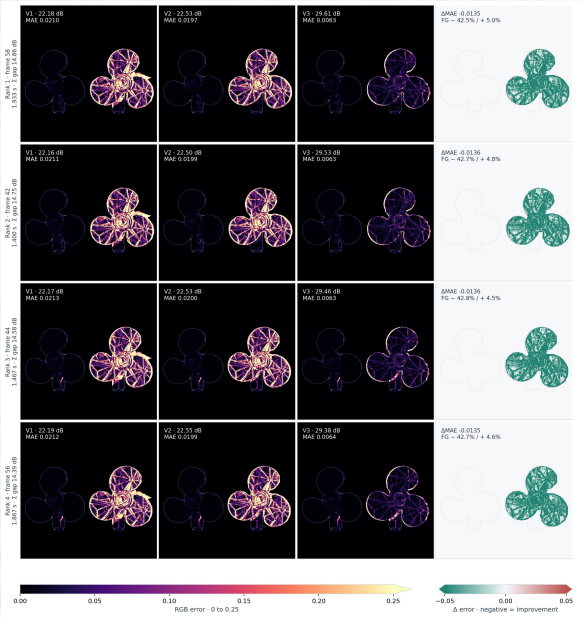


LEGO (Crane)



Robotic Arm



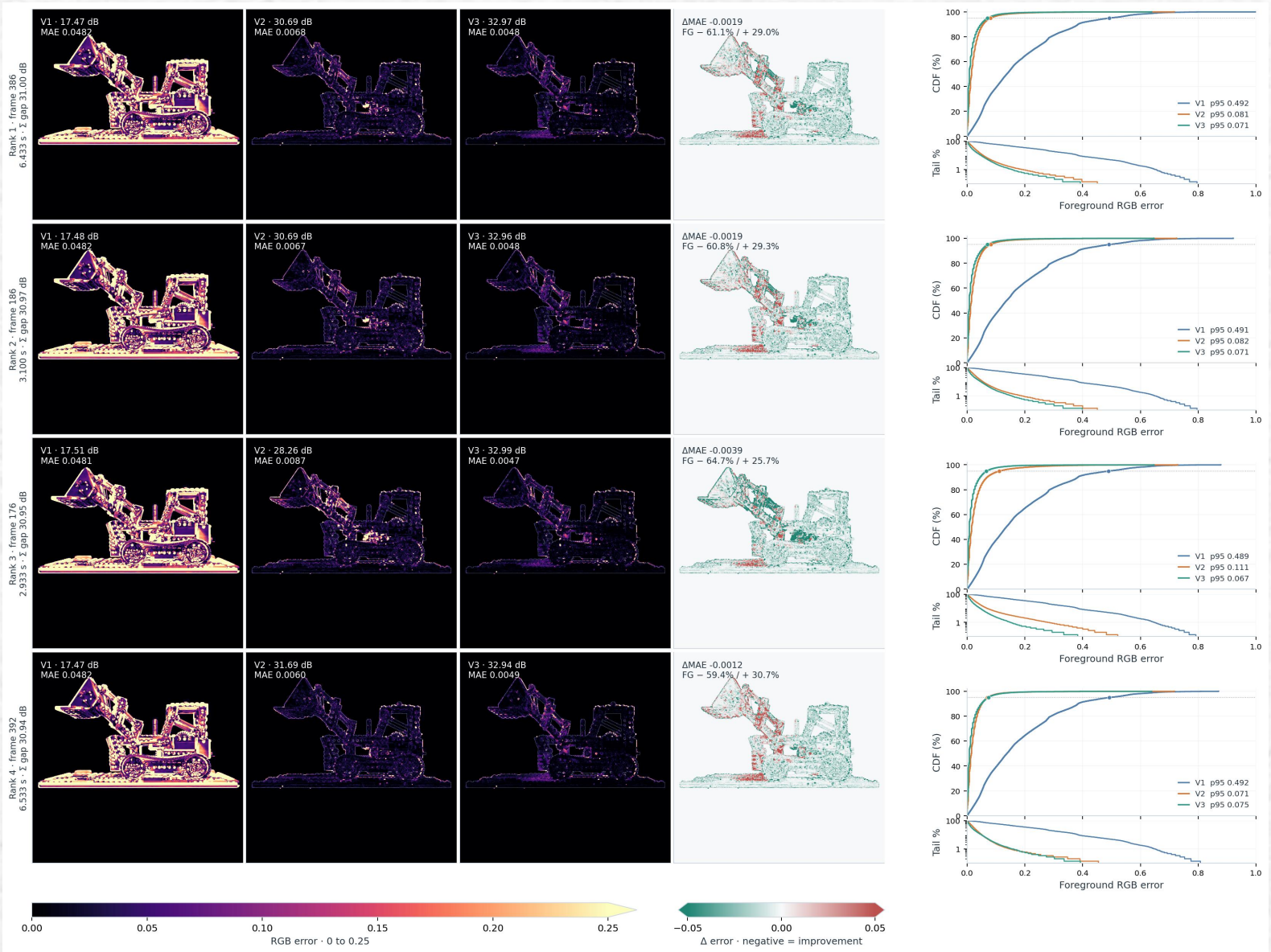
Fan

# LEGO (crane)

Reduced residual intensity & fewer error regions

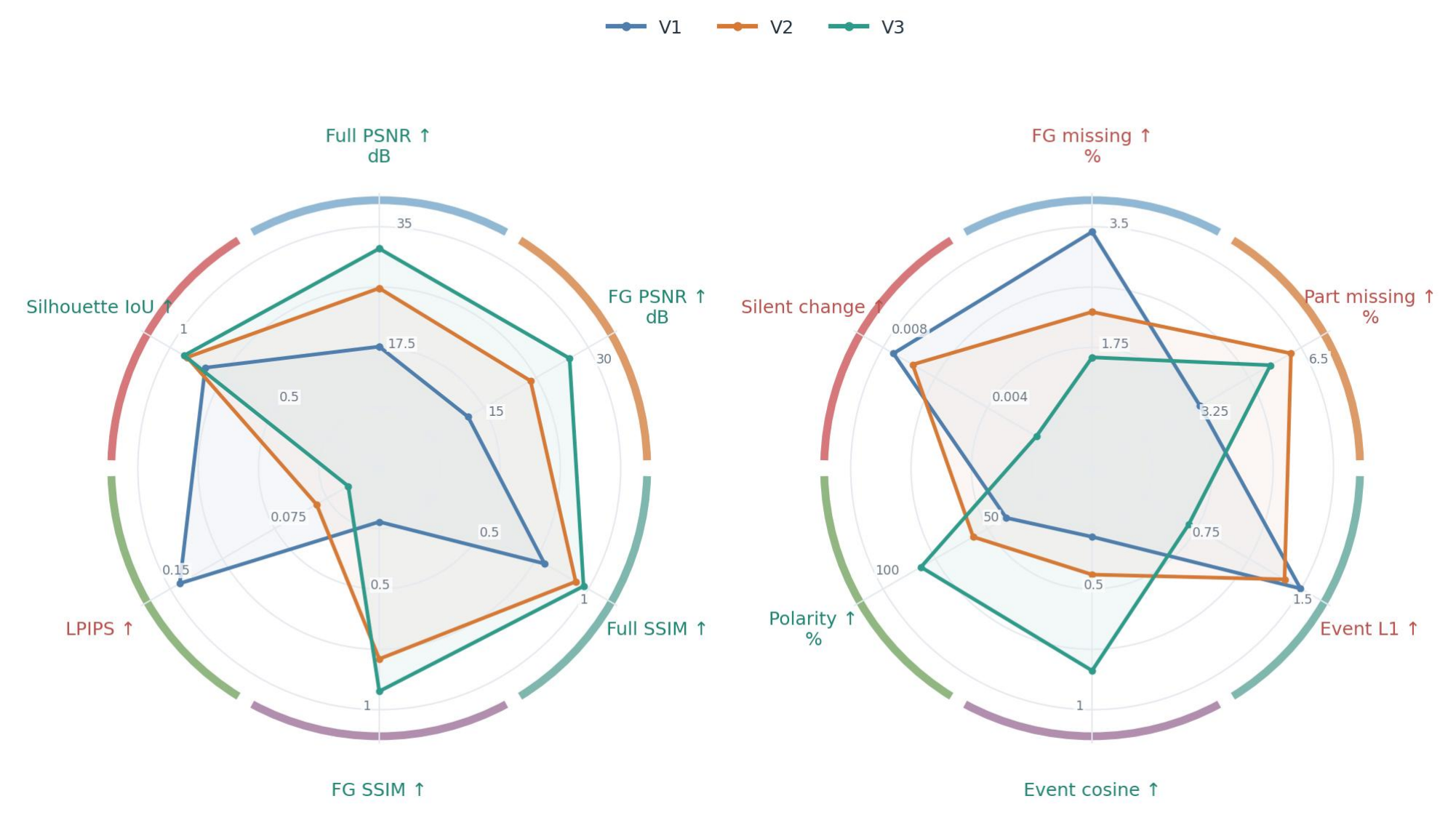
MaTGS MonoGS Mercy

Improved Spatial Consistency ↑



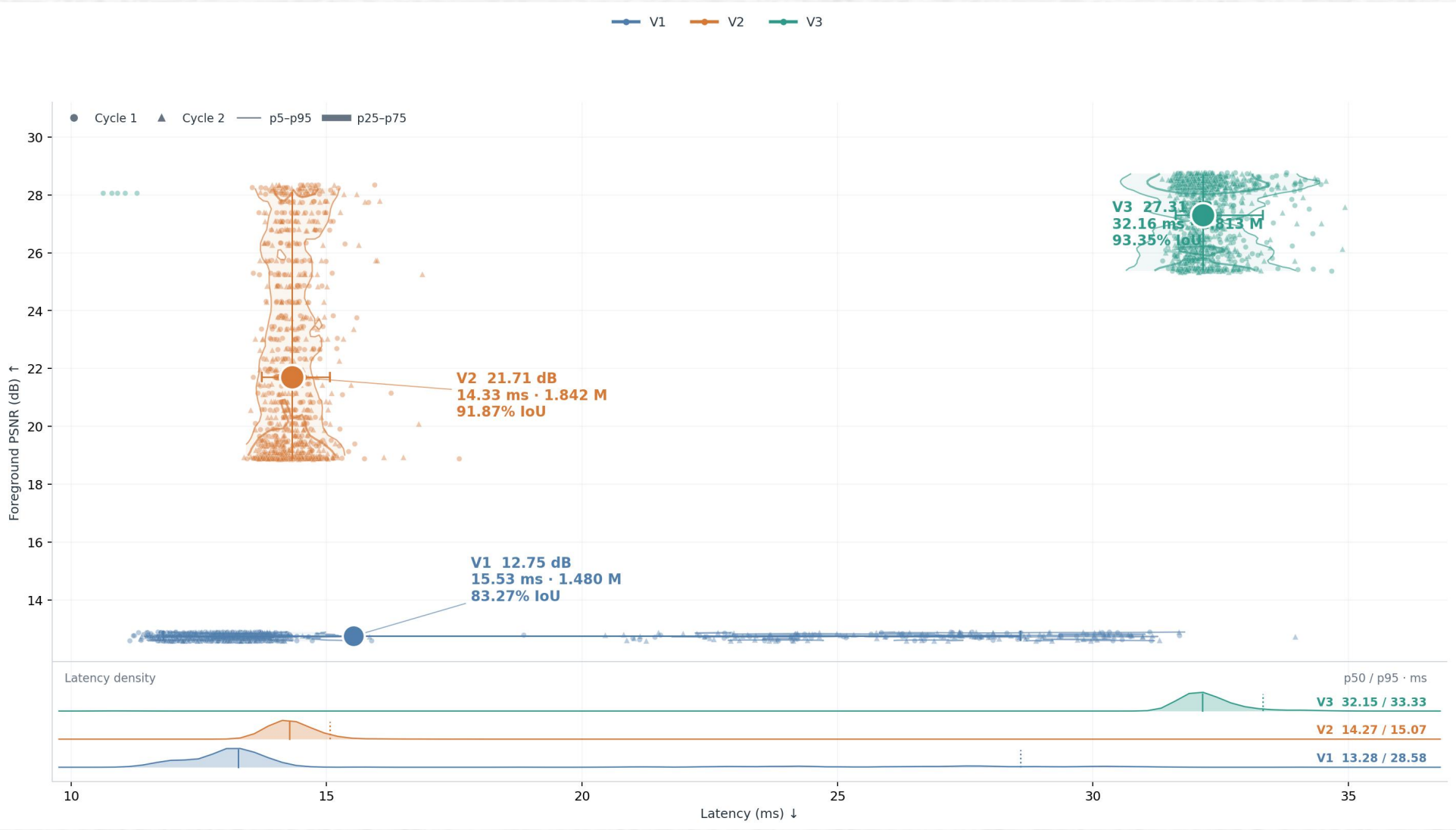
# LEGO (crane)

V1: MaTGS V2: MonoGS V3: Mercy (our)




Statistics Improvement Radar

# Result & Evaluation: LEGO (crane)




## Foreground Fidelity

V3 achieves the highest PSNR (32.16 dB) and IoU (93.35%)

## Latency

Accuracy Trade-off

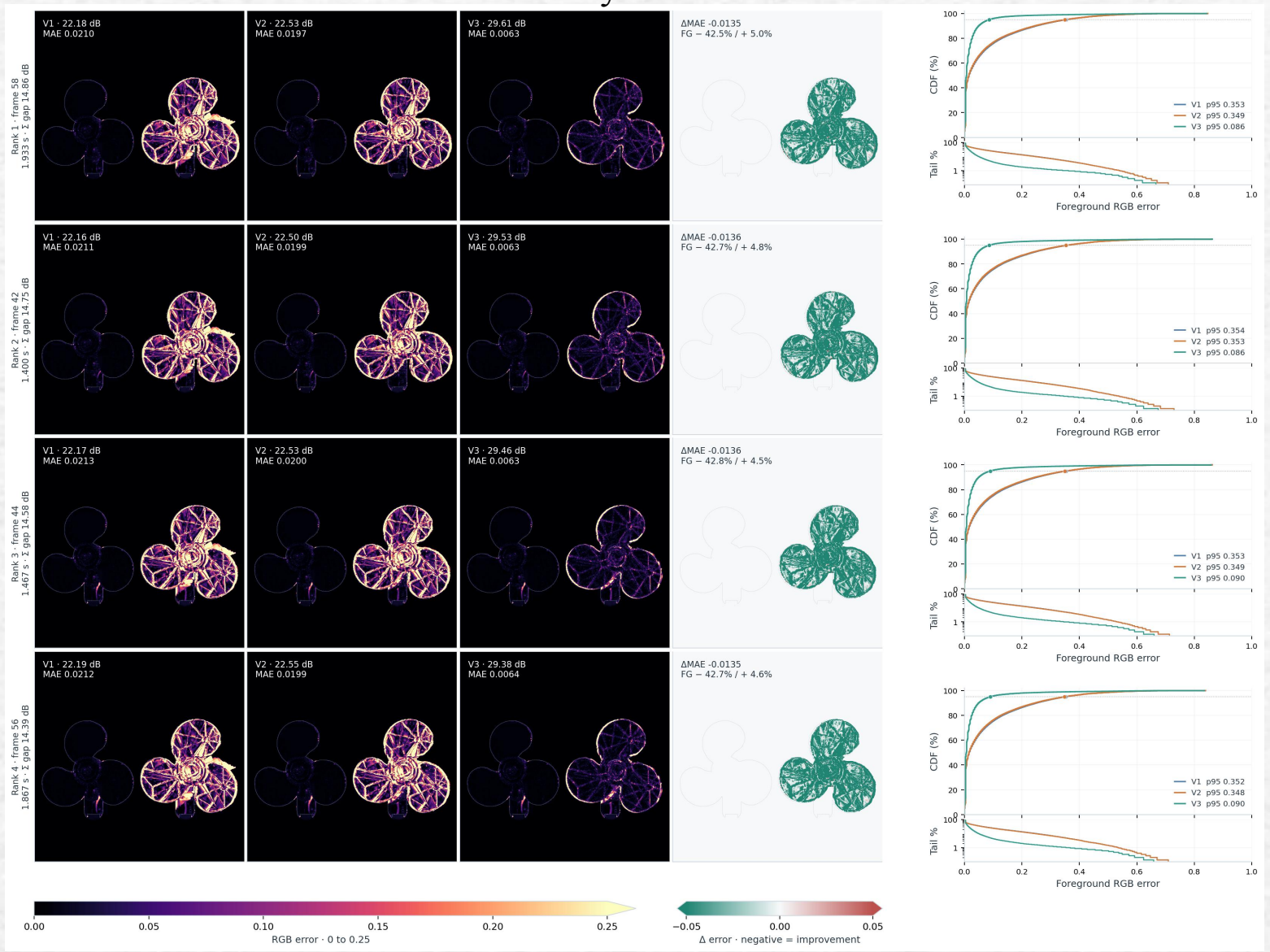
## Geometric Coverage ↑

IoU increases progressively from V1 (83.27%) to V2 (91.87%) and V3 (93.35%)

# Fan (static + dynamic)

Reduced residual intensity & fewer error regions

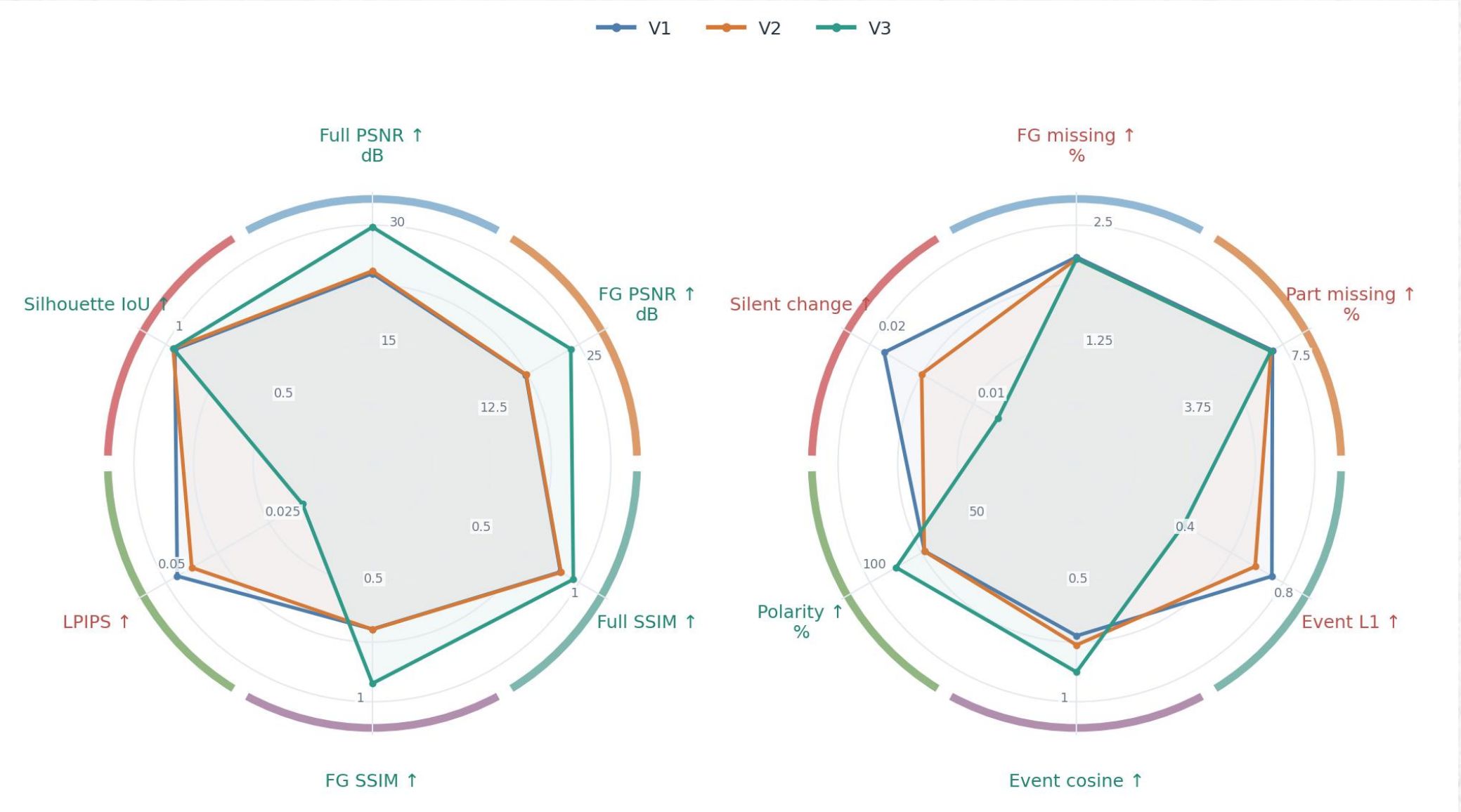
MaTGS MonoGS Mercy




Improved Spatial Consistency ↑

# Fan (static + dynamic)

V1: MaTGS V2: MonoGS V3: Mercy (our)




Statistics Improvement Radar

# Fan (static + dynamic)

## Foreground Fidelity

V3 achieves the highest PSNR (24.02 dB), substantially outperforming V1 and V2.

## Geometric Coverage ↑

V2 and V3 highest IoU (96.57%), stronger foreground alignment.

## Stable Runtime Efficiency ↑

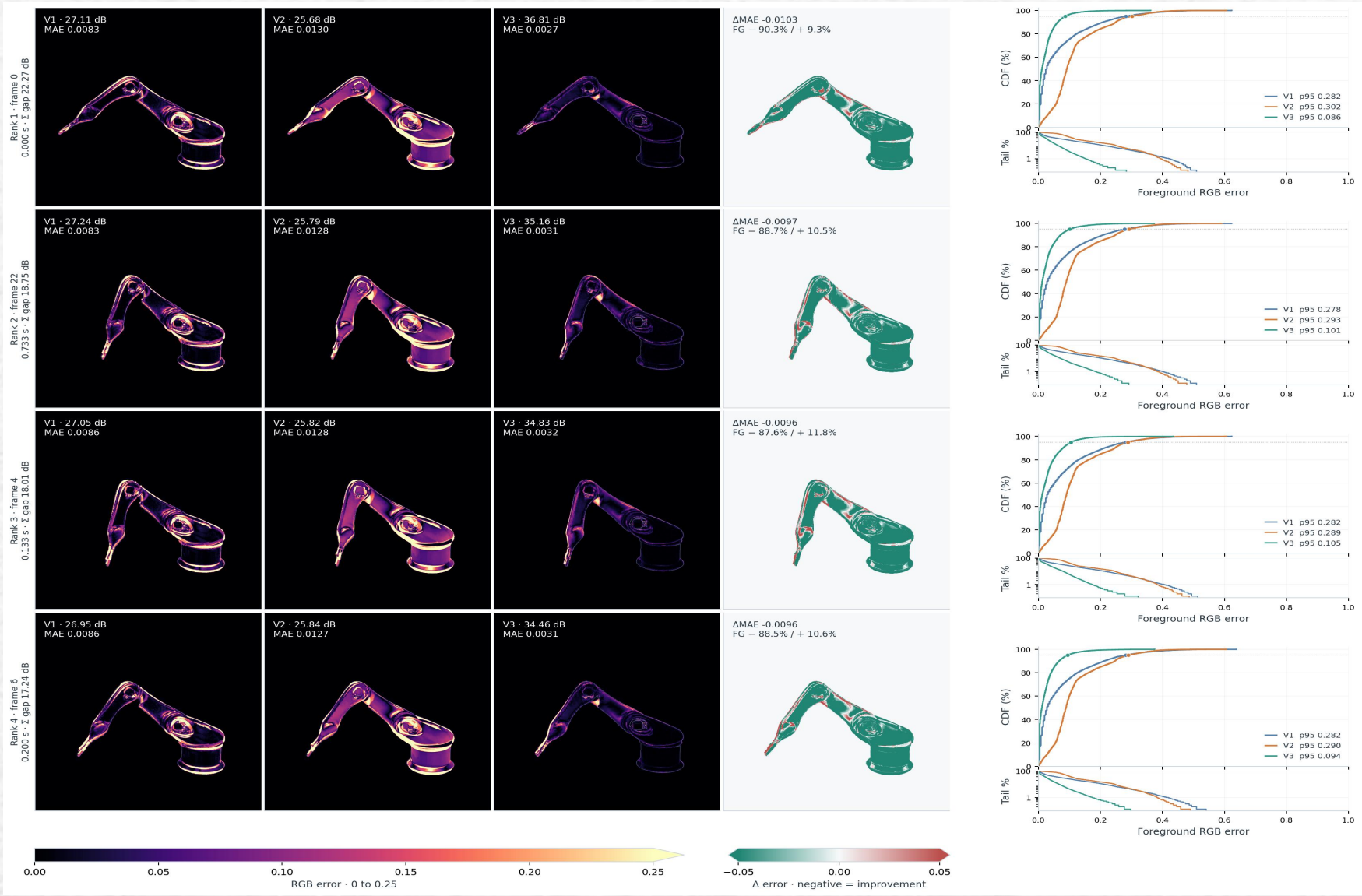
All maintain latency (13.44–13.94 ms) with identical Gaussian counts (0.906M).

# Robotic Arm

Reduced residual intensity & fewer error regions

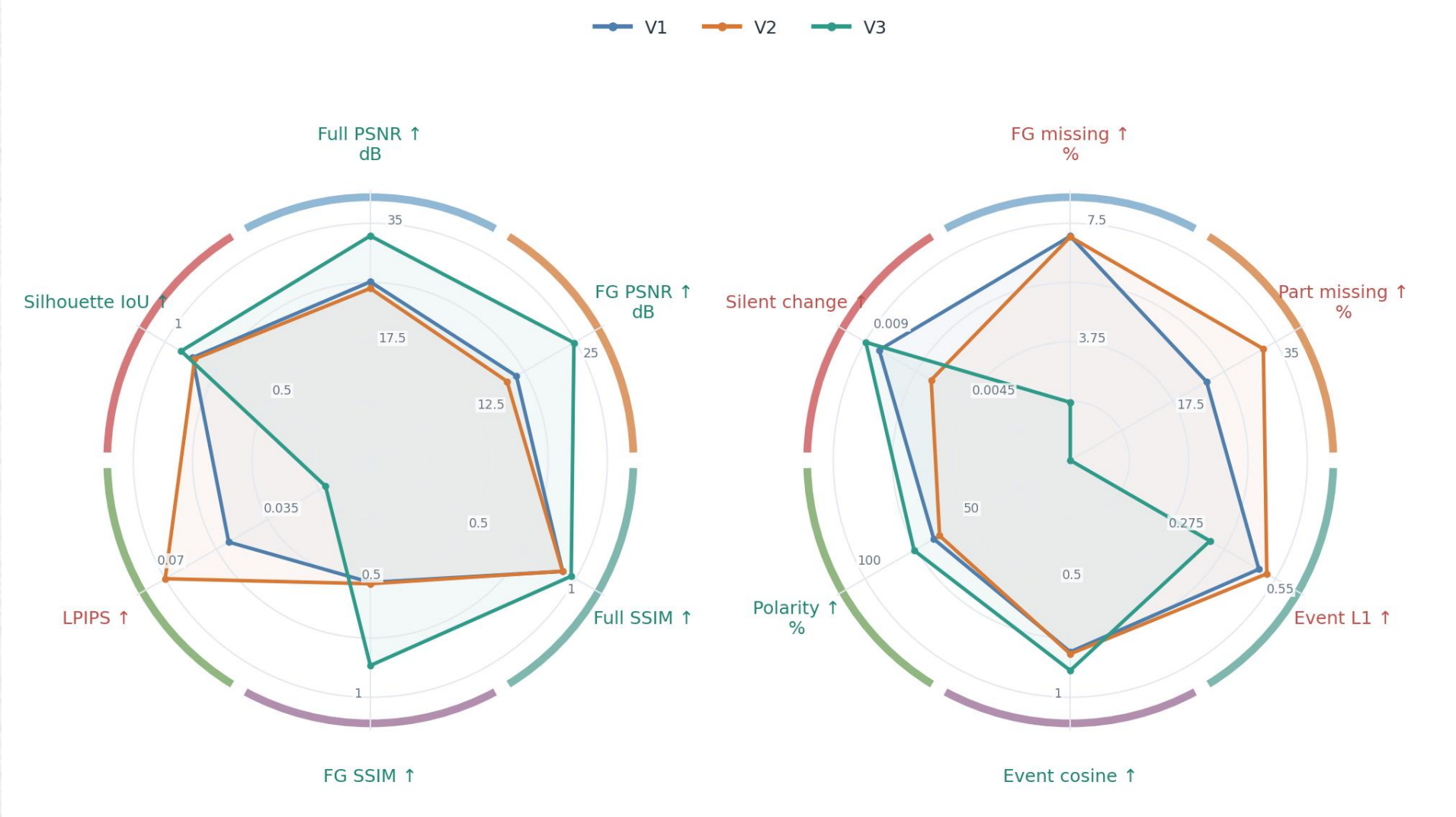
MaTGS MonoGS Mercy

Improved Spatial Consistency ↑



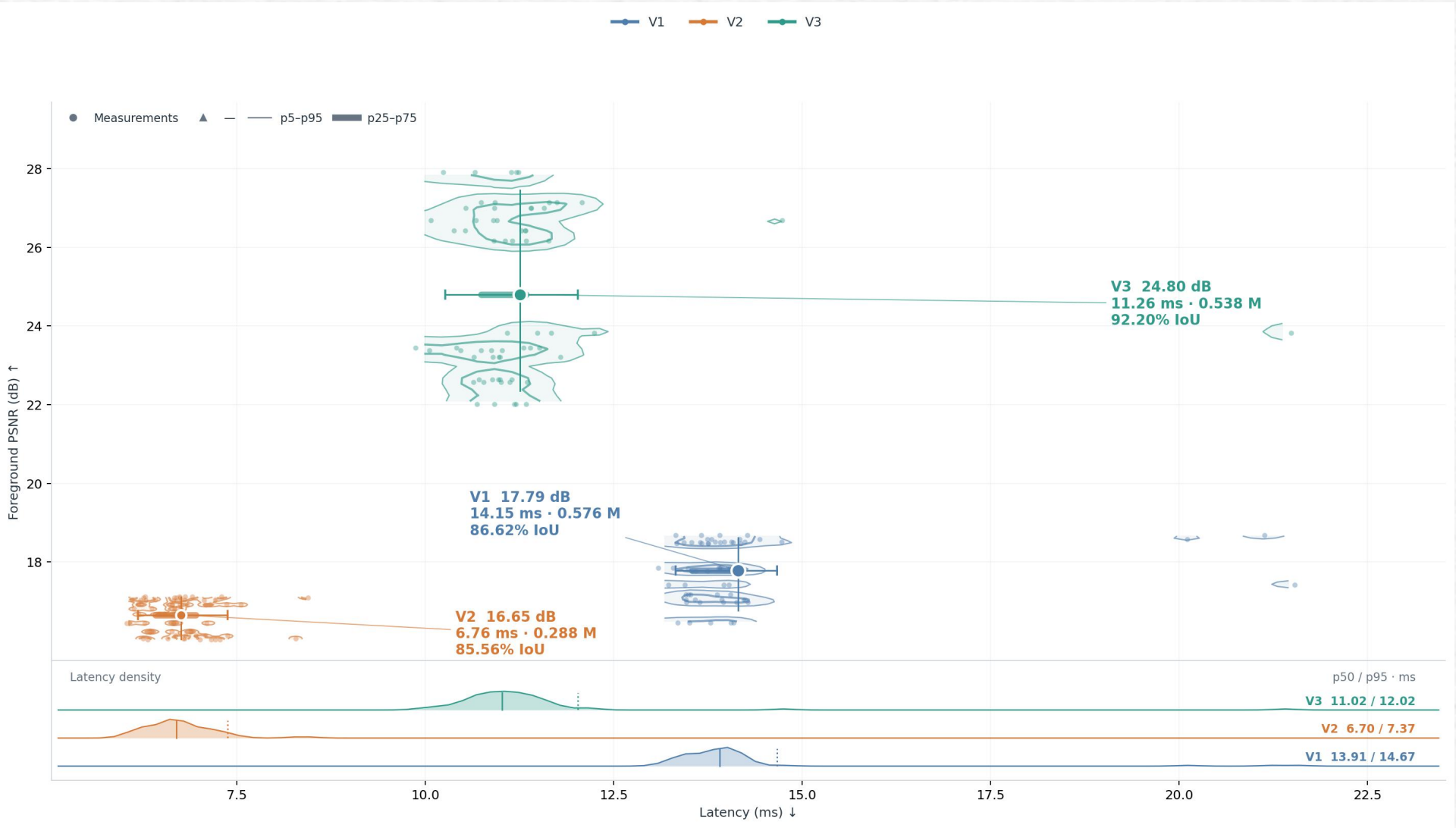
# Robotic Arm

V1: MaTGS V2: MonoGS V3: Mercy (our)




Statistics Improvement Radar

# Result & Evaluation: Robotic Arm




## Foreground Fidelity

V3 achieves the highest PSNR (24.80 dB), substantially outperforming V1 and V2.

## Geometric Accuracy ↑

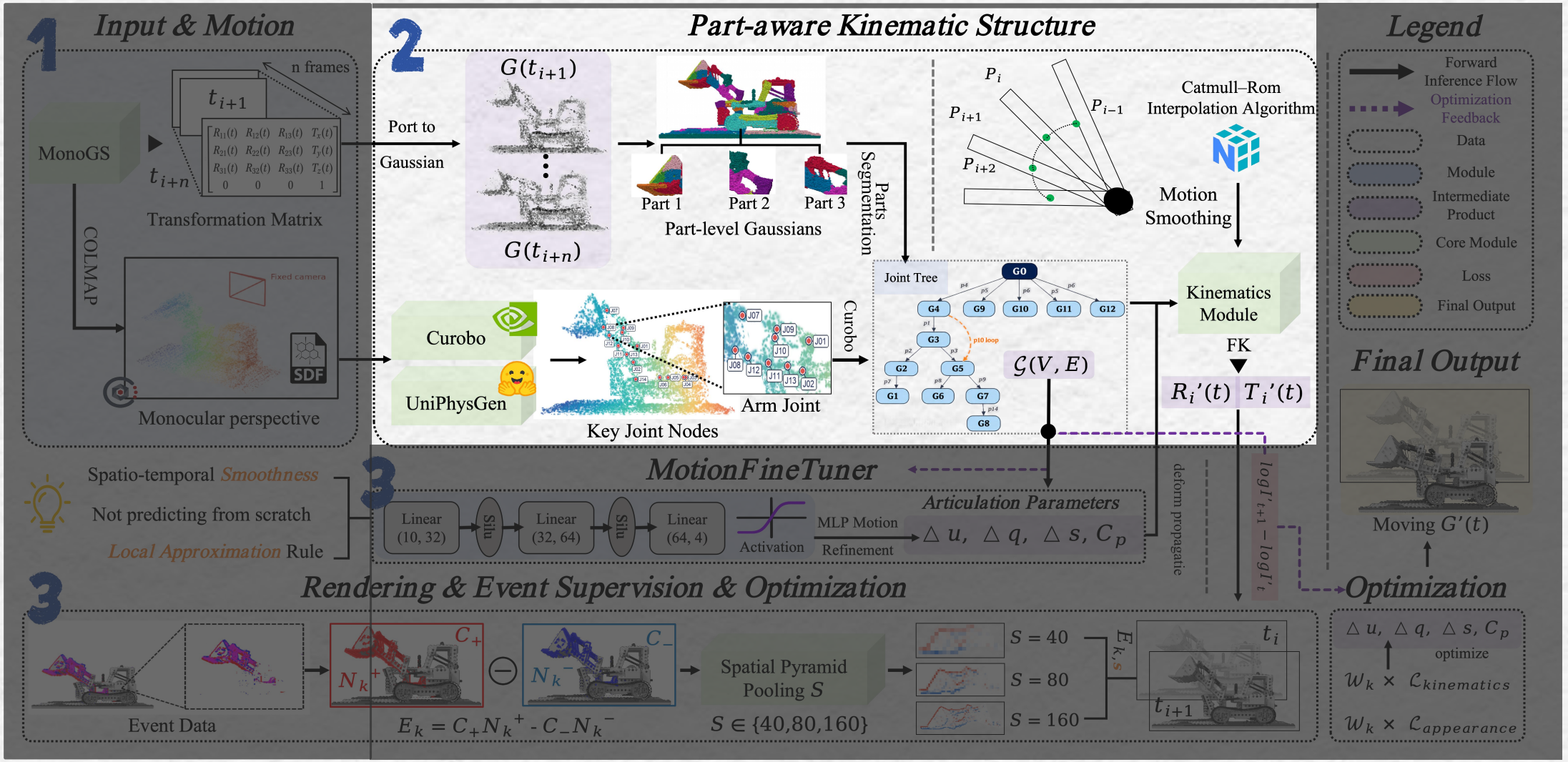
V3 highest IoU (92.20%), better foreground alignment and structural completeness

## Accuracy–Efficiency Trade-off ↑

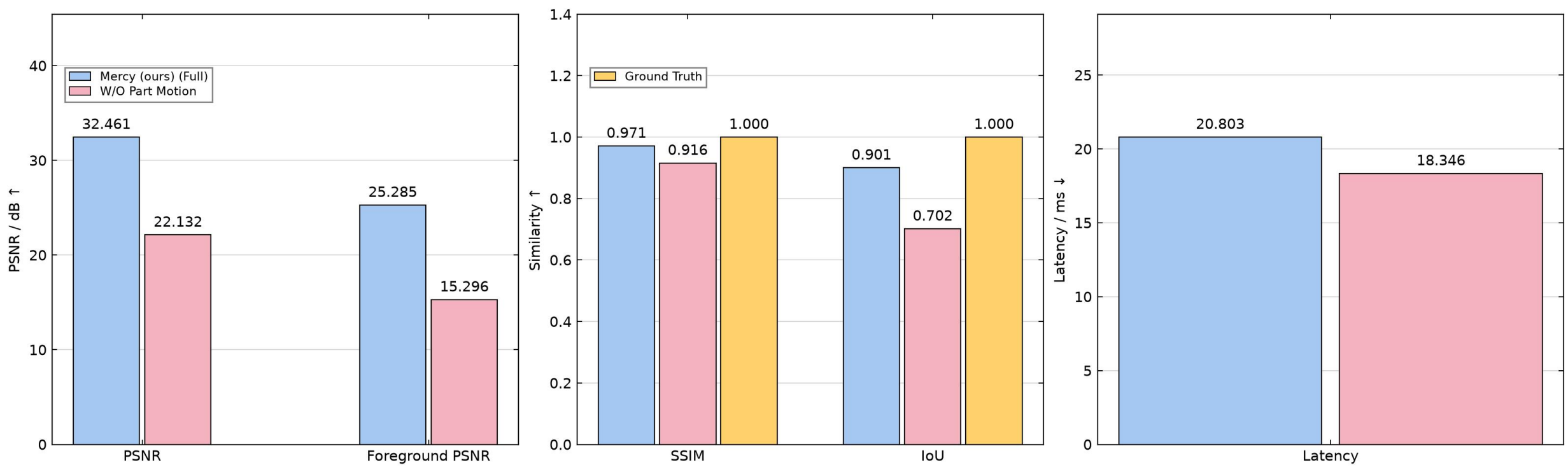
V3 reduces latency to 11.26 ms outperforming V1 in both accuracy and runtime.

**Ablation:** Full (with part motion modeling) **VS.** model without part motion modeling (W/O)

## Part Motion Modeling → Part-aware Kinematic Module



# Ablation 1: Full (with part motion modeling) VS. model without part motion modeling (W/O)



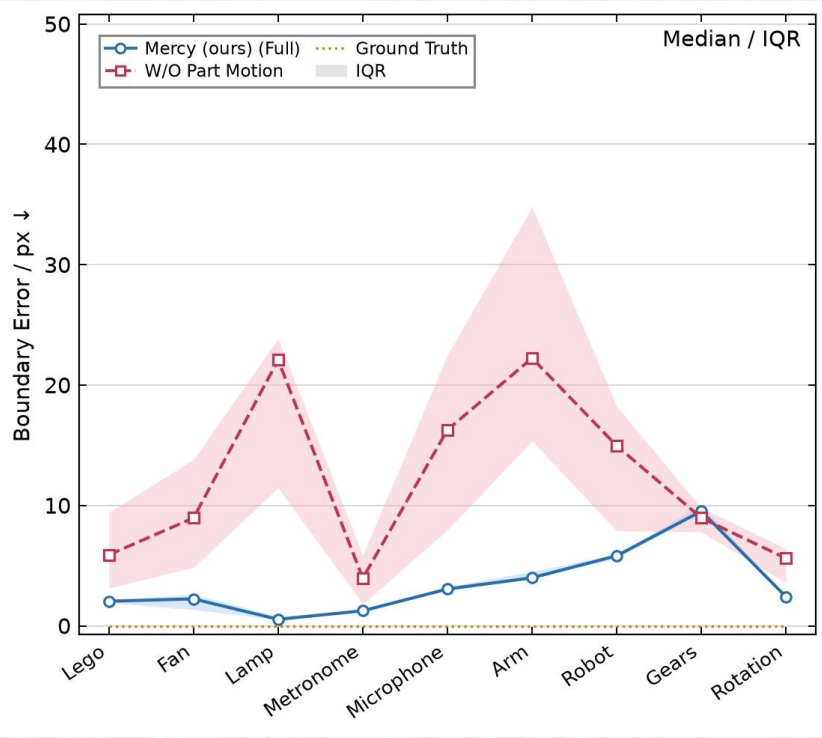

## Reconstruction Fidelity ↑

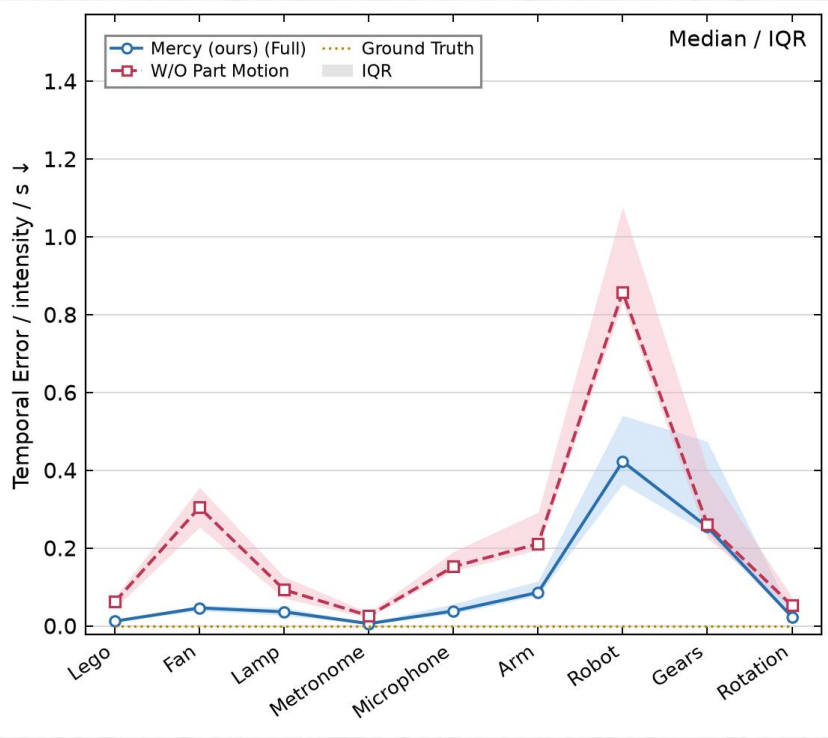
Full Mercy achieves substantially higher PSNR (32.46 vs. 22.13 dB) and Foreground PSNR (25.29 vs. 15.30 dB)

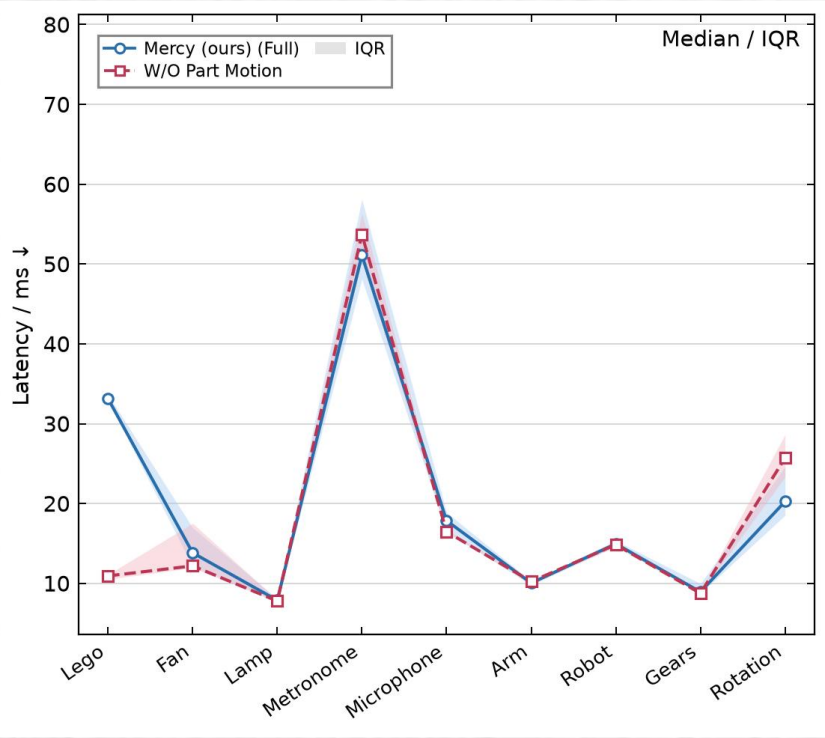
## Structural Consistency ↑

SSIM increases from 0.916 to 0.971, while IoU improves from 0.702 to 0.901, demonstrating stronger geometric alignment

# Ablation 2: Full (with part motion modeling) VS. model without part motion modeling (W/O)




## Boundary Error ↓

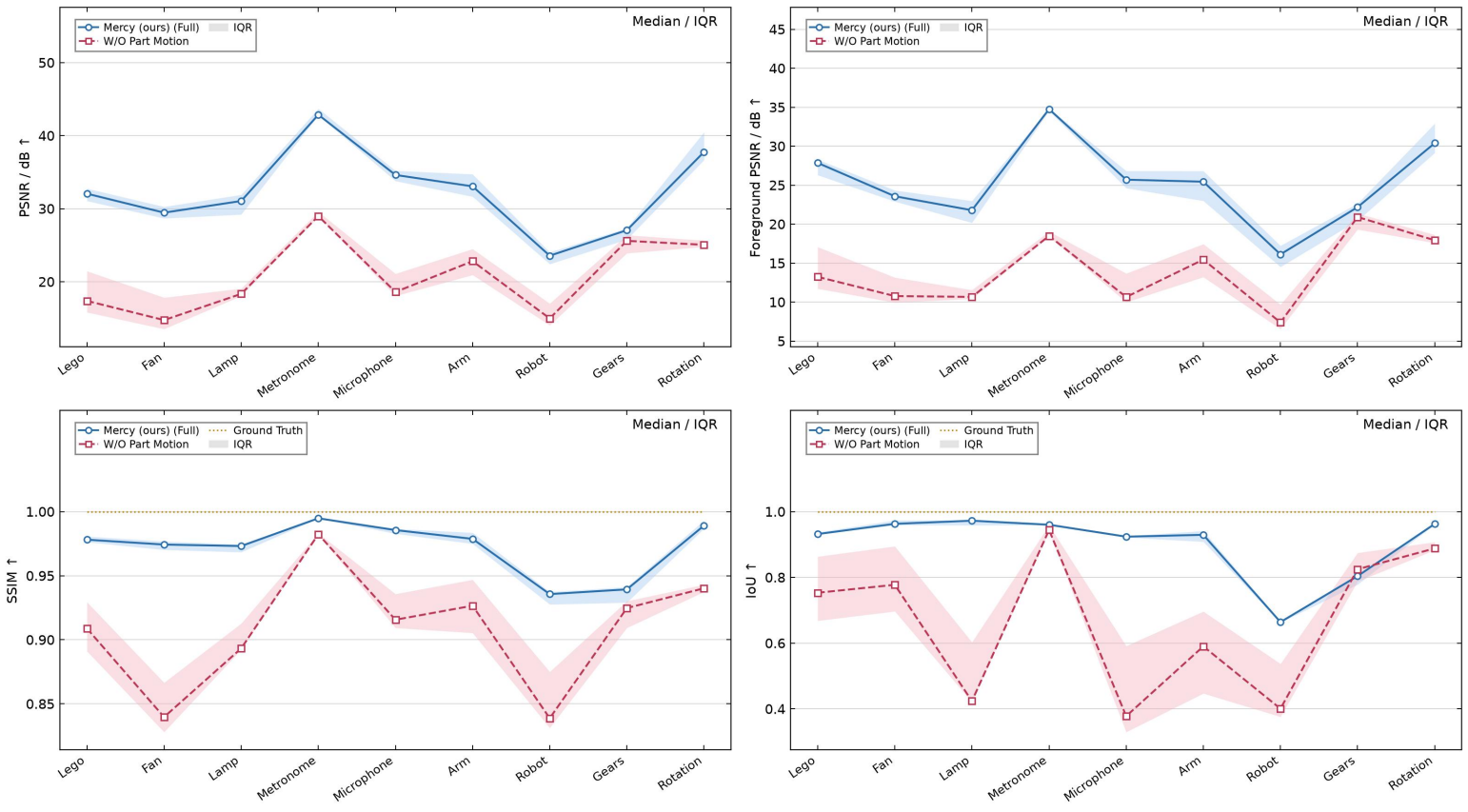
Mercy reduced geometric boundary error across most objects, with particularly improved on Lamp, Microphone, and Arm.

## Temporal Error ↓

Part-level motion modeling consistently lower temporal inconsist, especially on Robot (approximately 0.42 vs. 0.85).

# Ablation 3:

Full (with part motion modeling) **VS.** model without part motion modeling (W/O)




## Reconstruction Fidelity ↑

Mercy higher PSNR and Foreground PSNR, particularly larger gains on Metronome and Robot

## Structural & Geometric Consistency ↑

Higher SSIM and IoU improved structural preservation and part-level alignment, especially on Lamp, Microphone, Arm, Robot.